

UNIVERSIDAD DE ZARAGOZA
ESCUELA DE INGENIERÍA Y ARQUITECTURA

Grado en Ingeniería Informática



**Escuela de
Ingeniería y Arquitectura**
Universidad Zaragoza

TRABAJO DE FIN DE GRADO

**ANÁLISIS COMPARATIVO DE PROTECCIONES DE SEGURIDAD Y
SU INFLUENCIA EN LA EFICIENCIA DE PROGRAMAS
COMPILADOS**

Autor

Javier Julve Yubero

Director

Ricardo Julio Rodríguez Fernández

Junio de 2025

Curso académico 2024/2025

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Estructura del documento	2
2. Fundamentos teóricos	3
2.1. Medidas de seguridad evaluadas	3
2.2. Medidas exclusivas de Clang	4
2.3. Medidas exclusivas de Rustc	4
2.4. Relación con vulnerabilidades comunes	5
3. Metodología	7
3.1. Descripción general del proceso	7
3.2. Programas utilizados	8
3.3. Enfoque visual y tipo de gráficas	8
4. Análisis de resultados	10
4.1. Entorno de experimentación	10
4.2. Descripción de conjuntos de datos	11
4.2.1. Programas utilizados	11
4.2.2. Justificación del diseño	11
4.2.3. Compatibilidad	11
4.3. Descripción de experimentos y resultados	12
4.3.1. Diseño del experimento	12

4.3.2. Combinaciones	12
4.4. Análisis de resultados	15
4.4.1. Tiempo de ejecución	15
4.4.2. Uso de memoria	18
4.4.3. Tamaño del binario	21
4.4.4. Funciones fortificadas	23
5. Trabajo relacionado	24
6. Conclusiones y trabajo futuro	25
6.1. Conclusiones	25
6.1.1. Equilibrio entre seguridad y rendimiento	25
6.1.2. Flags con penalizaciones elevadas	26
6.1.3. Rust y su diseño	26
6.1.4. nconsistencias entre compiladores	26
6.1.5. Recomendaciones generales	27
6.2. Trabajo futuro	27
Bibliografía	29
Anexos	31

Agradecimientos

A Ricardo por darme esta oportunidad.

A mis padres.

A mi novia.

Resumen

En los últimos años, la seguridad en el desarrollo de software ha adquirido una relevancia crítica debido al aumento de vulnerabilidades y ataques contra sistemas informáticos. Como respuesta, los compiladores modernos han incorporado mecanismos de protección que refuerzan la integridad y robustez de los programas generados. Sin embargo, estas medidas introducen un coste en términos de eficiencia, afectando al tiempo de ejecución, el uso de memoria y el rendimiento general del software.

Este Trabajo de Fin de Grado realiza un análisis comparativo del impacto que tienen las medidas de seguridad aplicadas durante la compilación sobre la eficiencia de los programas generados. Para ello, se han evaluado diversos compiladores ampliamente utilizados, como GCC, Clang y Rustc, combinando distintas opciones de seguridad con niveles de optimización. Durante la experimentación, se han recogido métricas clave como el tiempo de ejecución, el uso de memoria, el tamaño del binario resultante y la presencia de funciones fortificadas, que reemplazan fragmentos de código potencialmente peligrosos por versiones más seguras.

Este trabajo proporciona una visión clara y práctica sobre el coste asociado a la seguridad en la compilación de programas, ofreciendo recomendaciones específicas para distintos escenarios, desde aplicaciones críticas hasta entornos donde el rendimiento es la prioridad. Los hallazgos contribuyen a fundamentar decisiones técnicas en proyectos donde ambos factores, seguridad y eficiencia, son determinantes.

Abstract

In recent years, security in software development has gained critical importance due to the rise in vulnerabilities and attacks on computer systems. In response, modern compilers have incorporated protection mechanisms that enhance the integrity and robustness of the generated programs. However, these measures come at a cost in terms of efficiency, affecting runtime, memory usage, and overall software performance.

This Bachelor's Thesis presents a comparative analysis of the impact that security measures applied during compilation have on the efficiency of the resulting programs. To achieve this, widely used compilers such as GCC, Clang, and Rustc have been evaluated, combining different security options with various optimization levels. During experimentation, key metrics such as execution time, memory usage, resulting binary size, and the presence of fortified functions—which replace potentially dangerous code segments with safer versions—were collected.

This work provides a clear and practical perspective on the cost associated with security in program compilation, offering specific recommendations for different scenarios, ranging from critical applications to environments where performance is the priority. The findings help support technical decisions in projects where both security and efficiency are crucial factors.

Capítulo 1

Introducción

1.1. Motivación

En los últimos años, la preocupación por la seguridad en el desarrollo de software ha crecido notablemente. Como respuesta, los compiladores modernos han incorporado mecanismos de protección que refuerzan la integridad y robustez del programa generado.

Estas medidas, sin embargo, introducen un coste en términos de eficiencia. Se observa un impacto en el tiempo de ejecución, en el uso de memoria y, en general, en el rendimiento del programa.

Por ello, resulta de interés analizar en qué medida estas protecciones afectan al comportamiento de los programas, especialmente cuando se combinan con distintos niveles de optimización. En este trabajo se estudia el efecto de activar *flags* de seguridad junto con *flags* de optimización durante el proceso de compilación, observando cómo esta combinación influye en el rendimiento final.

El análisis se realiza evaluando diferentes compiladores, lo que permite identificar posibles diferencias en las estrategias utilizadas por estos compiladores para aplicar estas medidas. Este enfoque busca comprender con mayor precisión el equilibrio entre seguridad y eficiencia, aportando datos que ayuden a fundamentar decisiones técnicas en entornos donde ambos factores son críticos.

1.2. Objetivos

El objetivo principal de este trabajo es realizar un análisis comparativo del impacto que tienen ciertas medidas de seguridad aplicadas durante la compilación sobre la eficiencia de ejecución de los programas generados. Para ello, se han evaluado compiladores de uso común, como *GCC*, *Clang* y *Rustc*. Se han diseñado pruebas en las que se combinan distintas medidas de seguridad con niveles de optimización.

Durante la experimentación, se recogen métricas como el tiempo de ejecución, el uso de

memoria, el tamaño del programa generado y la presencia de funciones fortificadas (son aquellas que sustituyen fragmentos de código potencialmente peligrosos por versiones más seguras con comprobaciones adicionales). Estas métricas permiten evaluar el efecto real de la combinación entre seguridad y optimización en cada entorno de compilación.

Con los resultados obtenidos, se pretende entender cómo responde cada compilador ante estas configuraciones y en qué medida se ve afectado el rendimiento. Se busca así ofrecer una visión clara y práctica sobre el coste asociado a la seguridad en el contexto de la compilación de programas.

1.3. Estructura del documento

Este documento se encuentra dividido en cuatro capítulos y un anexo. En el [Capítulo 2](#) se presentan los conceptos previos necesarios para comprender el análisis, con especial atención a las protecciones de seguridad aplicadas a nivel de compilador. En el [Capítulo 3](#) se describe el *pipeline* automatizado desarrollado para realizar los experimentos: desde la entrada (código fuente y combinaciones de *flags*), pasando por el proceso (compilación, ejecución, recolección de métricas), hasta la generación automática de informes y gráficas. También se justifican los programas utilizados y el enfoque visual adoptado. En el [Capítulo 4](#) se detallan el entorno de experimentación, los conjuntos de datos utilizados y los resultados obtenidos, incluyendo una discusión crítica sobre las observaciones realizadas. En el [Capítulo 5](#) se detallan los trabajos similares al que se ha realizado. Finalmente en el [Capítulo 6](#) se exponen las conclusiones obtenidas tras la realización de este trabajo, así como posibles líneas de trabajo futuro que pueden continuar y ampliar el estudio aquí presentado. Adicionalmente en el [Anexo](#) se incluye el diagrama de Gantt con la distribución temporal del trabajo y la colección completa de gráficas generadas, que no han sido incorporadas en el cuerpo principal del documento.

Capítulo 2

Fundamentos teóricos

La seguridad en el desarrollo de software depende no solo de la lógica de programación, sino también de las decisiones adoptadas durante la compilación. En este capítulo se describen las principales medidas de seguridad ofrecidas por distintos compiladores y cómo contribuyen a mitigar vulnerabilidades comunes. Se presentan primero las técnicas generales, disponibles en varios compiladores, y luego se detallan aquellas específicas de Clang y Rustc. Estas medidas constituyen la base teórica sobre la que se estructura el análisis de este trabajo.

2.1. Medidas de seguridad evaluadas

En este trabajo se analizan distintas opciones de compilación que permiten reforzar la seguridad de los programas. A continuación, se describen de forma general las principales medidas evaluadas.

1. **Ejecución independiente de la posición (*PIE*)**. Permite que el programa se cargue en una dirección de memoria distinta cada vez que se ejecuta, lo que dificulta ciertos ataques basados en direcciones fijas de memoria. Esta característica se puede activar o desactivar mediante opciones específicas del compilador.[1]
2. **Protección contra errores en la pila (*stack protector*)**. Añade medidas de seguridad para evitar que se modifique accidentalmente o de forma malintencionada la información que una función guarda temporalmente en la pila. Se puede ajustar el nivel de protección según las necesidades o incluso desactivarla si no se considera oportuno.[2]
3. **Fortificación de funciones estándar**. Reemplaza funciones comunes de la biblioteca estándar (como copiar cadenas de texto) por versiones más seguras que realizan comprobaciones adicionales para evitar errores como por ejemplo desbordamientos. Se puede ajustar el nivel de fortificación o desactivarla por completo.
4. **Pila no ejecutable**. Evita que la parte de memoria destinada a la pila pueda ejecutarse. Esta medida impide que un atacante inserte y ejecute código mali-

cioso colocado en esa zona. También puede activarse o desactivarse durante la compilación.

5. **Protección de la tabla de enlaces (*RELRO*)**. Refuerza ciertas estructuras internas que se utilizan durante la carga del programa en memoria. Estas estructuras son a veces objetivo de ataques y esta opción evita que puedan ser modificadas una vez cargadas.[3]
6. **Sanitizadores**. Son herramientas que permiten detectar distintos tipos de errores durante la ejecución del programa. Entre los utilizados en este trabajo están:
 - **AddressSanitizer**: identifica accesos incorrectos a memoria.[4]
 - **UndefinedBehaviorSanitizer**: detecta usos incorrectos del lenguaje que pueden provocar fallos difíciles de localizar.[5]
 - **MemorySanitizer**: avisa cuando se usa memoria sin haberla inicializado.[6]
 - **LeakSanitizer**: detecta si el programa pierde memoria sin liberarla (utilizado solo en Rust).[7]
7. **Eliminación de redireccionamientos en llamadas a funciones externas (*PLT*)**. Impide el uso de ciertas estructuras que facilitan la ejecución de funciones compartidas. Aunque puede mejorar el rendimiento, desactivarlas incrementa la seguridad ante ataques que modifican estos enlaces.

Además de las opciones comunes descritas anteriormente, algunos compiladores incorporan medidas de seguridad particulares que también se han evaluado en este trabajo. Estas medidas se exponen a continuación.

2.2. Medidas exclusivas de Clang

- **Pila segura (*Safe Stack*)**. Separa las variables locales sensibles del resto de la memoria de trabajo de una función, almacenándolas en una zona aislada y protegida. Esto reduce las posibilidades de que un atacante las manipule aprovechando errores como por ejemplo desbordamientos.[8]
- **Optimización en tiempo de enlace (*Link-Time Optimization*, LTO)**. Aunque no es una medida de seguridad directa, permite que el compilador analice y optimice el código completo al enlazar. Esto puede influir en cómo se aplican otras protecciones y en el comportamiento final del ejecutable.

2.3. Medidas exclusivas de Rustc

- **Comprobación de desbordamientos aritméticos**. Verifica que las operaciones con enteros no sobrepasen los límites del tipo de dato. Si ocurre un desbordamiento, el programa lo detecta y lo informa.[9]

- **Aserciones de depuración.** Introduce comprobaciones adicionales durante la ejecución para ayudar a detectar errores lógicos o condiciones inesperadas en tiempo de desarrollo.
- **Gestión de errores en tiempo de ejecución (*panic*).** Permite configurar el comportamiento del programa ante fallos graves: puede detenerse inmediatamente (modo *abort*) o intentar limpiar antes de salir (modo *unwind*).[10]
- **Optimización en tiempo de enlace (LTO y Thin LTO).** Similar al caso de Clang, Rustc puede aplicar optimizaciones globales durante el proceso de enlace. La variante *Thin LTO* es más rápida, pero con menor impacto.

2.4. Relación con vulnerabilidades comunes

Las medidas de seguridad presentadas en este capítulo están pensadas para reducir los riesgos frente a errores y ataques comunes en programas modernos. A continuación, se explican algunas amenazas frecuentes y cómo estas protecciones ayudan a evitarlas:

Los desbordamientos de búfer[11] ocurren cuando un programa escribe más datos de los que una variable puede almacenar, sobrescribiendo otras zonas de memoria. Esto puede usarse para tomar el control del programa. Técnicas como el *stack protector*, el uso de pila no ejecutable y el mecanismo *Safe Stack* ayudan a evitar este tipo de problemas al vigilar y proteger la memoria que usan las funciones.

Los ataques conocidos como *Return-Oriented Programming* (ROP) [12] intentan ejecutar código malicioso reutilizando partes del programa original. Para hacer esto, el atacante necesita saber exactamente dónde están ciertos fragmentos de código en memoria. Medidas como la ejecución independiente de la posición, la protección de las estructuras internas del programa y la desactivación de ciertos mecanismos de salto (como PLT) hacen que el binario se cargue de forma diferente cada vez, lo que dificulta este tipo de ataque.

Los errores de memoria, como usar memoria ya liberada o no inicializada, o dejar sin liberar la memoria usada, pueden hacer que el programa falle o se comporte de forma inesperada. Herramientas llamadas *sanitizadores*, *AddressSanitizer* o *LeakSanitizer*, ayudan a encontrar estos problemas durante las pruebas del programa, haciendo que sean más fáciles de detectar y corregir.

Los errores en operaciones con números, como cuando un número entero se sale de su límite y "*vuelve a empezar desde cero*", pueden causar fallos sutiles y difíciles de rastrear. En Rust, este tipo de errores se pueden detectar automáticamente si se activa la comprobación de desbordamientos, especialmente durante el desarrollo.

Algunos errores en el código, aunque no siempre provocan fallos visibles, pueden llevar a situaciones incorrectas o peligrosas si no se controlan. Herramientas como *UndefinedBehaviorSanitizer* y las aserciones de depuración ayudan a identificar este tipo de comportamientos que el lenguaje considera "indefinidos", como dividir por cero o acceder fuera de un arreglo.

Finalmente, los fallos graves que el programa no sabe manejar pueden provocar que se quede colgado o que actúe de forma errática. En Rust, el sistema de manejo de errores en tiempo de ejecución (*panic*) permite decidir si se quiere detener el programa inmediatamente o intentar cerrarlo de forma ordenada.

En conjunto, estas medidas contribuyen a que los programas sean más resistentes a fallos y ataques, algo especialmente importante cuando se enfrentan a datos externos o entornos inseguros.

Por ejemplo sin el uso de *stack protector*, un atacante podría escribir más allá del espacio reservado y modificar la dirección a la que volverá el programa al terminar la función, logrando así ejecutar su propio código.

Capítulo 3

Metodología

En este capítulo se expone de manera concisa la estructura y los principios que han regido el desarrollo de la metodología empleada en este trabajo. Se describe un flujo de trabajo totalmente automatizado —o *pipeline*— que abarca desde la generación de combinaciones de compilación hasta la obtención de métricas y la producción de informes y gráficas. Gracias a su diseño modular, este proceso permite evaluar sistemáticamente cómo distintos compiladores, niveles de optimización y *flags* de seguridad afectan al rendimiento, al tamaño del binario y al grado de fortificación de las funciones. Finalmente, se detallan los programas sobre los que se ha aplicado el análisis y los criterios adoptados para la representación visual de los resultados, con el objetivo de facilitar la interpretación y comparar de forma clara las diferentes combinaciones.

3.1. Descripción general del proceso

Para llevar a cabo el análisis, se ha desarrollado un *pipeline* automatizado que permite compilar, ejecutar y evaluar de forma sistemática un conjunto de programas fuente, utilizando distintos compiladores y combinaciones de *flags* de seguridad y optimización.

El flujo del proceso se divide en tres etapas principales:

Entrada: Se parte de programas escritos en C++ y Rust. Para cada uno, se generan múltiples configuraciones mediante la combinación de diferentes opciones de seguridad y niveles de optimización. Estas combinaciones se aplican por separado en tres compiladores: *GCC*, *Clang* y *Rustc*.

Proceso: Cada configuración se compila y ejecuta múltiples veces para obtener métricas estables. Durante las ejecuciones, se recogen automáticamente datos sobre el tiempo de ejecución, el uso de memoria, el tamaño final del binario y el porcentaje de funciones fortificadas detectadas.

Los datos se almacenan inicialmente en ficheros de texto y JSON como salida bruta. A partir de estos ficheros, se genera un informe estructurado en formato

PDF para cada compilador, que organiza los resultados de forma clara para su lectura directa.

Posteriormente, se ejecutan scripts que transforman esa información en un conjunto de gráficas automáticas, también en formato PDF, que permiten ver las comparaciones de forma más efectiva.

Salida: La salida final del *pipeline* incluye informes de texto y JSON: con los datos sin procesar, informes PDF que presentan los resultados estructurados y explicados para cada compilador, y gráficas PDF que muestran comparaciones entre compiladores, *flags* de seguridad y niveles de optimización, permitiendo detectar patrones y diferencias de forma intuitiva.

La Figura 1 muestra el flujo del pipeline creado. Este diseño modular y automatizado garantiza que los resultados puedan obtenerse fácilmente si se añaden nuevos programas o configuraciones, y permite centrarse en el análisis y la interpretación de resultados.

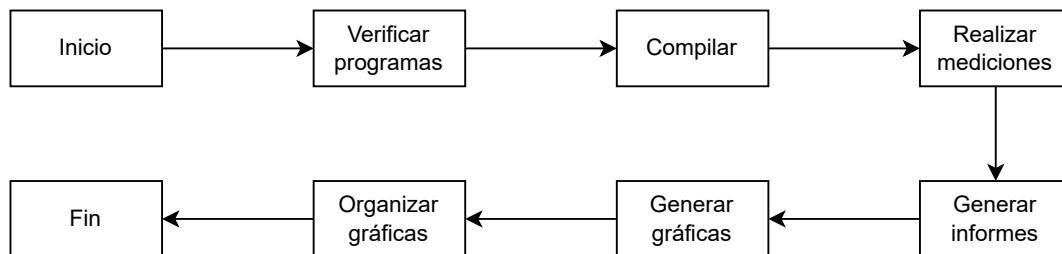


Figura 1: Flujo de la pipeline

3.2. Programas utilizados

Los programas seleccionados están diseñados para revelar de forma clara el impacto de las medidas de seguridad y de las optimizaciones aplicadas. Incluyen operaciones básicas sobre memoria, uso de funciones estándar susceptibles de fortificación, y estructuras de control simples. Esta elección permite centrarse en los efectos que introducen los compiladores y sus configuraciones, sin interferencias derivadas de la complejidad del código fuente.

3.3. Enfoque visual y tipo de gráficas

Debido a la gran cantidad de combinaciones evaluadas y métricas recogidas, se ha optado por una presentación gráfica que permita identificar patrones y diferencias de forma clara. Todas las gráficas comparan los tres compiladores de manera simultánea y están agrupadas en función del eje de comparación principal:

- **Comparación por nivel de optimización.** Se fija una medida de seguridad concreta y se comparan diferentes niveles de optimización. Permite observar cómo afecta la optimización al rendimiento, tamaño o fortificación del binario bajo una misma política de seguridad.
- **Comparación por medida de seguridad.** Se fija un nivel de optimización y se comparan distintas *flags* de seguridad. Esto permite analizar el impacto relativo de cada protección bajo un mismo entorno de rendimiento.
- **Versión porcentual.** Además de las comparaciones directas, también se generan versiones en las que los resultados se expresan como porcentajes respecto al programa base sin optimizaciones ni medidas de seguridad. Esto permite ver de forma clara cuánto mejora o empeora cada opción, sin fijarse en los valores exactos.
- **Gráfica radar de cobertura de protecciones.** Resume qué medidas de seguridad soporta cada compilador. Esta representación permite comparar de forma rápida las capacidades de cada uno.
- **Boxplot general.** Muestra cómo varían los resultados entre las diferentes configuraciones y ayuda a detectar casos que se comportan de forma muy diferente al resto.
- **Facet grid comparativo.** Presenta todas las configuraciones como puntos individuales y permite ver varias métricas al mismo tiempo. Así, se pueden comparar fácilmente los tres compiladores sin separar por tipo de optimización o seguridad.

Todas estas visualizaciones permiten explorar simultáneamente el efecto de las configuraciones sobre el tiempo de ejecución, el uso de memoria, el tamaño del binario y el porcentaje de funciones fortificadas. El enfoque visual ha sido clave para interpretar resultados de forma clara y comprensible, evitando tablas extensas y facilitando la comparación entre cada compilador.

Capítulo 4

Análisis de resultados

4.1. Entorno de experimentación

Las pruebas se han llevado a cabo en un entorno controlado, utilizando un único ordenador para asegurar la coherencia de los resultados. A continuación se detallan las características principales del ordenador donde se han realizado las pruebas:

- **Procesador:** Intel Core i7-10750H (6 núcleos físicos, 12 hilos) con frecuencia base de 2.60 GHz y turbo de hasta 5.00 GHz
- **Memoria RAM:** 16 GB
- **Tarjeta gráfica:** NVIDIA GeForce RTX 3060 Mobile / Max-Q
- **Sistema operativo:** Kali GNU/Linux Rolling (x86_64)
- **Kernel:** Linux 6.12.25

Para realizar las pruebas se emplearon las siguientes versiones de compiladores:

- ***GCC*** (*gcc*): versión 14.2.0 (Debian 14.2.0-19)
- ***Clang*** (*clang*): versión 19.1.7 (Debian)
- ***Rustc*** (*rustc*): versión 1.89.0-nightly

La elección de la versión nightly de *Rustc* es debido a que las opciones avanzadas de seguridad, como los sanitizadores (`-Z sanitizer=...`) y los protectores de pila (`-Z stack-protector=...`) no están disponibles en la rama estable.

4.2. Descripción de conjuntos de datos

Para realizar las pruebas se han utilizado dos programas escritos en C++ y sus equivalentes en Rust. Estos programas han sido diseñados para activar de forma controlada las protecciones de seguridad evaluadas, y para generar un nivel de trabajo suficiente como para observar el efecto de las optimizaciones.

4.2.1. Programas utilizados

`suma.cpp` / `suma.rs`

Este programa realiza una operación intensiva sobre un vector de tamaño considerable (1 millón de posiciones), inicializado con un valor constante.

En ambos casos, el programa permite observar el impacto de la optimización sobre bucles y uso de estructuras (`std::vector` en C++, `Vec` en Rust). Los efectos sobre el uso de memoria y el tamaño del binario. La activación de protecciones básicas sin provocar errores.

`final.cpp` / `final.rs`

Este programa está específicamente diseñado para mostrar vulnerabilidades que pueden ser detectadas por mecanismos como *AddressSanitizer*, *MemorySanitizer* o *stack protector*. Realiza una copia insegura de una cadena de entrada fija con tamaño de 47 bytes a un búfer pequeño de 32 bytes, lo que produce un **desbordamiento de memoria en la pila** si no se aplican medidas de protección.

Las características más relevantes son las siguientes: en C++, el uso de `strcpy` (función insegura) junto con un búfer de 32 bytes activa múltiples protecciones. La versión de Rust simula un comportamiento similar mediante copia manual sin validación de límites. En ambos casos, se incluye uso de memoria dinámica para ampliar el análisis.

4.2.2. Justificación del diseño

Ambos programas son breves y están pensados para aislar el efecto de las medidas de seguridad y las optimizaciones del compilador, evitando interferencias provocadas por lógica compleja o dependencias externas. Esto facilita la interpretación de los resultados, especialmente en gráficos donde se comparan directamente métricas como el tiempo de ejecución, el uso de memoria o el porcentaje de funciones fortificadas.

4.2.3. Compatibilidad

Las versiones fueron compiladas sin modificar para cada compilador evaluado: *GCC*, *Clang* y *Rustc*. No dependen de bibliotecas externas más allá de la biblioteca estándar, lo que garantiza la reproducibilidad del experimento en distintos entornos.

4.3. Descripción de experimentos y resultados

4.3.1. Diseño del experimento

Para cada programa —uno en C++ y su versión equivalente en Rust— se generan múltiples binarios combinando una **única flag de seguridad**, seleccionada de entre un conjunto representativo, y un **nivel de optimización**, específico de cada compilador.

Esto da lugar a un conjunto amplio de combinaciones para cada lenguaje y compilador, que posteriormente se comparan en términos de tiempo de ejecución, uso de memoria, tamaño del binario resultante y porcentaje de funciones fortificadas.

Por defecto, cada combinación se ejecuta al menos una vez, aunque este valor puede aumentarse para reducir variabilidad. En el caso de los experimentos se ha hecho una media con los valores obtenidos de 10 ejecuciones por combinación.

4.3.2. Combinaciones

A continuación se resumen las *flags* con las que se realizan las combinaciones evaluadas para cada compilador, las tablas 1, 3 y 5 muestran las opciones de optimización para cada compilador, mientras que las opciones de seguridad se encuentran en las tablas 2, 4 y 6

GCC (*GCC*)

<i>Flag</i>	Descripción
-O0	Sin optimización. Compilación rápida, ejecución lenta.
-O1	Optimización básica, reduce algo el tamaño y mejora rendimiento.
-O2	Optimización estándar, buena relación entre velocidad y tamaño.
-O3	Optimización agresiva, busca máximo rendimiento (puede aumentar el tamaño).
-Os	Optimiza para reducir el tamaño del binario.

Tabla 1 Niveles de optimización en *GCC*[15]

<i>Flag</i>	Descripción
(vacío)	Sin protecciones adicionales.
-no-pie	Ejecutable con dirección de carga fija (menos seguro).
-pie	Ejecutable con dirección de carga aleatoria (<i>PIE</i>).
-D_FORTIFY_SOURCE=2	Fortificación avanzada de funciones inseguras.
-D_FORTIFY_SOURCE=1	Fortificación básica.
-U_FORTIFY_SOURCE	Desactiva la fortificación.
-z noexecstack	Evita que la memoria de ejecución tenga permiso de ejecución.
-z execstack	Permite ejecutar código en memoria (menos seguro).
-fno-plt	Desactiva tabla de enlaces indirectos (ligero refuerzo de seguridad).
-fsanitize=address	Detecta accesos inválidos a memoria (<i>AddressSanitizer</i>).
-fsanitize=undefined	Detecta comportamientos indefinidos del lenguaje.
-Wl,-z,relro,-z,now	Protege la tabla de enlaces contra sobrescrituras (<i>Full RELRO</i>).
-fno-stack-protector	Desactiva protección contra desbordamiento de variables locales.
-fstack-protector	Protección básica contra desbordamiento de pila.
-fstack-protector-strong	Protección más amplia contra desbordamientos.
-fstack-protector-all	Protección total: cualquier función con variables en pila será protegida.

Tabla 2 *Flags* de seguridad en *GCC*[16]

Clang (*Clang*)

<i>Flag</i>	Descripción
-O0	Sin optimización. Compilación rápida, ejecución lenta.
-O1	Optimización básica, reduce algo el tamaño y mejora rendimiento.
-O2	Optimización estándar, buena relación entre velocidad y tamaño.
-O3	Optimización agresiva, busca máximo rendimiento (puede aumentar el tamaño).
-Os	Optimiza para reducir el tamaño del binario.
-Oz	Optimiza para reducir el tamaño del binario al máximo.
-Ofast	Optimiza con máxima agresividad, desactiva comprobaciones de precisión numérica.

Tabla 3 Niveles de optimización en *Clang*[17]

<i>Flag</i>	Descripción
(vacío)	Sin protecciones adicionales.
-fstack-protector	Igual que en <i>GCC</i> .
-fstack-protector-strong	Igual que en <i>GCC</i> .
-fstack-protector-all	Igual que en <i>GCC</i> .
-fno-stack-protector	Igual que en <i>GCC</i> .
-pie -fPIE	Activa ejecutable independiente de la dirección de carga (<i>PIE</i>).
-no-pie	Desactiva <i>PIE</i> .
-D_FORTIFY_SOURCE=2	Nivel de fortificación.
-D_FORTIFY_SOURCE=1	Nivel de fortificación.
-D_FORTIFY_SOURCE=0	Desactivación.
-z noexecstack	Evita que la memoria de ejecución tenga permiso de ejecución.
-Wl,-z,execstack	Permite ejecutar código en memoria (menos seguro).
-fno-plt	Desactiva la tabla de enlaces indirectos.
-fsanitize=address	Sanitizadores para errores de memoria.
-fsanitize=undefined	Sanitizadores para comportamiento indefinido.
-Wl,-z,relro,-z,now	Protege estructuras internas al enlazar ficheros (<i>Full RELRO</i>).
-flto	Activación de optimization al enlazar ficheros (global entre archivos).
-fsanitize=safe-stack	Protección específica: separa datos vulnerables de otras variables en pila.
-fsanitize=memory	Detecta uso de memoria sin inicializar (<i>MemorySanitizer</i>).

Tabla 4 *Flags* de seguridad en *Clang*[17]

Rustc

<i>Flag</i>	Descripción
-C opt-level=0	Sin optimización.
-C opt-level=1	Optimización básica.
-C opt-level=2	Optimización estándar.
-C opt-level=3	Optimización máxima.
-C opt-level=s	Optimiza para tamaño reducido.
-C opt-level=z	Optimiza para tamaño mínimo absoluto.

Tabla 5 Niveles de optimización en *Rustc*[18]

<i>Flag</i>	Descripción
(vacío)	Sin medidas adicionales.
-C overflow-checks=on	Comprobación en tiempo de ejecución de desbordamientos aritméticos.
-C debug-assertions=on	Comprobaciones de depuración activadas.
-C panic=abort	El programa se aborta directamente ante un error grave (más rápido, menos seguro).
-C lto	Optimization al enlazar ficheros (convencional).
-C lto=thin	Optimization al enlazar ficheros (más ligera).
-Z sanitizer=address	Detecta accesos inválidos a memoria (<i>AddressSanitizer</i>).
-Z sanitizer=leak	Detecta fugas de memoria (<i>LeakSanitizer</i>).
-Z sanitizer=memory	Detecta uso de memoria sin inicializar (<i>MemorySanitizer</i>).
-C link-arg=-Wl,-z,relro,-z,now	Protege estructuras internas al enlazar (<i>Full RELRO</i>).
-Z stack-protector=strong	Protección fuerte contra desbordamientos.
-Z stack-protector=basic	Protección básica de pila.
-Z stack-protector=none	Sin protección de pila.
-C link-arg=-Wl,-z,noexecstack	Memoria no ejecutable.
-C link-arg=-Wl,-z,execstack	Memoria ejecutable.
-C relocation-model=pic -C link-arg=-pie	<i>PIE</i> habilitado.
-C relocation-model=static -C link-arg=-no-pie	<i>PIE</i> deshabilitado.

Tabla 6 *Flags* de seguridad en *Rustc*[19]

4.4. Análisis de resultados

Para los resultados se usan medias de varias pruebas realizadas con los distintos programas.

4.4.1. Tiempo de ejecución

Impacto de las Optimizaciones (-O0 a -Ofast)

El objetivo es evaluar cómo cada nivel de optimización afecta el rendimiento en cada compilador, en su versión por defecto sin *flags*, los tiempos obtenidos se detallan en la

tabla 7

Nivel	<i>Clang</i>	<i>GCC</i>	<i>Rustc</i>	Observaciones Clave
-O0	1.34 ms	1.40 ms	1.66 ms	Rust es ~24 % más lento que <i>Clang</i> por verificaciones de seguridad integradas.
-O1	1.33 ms (-0.7 %)	1.74 ms (+24 %)	1.35 ms (-19 %)	<i>GCC</i> sufre un retroceso inusual; Rust optimiza mejor.
-O2	1.55 ms (+16 %)	1.08 ms (-23 %)	2.32 ms (+40 %)	<i>GCC</i> brilla en -O2; Rust penalizado por checks adicionales.
-O3	1.31 ms (-2 %)	1.23 ms (-12 %)	1.52 ms (-8 %)	<i>Clang</i> y <i>GCC</i> cercanos; Rust se recupera.
-Os	1.74 ms (+30 %)	1.12 ms (-20 %)	1.35 ms (-19 %)	-Os en <i>GCC</i> es el más equilibrado.
-Oz	2.00 ms (+49 %)	N/A	1.53 ms (-8 %)	-Oz en <i>Clang</i> sacrifica velocidad por tamaño.
-Ofast	1.34 ms (=)	N/A	N/A	<i>Clang</i> ignora seguridad por velocidad.

Tabla 7 Impacto de las optimizaciones por compilador

En el nivel sin optimización (-O0), *Clang* registra un tiempo de 1.34 ms, *GCC* es ligeramente más lento con 1.40 ms, y *Rustc* resulta aproximadamente un 24 % más lento que *Clang* debido a las verificaciones de seguridad integradas. Al aplicar la optimización -O1, *Clang* mantiene un rendimiento estable con una ligera mejora, *GCC* sufre un retroceso notable y *Rustc* mejora significativamente, superando a *GCC*, lo que indica una mejor optimización en este nivel. Con -O2, *GCC* destaca con la mejor mejora en velocidad, mientras que *Clang* empeora y *Rustc* se penaliza por los checks adicionales que realiza. En -O3, *Clang* y *GCC* alcanzan tiempos cercanos, con mejoras respecto a niveles previos, mientras que *Rustc* se recupera, aunque sigue siendo más lento. Al usar la optimización para tamaño -Os, *GCC* ofrece un buen equilibrio con una mejora considerable, mientras *Clang* empeora y *Rustc* mantiene un rendimiento aceptable. En -Oz, *Clang* sacrifica velocidad por un tamaño más reducido, *Rustc* mantiene un rendimiento similar a -O3, y no hay datos disponibles para *GCC*. Finalmente, -Ofast, disponible solo para *Clang*, mantiene un rendimiento similar al nivel sin optimización pero omite comprobaciones de seguridad para maximizar la velocidad. En resumen, *GCC* presenta el mejor rendimiento en niveles como -O2 y -Os, *Clang* es consistente y rápido en general, especialmente con -Ofast, y *Rustc*, aunque generalmente más lento, mejora significativamente en ciertos niveles pese a sus medidas de seguridad.

Impacto de las medidas de seguridad

El objetivo es calcular el *overhead* exacto que implica activar protecciones de seguridad en aquellas combinaciones donde se observaron los peores resultados de rendimiento. Para ello, se evalúa el impacto de las *flags* de seguridad en cada compilador utilizando su nivel de optimización óptimo, determinado según los datos de rendimiento previamente

analizados: en el caso de *Clang*, se selecciona -O3 por su rendimiento consistente; para *GCC*, se opta por -O2, ya que dió el mejor tiempo en las pruebas; y para *Rustc*, se utiliza también -O3, al ofrecer un buen equilibrio entre seguridad y rendimiento. Tras las pruebas y observando los peores resultados, se han agrupado todos en la tabla 8

Overhead de *flags* de seguridad

<i>Flag</i>	<i>Clang</i> (-O3)	<i>GCC</i> (-O2)	<i>Rustc</i> (-O3)	<i>Overhead</i>
-fstack-protector-strong	1.31 → 1.35 ms	1.08 → 1.13 ms	1.52 → 1.55 ms	+2–5 %
-D_FORTIFY_SOURCE=2 (C/C++)	1.31 → 1.47 ms	1.08 → 1.24 ms	N/A	+12–15 %
Sanitizadores:				
-fsanitize=address	1.31 → 3.05 ms	1.08 → 3.03 ms	1.52 → 3.25 ms	+114–146 %
-Z sanitizer=memory (Rust)	N/A	N/A	1.52 → 2.71 ms	+78 %

Tabla 8 Casos con mayor *overhead* de flags de seguridad en niveles óptimos

GCC tiene el mayor *overhead* relativo en su nivel óptimo (-O2), mientras que *Rustc* mantiene un coste bajo en *flags* estándar (e.g., *overflow checks*).

Comparativa Final: Seguridad vs. Rendimiento

Para acabar en la tabla 9 se ha elegido un nivel típico para producción como -O2 y se han comparado los tiempos de ejecución de cada compilador sin *flags* de seguridad junto al overhead que suponen aquellas comunes en este ámbito como -fstack-protector-strong y -D_FORTIFY_SOURCE=2

Compilador	Tiempo (ms)	<i>Overhead</i> Seguridad	Ventaja Principal
<i>Clang</i>	1.55	+12 % (con <i>flags</i>)	Optimización estable.
<i>GCC</i>	1.08	+15 % (con <i>flags</i>)	Mejor rendimiento en -O2.
<i>Rustc</i>	2.32	+2 % (checks integrados)	Seguridad sin configurar.

Tabla 9 Comparativa de seguridad vs rendimiento

Conclusiones

En código con alta demanda de rendimiento la opción recomendada es *GCC* -O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong. Aunque sufre un *overhead* combinado de hasta +27 %, sigue siendo el más rápido (1.37 ms). Ideal para entornos donde se prioriza el rendimiento pero se requiere cierta protección básica. Una alternativa estable es *Clang* -O3 -D_FORTIFY_SOURCE=2 -fstack-protector-strong con rendimiento algo menor (~1.47 ms), pero más consistente en otros niveles de optimización.

En código donde la seguridad es crítica la mejor opción es *Rustc* -O3 ya que mantiene un excelente equilibrio, posee seguridad por defecto, y solo un **+2 % de overhead** al sumar protecciones. Además, no requiere configuración adicional para la mayoría de los checks (desbordamiento, acceso inválido, etcétera).

En desarrollo, pruebas y depuración para C/C++, es muy recomendable el uso de `-fsanitize=address` junto con los niveles de optimización -O1 o -O2 para detectar errores de memoria. Aunque presenta un *overhead* elevado ($\sim +133\%$), resulta sumamente útil para capturar fallos complejos y difíciles de reproducir. En el caso de **Rust**, la opción `-Z sanitizer=memory` permite detectar accesos inválidos y condiciones de carrera con un impacto similar en el rendimiento ($\sim +114\%$), siendo una herramienta eficaz en entornos de prueba.

Los sanitizadores deben evitarse en producción, ya que pueden multiplicar el tiempo de ejecución hasta por $2.5\times$. **Rust** se destaca por ofrecer protecciones de memoria integradas con un impacto mínimo en el rendimiento, lo que lo convierte en una opción sólida cuando la seguridad no es negociable. *GCC*, por otro lado, alcanza el mejor rendimiento bruto incluso al aplicar medidas de seguridad, aunque requiere ajustes más precisos y presenta un comportamiento menos predecible en niveles de optimización intermedios.

4.4.2. Uso de memoria

Efecto de las optimizaciones

Las optimizaciones afectan el uso de memoria de manera diferente según el compilador y el nivel aplicado. A continuación, se detalla el consumo exacto en KB para cada caso en la tabla 10 para *Clang*, la 11 para *GCC* y la 12 para *Rustc*

Clang:

Nivel de optimización	Memoria (KB)	Variación respecto a -O0
-O0 (sin optimización)	3270	—
-O1	3446	+5.4 %
-O2	3489	+6.7 %
-O3	3433	+5.0 %
-Os (optimizado para tamaño)	3416	+4.5 %
-Oz (máxima reducción de tamaño)	3013	-7.9 % (el único que reduce memoria)
-Ofast	3448	+5.4 %

Tabla 10 Uso de memoria en *Clang* según nivel de optimización

En general, las optimizaciones aumentan ligeramente el uso de memoria en *Clang*, excepto -Oz, que logra una reducción significativa ($\sim 8\%$).

GCC:

Nivel de optimización	Memoria (KB)	Variación respecto a -O0
-O0	3052	—
-O1	3051	-0.03 % (prácticamente igual)
-O2	3052	0 %
-O3	3459	+13.3 % (el mayor aumento)
-Os	3038	-0.5 % (ligera mejora)

Tabla 11 Uso de memoria en *GCC* según nivel de optimización

GCC mantiene un consumo estable en -O0, -O1 y -O2, pero -O3 aumenta memoria un 13 %, mientras que -Os ofrece una ligera reducción.

Rustc:

Nivel de optimización	Memoria (KB)	Variación respecto a -O0
-O0	1919	—
-O1	1996	+4.0 %
-O2	2004	+4.4 %
-O3	1966	+2.4 %
-Os	2014	+4.9 %
-Oz	1976	+3.0 %

Tabla 12 Uso de memoria en *Rustc* según nivel de optimización

Rust tiene el menor consumo de memoria en general. Las optimizaciones aumentan ligeramente el uso ($\sim 2\text{--}5\%$), pero sin cambios drásticos.

En base a los resultados obtenidos, -Oz (*Clang*) y -Os (*GCC*) son los únicos niveles que reducen memoria respecto a -O0. -O3 es el que más aumenta memoria en *GCC* (13 %), pero en *Clang* y *Rustc* el impacto es menor ($\sim 5\%$). El compilador más consistente y eficiente es *Rustc*, que siempre está por debajo de 2100 KB.

Efecto de las medidas de seguridad

Algunas protecciones incrementan significativamente el uso de memoria. Los casos más notables incluyen los sanitizers, que son los mayores consumidores. Por ejemplo, *AddressSanitizer* puede aumentar la memoria en *Clang* desde aproximadamente 3400 KB hasta $\sim 12000\text{--}14000$ KB ($\times 3\text{--}4$). En *Rustc*, el aumento es de ~ 2000 KB a $\sim 8000\text{--}10000$ KB ($\times 4\text{--}5$). *MemorySanitizer* muestra un impacto similar.

Respecto a los **stack protectors**, usar `-fstack-protector-strong` aumenta la memoria entre 5 y 10 % en *Clang* y *GCC*. Con `-fstack-protector-all` el aumento puede

llegar al 15–20 %.

Otras protecciones como `-pie -fPIE` tienen un impacto menor (2–5 %). La opción `-flto` puede incluso reducir ligeramente el uso de memoria (3–5 %).

Rust tiende a manejar mejor ciertas protecciones como el stack protector (aumentos de 5–10 %), mientras que en C++ se elevan más (15–20 %). *Clang* y *GCC* muestran comportamientos similares, aunque *GCC* suele ser ligeramente más eficiente al aplicar sanitizers.

Comparación entre compiladores

Para estas comparaciones uso el consumo obtenido en -O0 y -O3 sin *flags* de seguridad en la tabla 13 para ver como varía en función del grado de optimización, también he hecho la comparativa con algunas protecciones intensivas como *AddressSanitizer* en la tabla 14

Compilador	Memoria en -O0 (KB)	Memoria en -O3 (KB)
<i>Clang</i>	3270	3433
<i>GCC</i>	3052	3459
<i>Rustc</i>	1919	1966

Tabla 13 Consumo comparado entre compiladores sin protecciones

Compilador	Memoria base (KB)	Memoria con ASan (KB)
<i>GCC</i>	~3050	~11000–13000
<i>Clang</i>	~3400	~12000–14000
<i>Rustc</i>	~2000	~8000–10000

Tabla 14 Impacto de *AddressSanitizer* en uso de memoria

Optimización para tamaño (-Os / -Oz):

- *Clang* -Oz: 3013 KB (mejor resultado en C++).
- *GCC* -Os: 3038 KB.
- *Rustc* -Oz: 1976 KB (el más eficiente en absoluto).

Conclusión final

Rust es el más eficiente en cuanto al uso de memoria en todos los escenarios analizados, mostrando un consumo estable incluso al aplicar distintas medidas de seguridad.

Por su parte, *Clang* y *GCC* presentan comportamientos comparables, aunque *Clang* logra un mejor rendimiento cuando se compila con la opción `-Oz` orientada a reducir el tamaño.

Las protecciones que generan un mayor impacto en el uso de memoria son los **sanitizers**, ya que pueden **multiplicar el consumo por un factor de 3 a 5 veces**. En cuanto a las optimizaciones, la opción `-O3` **no siempre mejora el uso de memoria**: en *GCC*, por ejemplo, puede incrementar el consumo en un 13 %, mientras que en *Rustc* su efecto es mucho más limitado, con un aumento cercano al 2 %.

4.4.3. Tamaño del binario

Optimización

Las opciones de compilación como `-Os` y `-Oz` están diseñadas para reducir el tamaño de los archivos binarios que se generan al compilar un programa.

Según los datos analizados, estas opciones logran binarios más pequeños que los niveles de optimización estándar como `-O0`, `-O1` o `-O2`. Entre ellas, `-Oz` es más efectiva en esta tarea que `-Os`, ya que logra tamaños incluso menores en algunos casos, con diferencias que pueden llegar hasta los 6000 KB.

Por otro lado, la opción `-Ofast`, aunque no está disponible en todos los compiladores, produce binarios más pequeños que los generados con `-O3`, pero siguen siendo más grandes que los de `-Os` o `-Oz`, ya que está enfocada en la velocidad de ejecución y no tanto en el tamaño, aunque algunas optimizaciones ayudan a eliminar código innecesario.

En cuanto a los niveles más agresivos como `-O3` y `-Ofast`, se observa que generan binarios considerablemente más grandes, especialmente con *Clang* y *GCC*, debido a técnicas como la inserción masiva de funciones (*inlining*) y la vectorización del código.

Aunque `-Ofast` puede resultar en un tamaño ligeramente menor que `-O3`, sigue teniendo un tamaño mayor que las opciones enfocadas en reducir el tamaño como `-Os`.

Seguridad

Cuando se activan ciertas protecciones en la compilación de programas, el tamaño de los binarios puede aumentar notablemente.

Por ejemplo, los sanitizadores como *Address* o *Memory* pueden hacer que el binario final sea entre dos y tres veces más grande debido a que se añade código adicional para revisar que no haya errores mientras el programa se está ejecutando.

Otras protecciones como los *stack protectors* también incrementan el tamaño, aunque en menor medida. Además, mecanismos como *RELRO* y *PIE* pueden sumar entre un 10 % y 20 % de tamaño adicional, según el compilador.

El problema se intensifica cuando se combinan varias protecciones simultáneamente,

por ejemplo, usar sanitizadores junto con protecciones de pila y PIE puede llevar a que el binario en *Clang* supere los 5000 KB, comparado con unos 1000 KB que tiene en una combinación básica sin *flags*.

Comparación entre Compiladores

En general, los programas compilados con Rust suelen ocupar menos espacio que los generados con compiladores de C++ como *Clang* o *GCC*, incluso cuando se usan combinaciones similares.

Por ejemplo, la combinación por defecto en Rust produce binarios más pequeños que la combinación por defecto en C++. Entre los compiladores de C++, *Clang* tiende a generar archivos ligeramente más pequeños que *GCC*, sobre todo cuando se usan opciones como `-Os` o `-Oz`, pensadas para reducir el tamaño.

Al aplicar protecciones adicionales como sanitizadores el aumento de tamaño es más notable en C++, usar el sanitizador de *Address* puede duplicar el tamaño del binario, mientras que en Rust el aumento es más moderado, alrededor de un 50 %.

En cuanto a otras protecciones como *RELRO* o protecciones de pila, su efecto sobre el tamaño es similar en todos los compiladores, sin embargo Rust parece integrar estas medidas de forma más eficiente, ya que los binarios que genera son menores en tamaño que los de C++.

Recomendaciones

Si lo que se quiere es reducir al máximo el tamaño de un programa, lo más efectivo es compilarlo con la opción `-Oz` (o `-Os` si `-Oz` no está disponible). Estas opciones le indican al compilador que priorice el tamaño del ejecutable por encima del rendimiento, lo que ayuda a que el resultado final ocupe lo menos posible. También conviene desactivar herramientas de depuración o protecciones adicionales que no sean estrictamente necesarias, ya que suelen aumentar el tamaño del binario.

Ahora bien, si se busca un equilibrio entre tamaño y seguridad, una buena opción es usar `-Os` combinado con medidas de protección como `-fstack-protector-strong`, que sirve para mitigar ciertos ataques a la pila. A esto se le pueden añadir opciones al enlazador como `-Wl,-z,relro,-z,now`, que activan protecciones en tiempo de ejecución para dificultar la modificación de partes sensibles del programa.

Por último, no es recomendable usar muchas medidas pesadas al mismo tiempo como los sanitizadores, ejecutables tipo *PIE* o protecciones muy estrictas para la pila a no ser que sea realmente necesario. Este tipo de combinaciones puede hacer que el programa aumente bastante de tamaño y también que tarde más en ejecutarse.

4.4.4. Funciones fortificadas

Una de las principales herramientas para aumentar la seguridad del código en C y C++ es la opción `FORTIFY_SOURCE`[13]. Estas opciones provocan que el compilador (*GCC* o *Clang*) inserte comprobaciones adicionales en ciertas funciones estándar que podrían ser un vector de ataque, como los errores de desbordamiento de búfer. En cambio, el compilador de Rust no admite esta opción, lo que se refleja en los archivos analizados como “N/A”.

Los sanitizadores pese a que su propósito principal no es la fortificación de funciones, contribuyen a la seguridad detectando errores de memoria y comportamiento indefinido durante la ejecución del programa.

La mejor combinación de medidas observada para maximizar la fortificación es usar `-D_FORTIFY_SOURCE=2` junto con sanitizadores. Este tipo de combinaciones son las que provocan un mayor porcentaje de funciones protegidas frente a errores, en comparación con usar `FORTIFY_SOURCE=1` por sí solo.

Pese a que Rust no implementa `FORTIFY_SOURCE`, sus sanitizadores pueden añadir una capa adicional de seguridad, complementando el modelo de seguridad del lenguaje.

Por parte de *GCC* y *Clang*, soportan tanto `FORTIFY_SOURCE` como los sanitizadores. No se observan diferencias notables en la cantidad de funciones fortificadas cuando se usan las mismas opciones.

Para finalizar es necesario ver que aunque Rust no posea soporte para `FORTIFY_SOURCE` esto no implica menor seguridad.

El lenguaje está diseñado para prevenir errores comunes de memoria mediante su sistema de tipos y el modelo de *ownership*, lo que reduce la necesidad de fortificación adicional.

Por otro lado, las opciones de optimización como `-Ofast`, `-Os` o `-Oz` no influyen directamente en la fortificación.

Capítulo 5

Trabajo relacionado

Numerosos trabajos han analizado las técnicas de defensa a nivel de binario, el coste asociado a activarlas y su integración en compiladores modernos como *GCC*, *Clang* o *Rustc*. Este apartado resume brevemente algunas de las líneas de investigación y herramientas más relevantes, contextualizando el presente trabajo dentro del panorama actual.

Uno de los enfoques más comunes es el análisis del impacto de medidas como *Stack Canaries*, *RELRO*, *PIE* o *FORTIFY_SOURCE* sobre la ejecución de programas en C y C++. Por ejemplo, proyectos como el Hardening Guide de Debian[20] o la documentación de Red Hat[21] han explorado las ventajas y limitaciones de estas protecciones, normalmente desde una perspectiva funcional o de seguridad, pero sin cuantificar en detalle su coste en eficiencia.

También existen herramientas académicas y profesionales que automatizan la evaluación de binarios protegidos como BINSEC. Algunos trabajos en entornos como LLVM han implementado marcos de análisis para evaluar cómo las optimizaciones afectan al rendimiento y seguridad del código generado, pero suelen centrarse en contextos más específicos (optimización interprocedimental, protección frente a ataques concretos, etc.).[22]

En el caso de Rust, algunos estudios han analizado su diseño como lenguaje seguro por defecto, comparándolo con C/C++ en términos de prevención de errores comunes, aunque pocos lo han integrado en estudios comparativos prácticos que midan rendimiento frente a compiladores tradicionales.[23] En este sentido, el presente trabajo aporta una comparación directa y reproducible entre *Rustc* y otros compiladores ampliamente utilizados, evaluando no solo el soporte de medidas de seguridad, sino también su impacto cuantitativo.

En cuanto a la representación de resultados, aunque es habitual encontrar tablas de rendimiento o medidas aisladas en benchmarks, la combinación de informes estructurados y visualizaciones gráficas automáticas como la desarrollada en este proyecto no es común en trabajos anteriores. Esto facilita la comprensión global del efecto de cada configuración, al mismo tiempo que permite detectar patrones generales entre compiladores.

Capítulo 6

Conclusiones y trabajo futuro

6.1. Conclusiones

6.1.1. Equilibrio entre seguridad y rendimiento

En la comparación entre distintos compiladores y niveles de optimización, se han identificado varias opciones relevantes según las prioridades del entorno de desarrollo, ya sea el rendimiento, la seguridad o un equilibrio entre ambos factores.

Por un lado, el compilador *GCC* con la opción `-O2` se presenta como una de las mejores alternativas cuando se busca un buen equilibrio entre rendimiento y seguridad básica. En concreto, la combinación de opciones `GCC -O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong` implica un incremento en el consumo de recursos del 27 % respecto a la compilación sin optimizaciones, pero a cambio ofrece un tiempo de ejecución muy competitivo, con una media de 1,37 milisegundos. Este enfoque es adecuado para entornos donde se requiere eficiencia sin renunciar a ciertos mecanismos de protección frente a errores comunes.

Por otro lado, el compilador *Clang* con `-O3` resulta ser una alternativa más estable en términos de consistencia. Aunque su rendimiento es ligeramente inferior (1,47 milisegundos), mantiene una buena estabilidad en distintos niveles de optimización. Esto lo convierte en una opción interesante cuando se necesita previsibilidad y coherencia en el comportamiento del software optimizado.

Finalmente, el lenguaje Rust con la optimización `-O3` destaca especialmente en entornos donde la seguridad es una prioridad absoluta. A diferencia de los compiladores tradicionales de C/C++, Rust integra de forma nativa múltiples mecanismos de protección frente a errores de memoria, como desbordamientos de búfer o accesos indebidos. A pesar de estas protecciones adicionales, el overhead es muy reducido, apenas un 2 %, lo que lo convierte en una opción ideal para aplicaciones críticas en las que los fallos de memoria no son tolerables.

6.1.2. Flags con penalizaciones elevadas

El uso de opciones de compilación avanzadas permite mejorar distintos aspectos del software, como la seguridad o el rendimiento. Sin embargo, estas mejoras pueden tener un impacto significativo en los recursos utilizados durante la ejecución. A continuación, se describen algunos de los casos más relevantes:

Los sanitizadores, como el de *Address*, son herramientas muy eficaces para detectar errores de memoria como desbordamientos de búfer o accesos a zonas inválidas. No obstante, su aplicación tiene un coste considerable: multiplican el tiempo de ejecución aproximadamente por 2,5 veces y el uso de memoria puede incrementarse entre 3 y 5 veces. Por esta razón, no se recomienda su uso en entornos de producción. En cambio, son especialmente útiles durante las fases de desarrollo, pruebas y depuración, donde la detección temprana de errores es prioritaria.

En cuanto a la seguridad en C++, la opción `-fstack-protector-all` proporciona una protección más agresiva contra ataques basados en corrupción de pila. Sin embargo, este enfoque puede incrementar el uso de memoria hasta un 20 %, y las mejoras de seguridad obtenidas respecto a la opción intermedia `-fstack-protector-strong` suelen ser mínimas. Por tanto, su uso debe evaluarse cuidadosamente, especialmente en aplicaciones donde los recursos son limitados.

Respecto a la optimización del rendimiento, la opción `-O3` en *GCC* mejora la velocidad de ejecución, pero a costa de un mayor consumo de memoria, que puede aumentar alrededor de un 13 %. En cambio, cuando se utiliza `-O3` en Rust, el impacto en el uso de memoria es mucho más reducido, con un incremento promedio de apenas un 2,4 %. Esto refuerza la idea de que Rust está diseñado para ofrecer eficiencia sin comprometer la seguridad ni el uso racional de los recursos.

6.1.3. Rust y su diseño

Rust ofrece un enfoque más eficiente para lograr seguridad en tiempo de compilación gracias a su modelo de *ownership*[14] y comprobaciones estáticas. Esto le permite obtener resultados comparables, o en algunos casos superiores a los de C++ con una penalización mucho menor.

Por ejemplo, el *overhead* de las protecciones de seguridad en Rust es de solo un 2 %, frente al 12–15 % habitual en C++ con mecanismos equivalentes. Además, consume hasta un 40 % menos de memoria que *Clang* o *GCC* en combinaciones similares y genera binarios más compactos, incluso al usar sanitizadores (con un incremento del 50 %, frente al 100–200 % en C++).

6.1.4. Inconsistencias entre compiladores

Las optimizaciones intermedias en *GCC* pueden ser inconsistentes; por ejemplo, con `-O1` se ha observado un rendimiento 24 % peor que sin optimización (`-O0`), lo que pone

en duda su fiabilidad en ciertos contextos.

La opción `-fstack-protector-all` no garantiza una protección significativamente superior a `-fstack-protector-strong`, y su mayor consumo de recursos puede no estar justificado en muchos casos.

6.1.5. Recomendaciones generales

■ Producción (C/C++):

- *gcc* `-O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong` para rendimiento.
- *clang* `-O3` para consistencia.

■ Seguridad crítica:

- *Rustc* `-O3` (protecciones integradas sin configuración adicional).

■ Desarrollo/Pruebas:

- Sanitizadores (`-fsanitize=address` en C++, `-Z sanitizer=memory` en Rust) solo en entornos de depuración.

■ Reducción de memoria/tamaño:

- `-Oz` en *clang* (mejor reducción en C++).
- Rust por defecto ya es óptimo en memoria y tamaño.

6.2. Trabajo futuro

Como continuación del trabajo realizado se plantean varias líneas de experimentación como ampliar el conjunto de programas utilizados en las pruebas. Aunque los ejemplos empleados han sido suficientes para observar el efecto de las combinaciones evaluadas, la incorporación de programas con un comportamiento más complejo, o incluso de fragmentos de código reales tomados de proyectos open source, permitiría observar el impacto de las medidas en contextos más realistas y variados.

Otra dirección interesante sería la incorporación de nuevos compiladores o lenguajes. En el caso de C++, podría incluirse el compilador de Microsoft (MSVC) o el de Intel (icc), mientras que a nivel de lenguajes, se podría extender el estudio a otros con orientación

a la seguridad como Go o Zig, ampliando así el espectro de comparativa más allá de los compiladores tradicionales.

También sería relevante ejecutar los experimentos en otras arquitecturas o sistemas operativos. Hasta ahora, todas las pruebas se han realizado en una única máquina con Linux y arquitectura x86_64. Repetir el análisis en entornos ARM o en sistemas como Windows o BSD permitiría evaluar la consistencia de los resultados en plataformas distintas.

A nivel metodológico, sería útil mejorar el pipeline de pruebas para hacerlo más modular y extensible. También se podrían introducir mecanismos de paralelización o automatización avanzada para reducir los tiempos de ejecución y facilitar la inclusión de nuevas configuraciones.

Por último, integrar el pipeline en entornos de integración continua (CI/CD) permitiría su uso en contextos de desarrollo real, facilitando evaluaciones automáticas de seguridad y eficiencia tras cada cambio en el código fuente.

Bibliografía

- [1] Wikipedia. Position-independent code. https://en.wikipedia.org/wiki/Position-independent_code, 2011.
- [2] Wiki.osdev. Stack smashing protector. https://wiki.osdev.org/Stack_Smashing_Protector, 2014.
- [3] Redhat. Hardening elf binaries using relocation read-only (relro). <https://www.redhat.com/en/blog/hardening-elf-binaries-using-relocation-read-only-relro>, 2019.
- [4] Google. Addresssanitizer. <https://github.com/google/sanitizers/wiki/AddressSanitizer>, 2017.
- [5] LLVM. Undefinedbehaviorsanitizer. <https://clang.LLVM.org/docs/UndefinedBehaviorSanitizer.html>, 2016.
- [6] Google. Memorysanitizer. <https://github.com/google/sanitizers/wiki/MemorySanitizer>, 2018.
- [7] Google. Addresssanitizerleaksanitizer. <https://github.com/google/sanitizers/wiki/AddressSanitizerLeakSanitizer>, 2018.
- [8] LLVM. Safestack in clang. <https://clang.LLVM.org/docs/SafeStack.html>, 2015.
- [9] Rust-lang. Integer overflow. <https://doc.rust-lang.org/book/ch03-02-data-types.html#integer-overflow>, 2019.
- [10] Rust-lang. Unrecoverable errors with panic! <https://doc.rust-lang.org/book/ch09-01-unrecoverable-errors-with-panic.html>, 2019.
- [11] OWASP. Buffer overflow. https://owasp.org/www-community/vulnerabilities/Buffer_Overflow, 2020.
- [12] Wikipedia. Return-oriented programming. https://en.wikipedia.org/wiki/Return-oriented_programming, 2014.
- [13] Redhat. Enhance application security with FORTIFY_SOURCE. <https://www.redhat.com/en/blog/enhance-application-security-fortifysource>, 2022.

- [14] Rust-lang. What is ownership? <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>, 2019.
- [15] GCC. Options that control optimization. <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>, 2002.
- [16] OpenSSF. Compiler options hardening guide for c and c++. <https://best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C%2B%2B.html>, 2023.
- [17] LLVM. Clang command line argument reference. <https://clang.llvm.org/docs/ClangCommandLineReference.html>, 2017.
- [18] Rust-lang. The embedded rust book. <https://docs.rust-embedded.org/book/unsorted/speed-vs-size.html>, 2021.
- [19] Rust-lang. The rustc book. <https://doc.rust-lang.org/beta/rustc/codegen-options/index.html>, 2021.
- [20] Debian org. Hardening. <https://wiki.debian.org/Hardening>, 2008.
- [21] Red Hat. Documentación de red hat sobre mitigaciones en binarios. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10, 2014.
- [22] Markaicode. Gcc 14 vs. clang 18: C++26 compiler optimization benchmarks (2025 edition). <https://markaicode.com/gcc-14-vs-clang-18-cpp26-benchmarks/>, 2025.
- [23] Jo Spencer. Comparing the speed of binaries produced by clang and gcc. <https://copyprogramming.com/howto/clang-vs-gcc-which-produces-faster-binaries>, 2023.

Anexo A

Tiempo invertido y diagrama de Gantt

El diagrama de Gantt en la Figura 2 refleja las diferentes fases del proyecto y la dedicación en el tiempo invertidos. Todo comienza con un periodo de investigación donde se adquieren los conocimientos previos necesarios. Seguidamente, comienza la etapa de diseño e implementación de la herramienta desarrollada. Finalmente, el periodo de elaboración de la memoria. Por otro lado, la Tabla 15 detalla el tiempo invertido en cada una de estas fases.

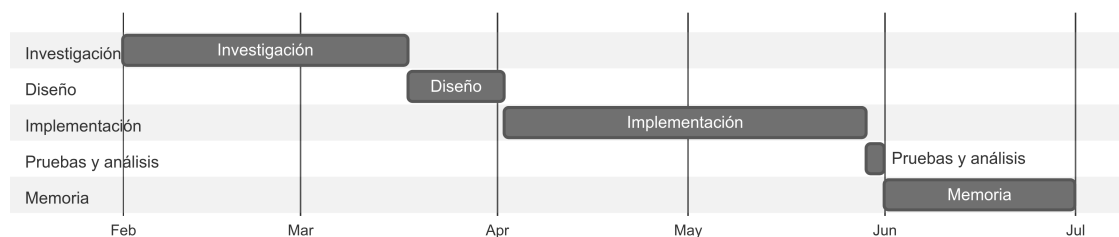


Figura 2: Diagrama de gantt

Fase	Horas
Investigación	25
Diseño	40
Implementación	220
Pruebas y análisis	20
Memoria	100
Total	405

Tabla 15 Horas invertidas

Anexo B

Gráficas

Este anexo contiene todas las gráficas generadas durante el análisis.

El color azul corresponde *GCC*, el naranja a *Clang* y el verde a *Rustc*

A continuación las tablas con los valores por defecto de los compiladores, necesario para las gráficas porcentuales.

Compilador	Optimización	Time (ms)	Memory usage (KB)	File size (KB)	Fortified functions (%)
clang++	-O0	1.34	3270	16.07	0.0
	-O1	1.33	3446	16.00	0.0
	-O2	1.55	3489	16.00	0.0
	-O3	1.31	3433	16.00	0.0
	-Os	1.74	3416	16.00	0.0
	-Oz	2.00	3013	15.98	0.0
	-Ofast	1.34	3448	17.45	0.0
g++	-O0	1.40	3052	16.26	0.0
	-O1	1.74	3051	16.04	0.0
	-O2	1.08	3052	15.99	0.0
	-O3	1.23	3459	16.02	0.0
	-Os	1.12	3038	16.04	0.0
rustc	-O0	1.66	1919	3758.80	0.0
	-O1	1.35	1996	3752.70	0.0
	-O2	2.32	2004	3752.35	0.0
	-O3	1.52	1966	3752.35	0.0
	-Os	1.35	2014	3752.62	0.0
	-Oz	1.53	1976	3752.70	0.0

Tabla 16 Valores por defecto (sin flags de seguridad)

Tabla 17 Valores de referencia y por flag de seguridad (optimización -O0)

Configuración	Time (ms)	Memory usage (KB)	File size (KB)	Fortified functions (%)
Valores por defecto (sin flags de seguridad)				
clang++	1.34	3270	16.07	0.0
g++	1.40	3052	16.26	0.0
rustc	1.66	1919	3758.80	0.0
Stack Protection				
<i>basic</i>				
clang++	1.66	3153	16.13	0.0
g++	1.29	3182	16.32	0.0
rustc	1.90	2039	3759.80	0.0
<i>strong</i>				
clang++	2.18	3118	16.13	0.0
g++	1.29	3182	16.32	0.0
rustc	1.99	1955	3759.91	0.0
<i>all</i>				
clang++	1.66	3153	16.13	0.0
g++	1.22	3188	16.32	0.0
rustc	N/A	N/A	N/A	N/A
<i>disabled</i>				
clang++	1.48	3322	16.07	0.0
g++	1.23	3045	16.26	0.0
rustc	1.25	1968	3758.80	0.0
Pie				
<i>enabled</i>				
clang++	1.19	3348	16.07	0.0
g++	1.06	3052	16.26	0.0
rustc	1.14	1999	3758.80	0.0
<i>disabled</i>				
clang++	1.34	3312	15.91	0.0

Continúa en la página siguiente

Cuadro 17 – Continuación de la página anterior

Configuración	Time (ms)	Memory usage (KB)	File size (KB)	Fortified functions (%)
g++	1.46	3045	16.09	0.0
rustc	1.20	1922	3740.59	0.0
Execstack				
<i>disabled</i>				
clang++	1.29	3310	16.07	0.0
g++	2.08	3052	16.26	0.0
rustc	1.22	1886	3758.80	0.0
<i>enabled</i>				
clang++	3.00	3322	16.07	0.0
g++	1.07	3050	16.26	0.0
rustc	1.52	1960	3758.80	0.0
Sanitizers				
<i>address</i>				
clang++	2.65	13462	1660.77	15.1
g++	1.22	24964	25.92	0.0
rustc	1.75	12688	6251.03	11.5
<i>undefined</i>				
clang++	2.27	5600	416.24	75.0
g++	1.09	3045	17.04	0.0
rustc	N/A	N/A	N/A	N/A
Fortify Source				
<i>level-2</i>				
clang++	1.52	3274	16.07	0.0
g++	1.33	3052	16.26	0.0
rustc	N/A	N/A	N/A	N/A
<i>level-1</i>				
clang++	1.54	3342	16.07	0.0
g++	1.24	3052	16.26	0.0

Continúa en la página siguiente

Cuadro 17 – Continuación de la página anterior

Configuración	Time (ms)	Memory usage (KB)	File size (KB)	Fortified functions (%)
rustc	N/A	N/A	N/A	N/A
<i>disabled</i>				
clang++	1.14	3283	16.07	0.0
g++	1.15	3039	16.26	0.0
rustc	N/A	N/A	N/A	N/A
Relro				
<i>full</i>				
clang++	2.70	3312	15.95	0.0
g++	1.12	3052	16.14	0.0
rustc	2.29	2001	3758.80	0.0
Safe Stack				
<i>enabled</i>				
clang++	1.34	3104	22.96	20.0
g++	N/A	N/A	N/A	N/A
rustc	N/A	N/A	N/A	N/A
Overflow Checks				
<i>enabled</i>				
clang++	N/A	N/A	N/A	N/A
g++	N/A	N/A	N/A	N/A
rustc	1.40	1962	3758.80	0.0
Debug Assertions				
<i>enabled</i>				
clang++	N/A	N/A	N/A	N/A
g++	N/A	N/A	N/A	N/A
rustc	1.25	1984	3758.80	0.0
Panic				
<i>abort</i>				
clang++	N/A	N/A	N/A	N/A

Continúa en la página siguiente

Cuadro 17 – Continuación de la página anterior

Configuración	Time (ms)	Memory usage (KB)	File size (KB)	Fortified functions (%)
g++	N/A	N/A	N/A	N/A
rustc	1.42	1996	3752.00	0.0

Boxplot

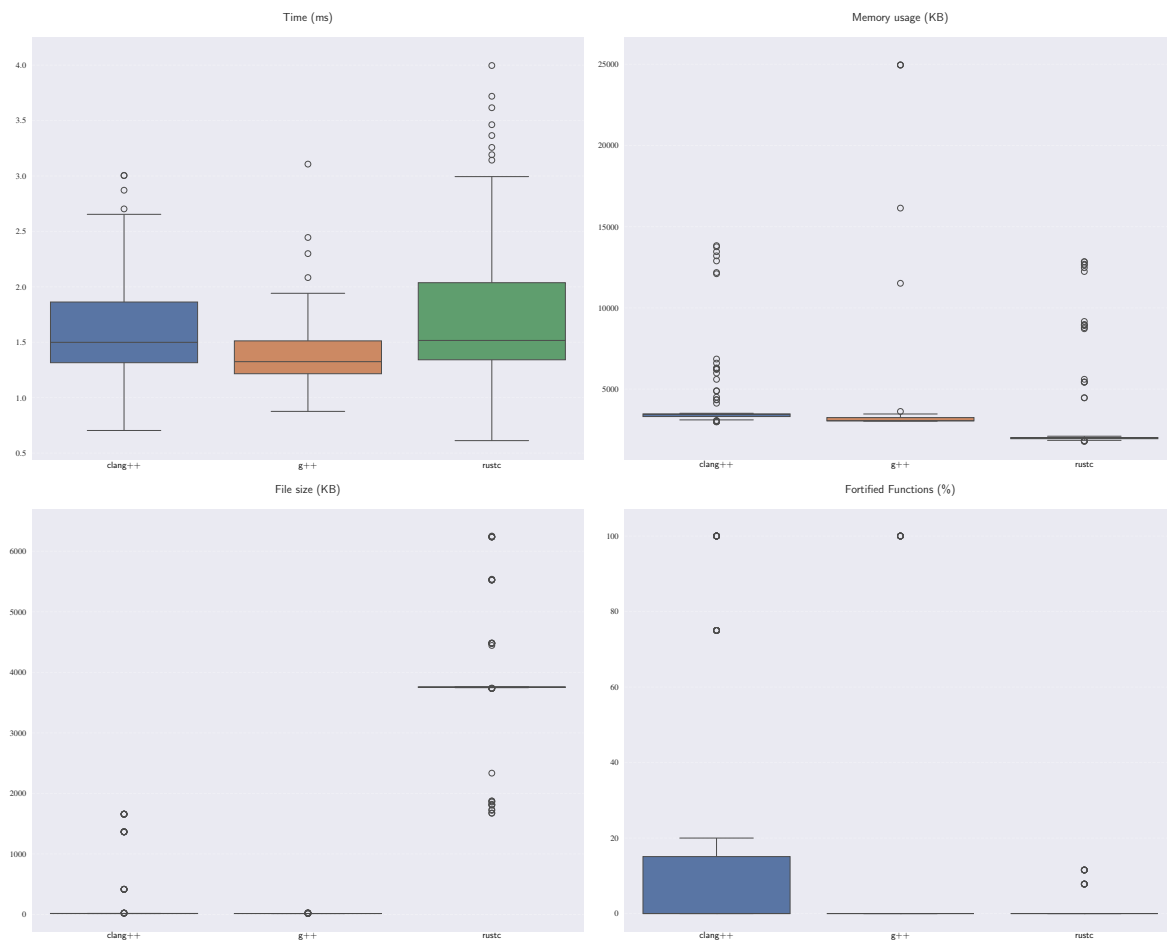


Figura 3: Boxplot

Facet Grid

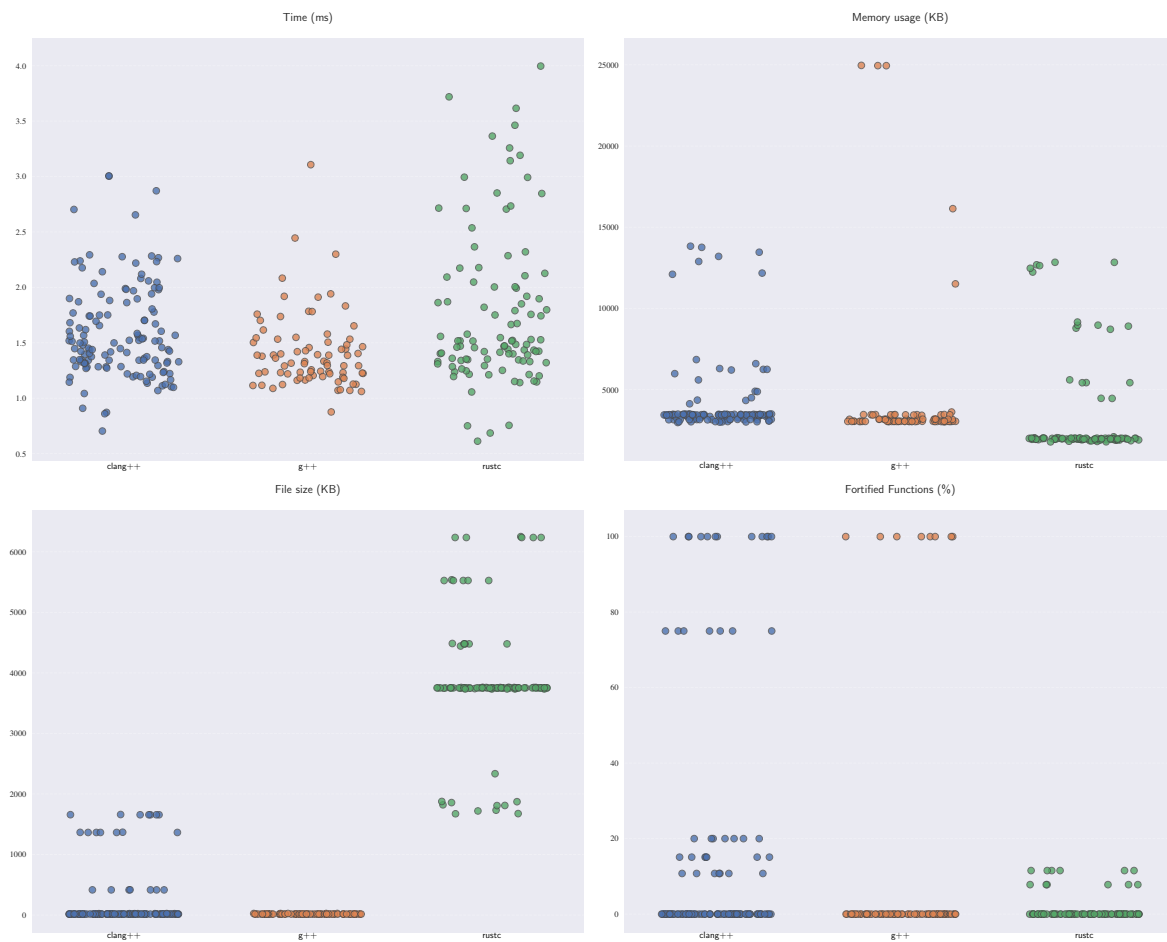


Figura 4: Facet Grid

Comparacion Compiladores

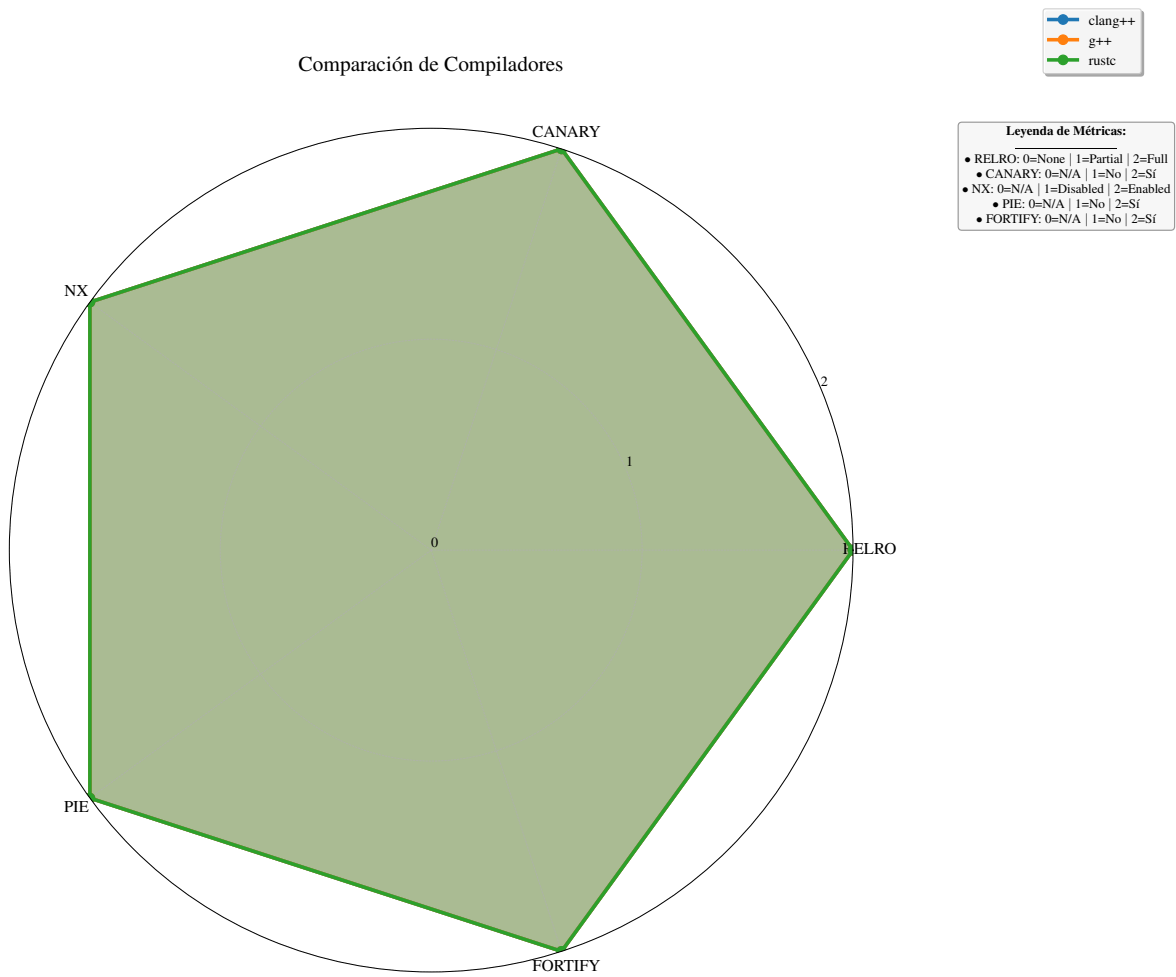


Figura 5: Comparacion Compiladores

Gráfico Protecciones

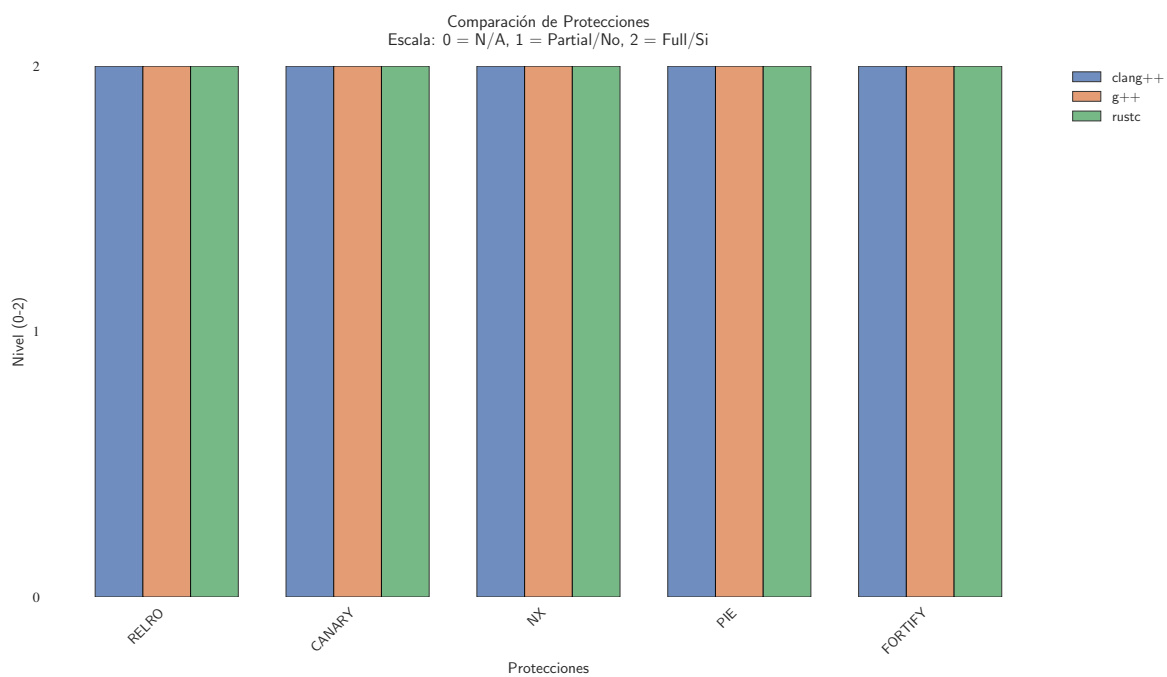


Figura 6: Gráfico Protecciones

Gráficas Por Optimización

-O0

Figura 7: Resultados de compilación con optimización -O0

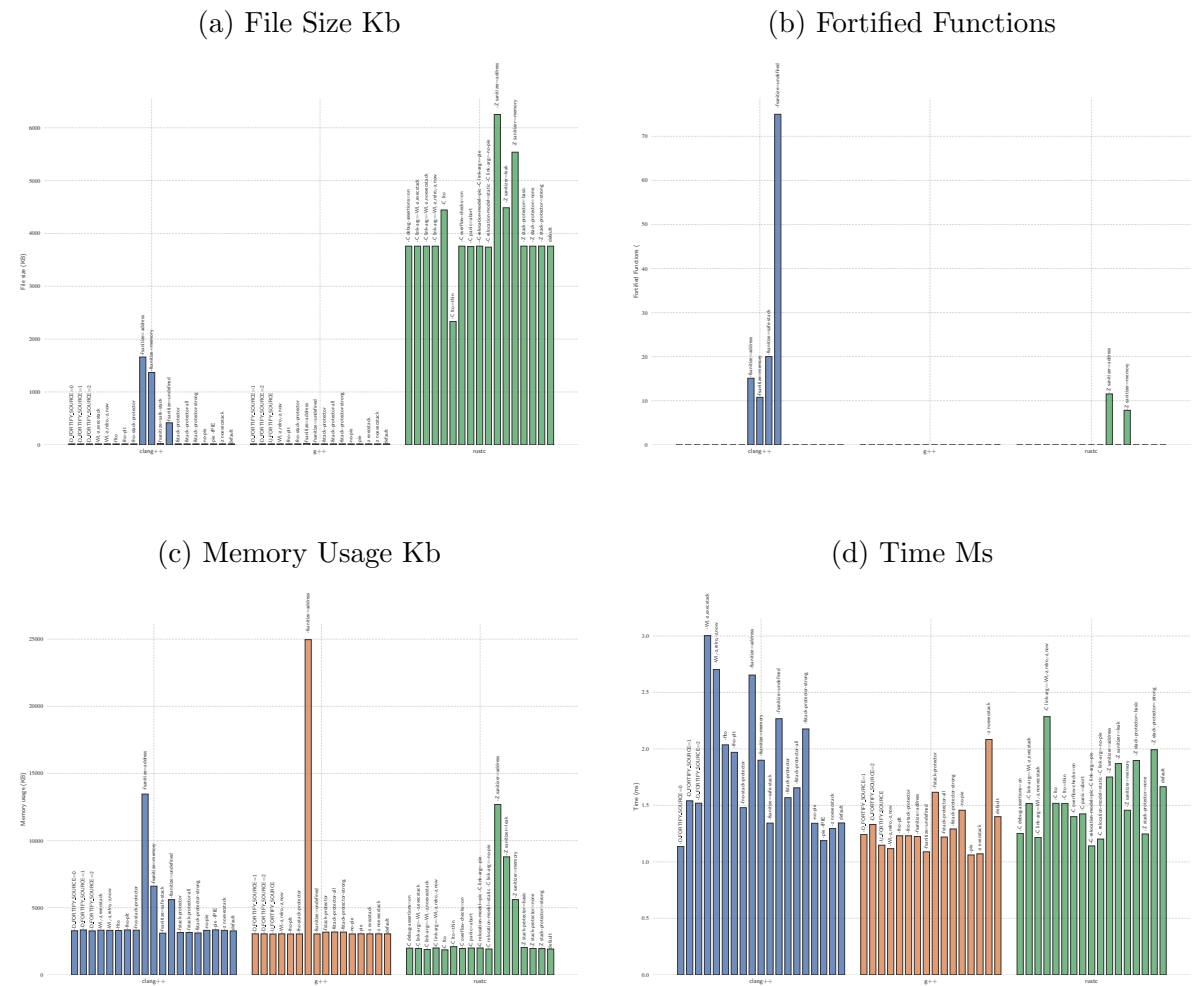
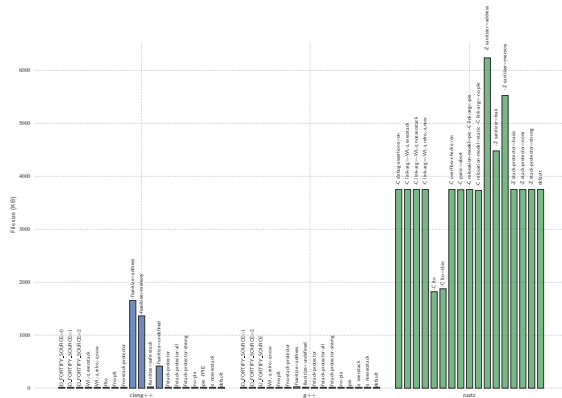
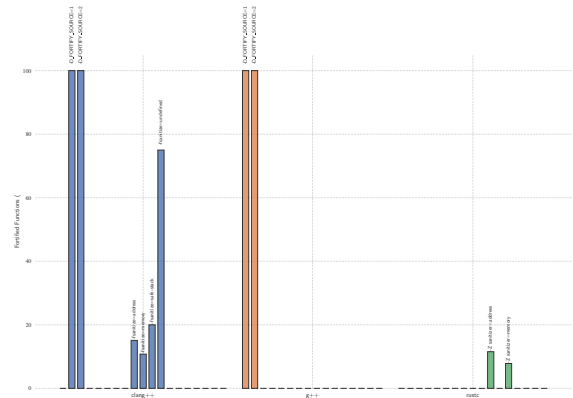


Figura 8: Resultados de compilación con optimización -O1

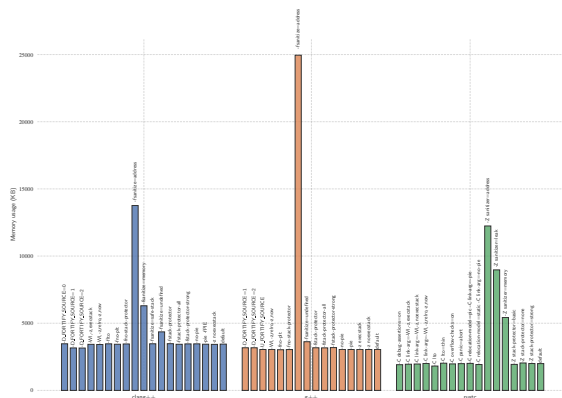
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

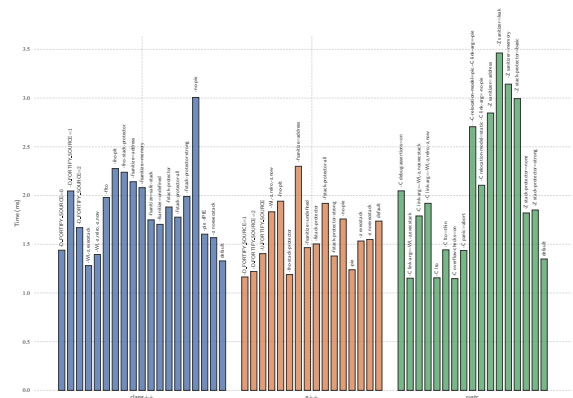
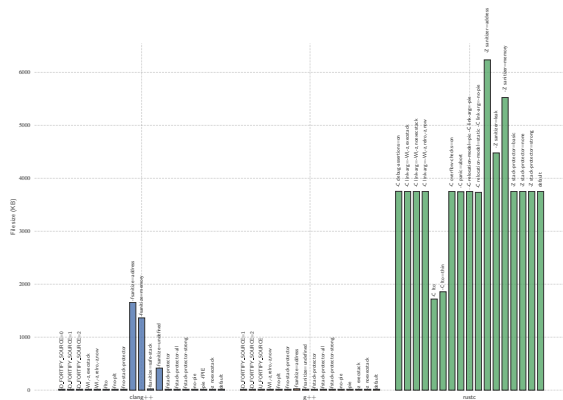
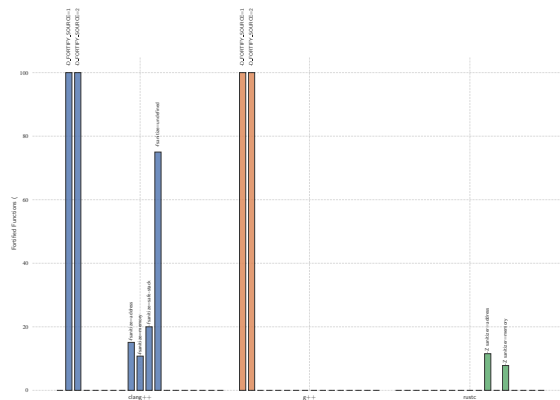


Figura 9: Resultados de compilación con optimización -O2

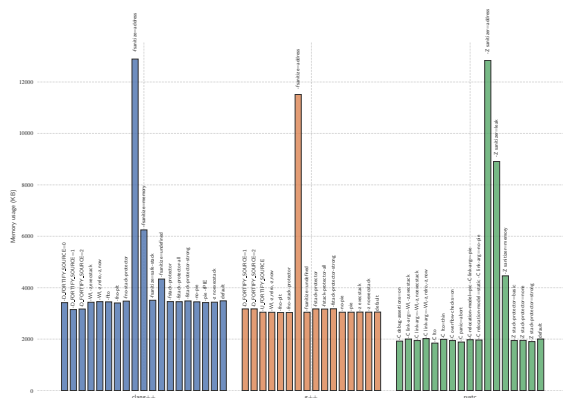
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

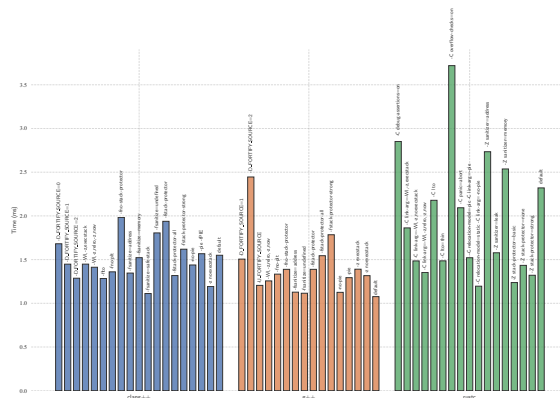
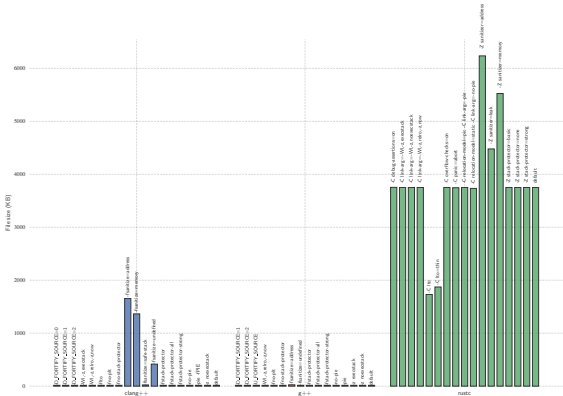
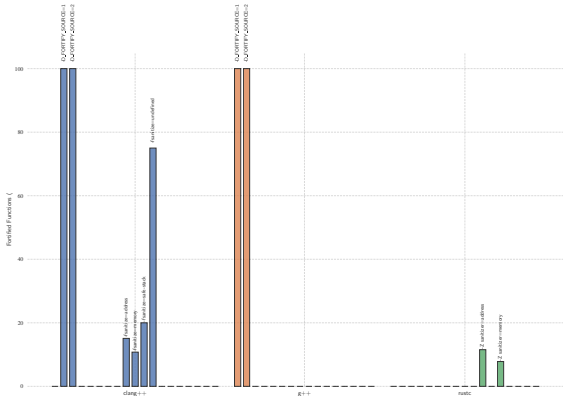


Figura 10: Resultados de compilación con optimización -O3

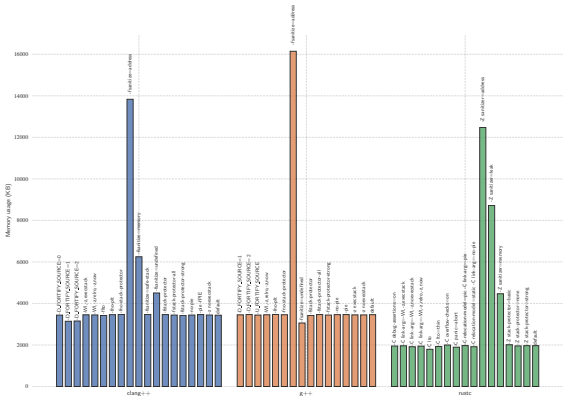
(a) File Size Kb



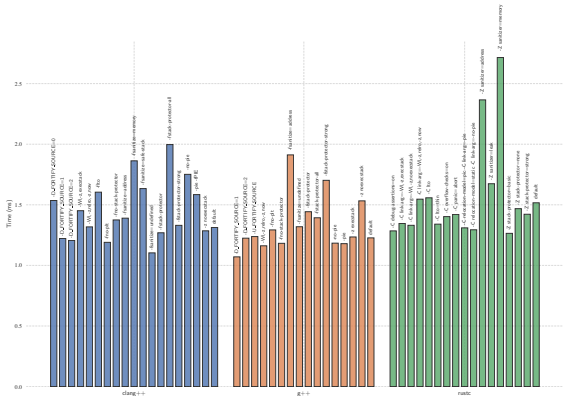
(b) Fortified Functions



(c) Memory Usage Kb



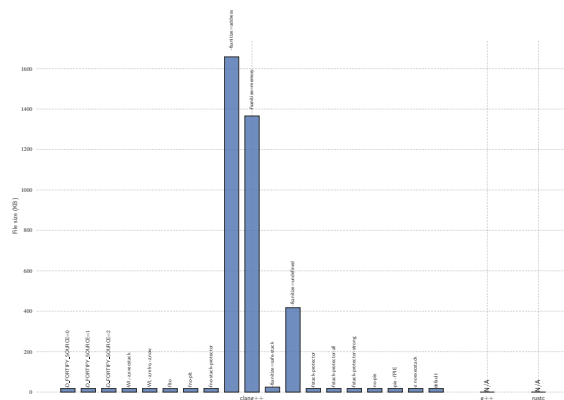
(d) Time Ms



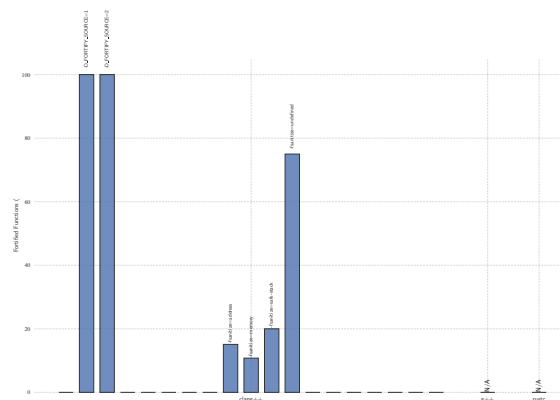
-OFAST

Figura 11: Resultados de compilación con optimización -OFAST

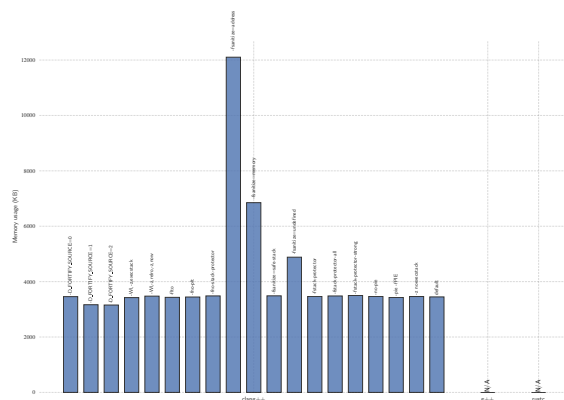
(a) File Size Kb



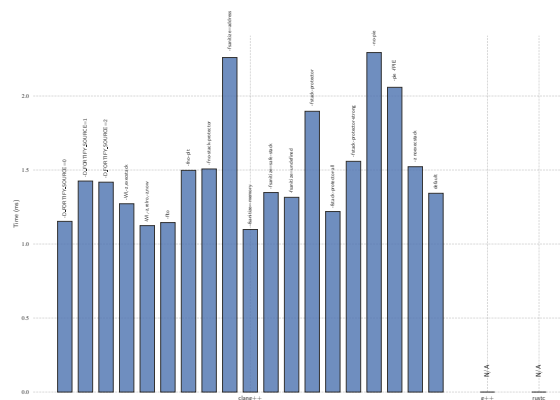
(b) Fortified Functions



(c) Memory Usage Kb



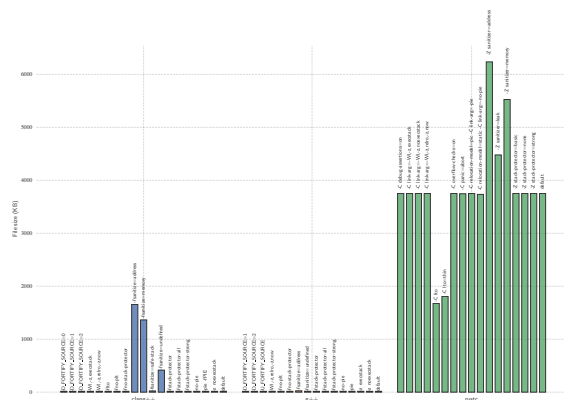
(d) Time Ms



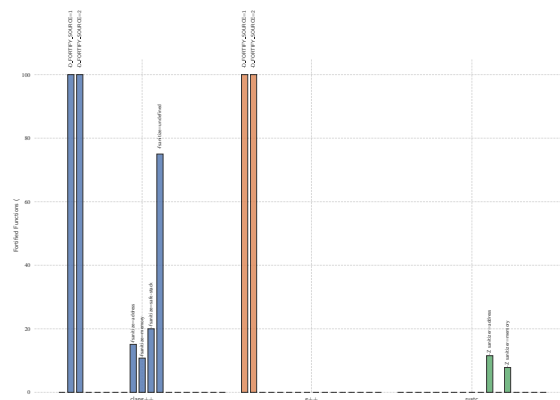
-OS

Figura 12: Resultados de compilación con optimización -OS

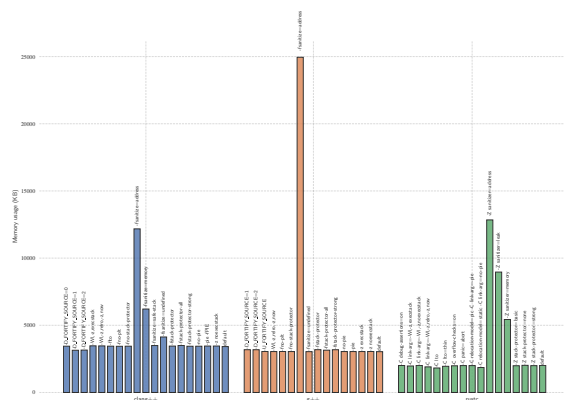
(a) File Size Kb



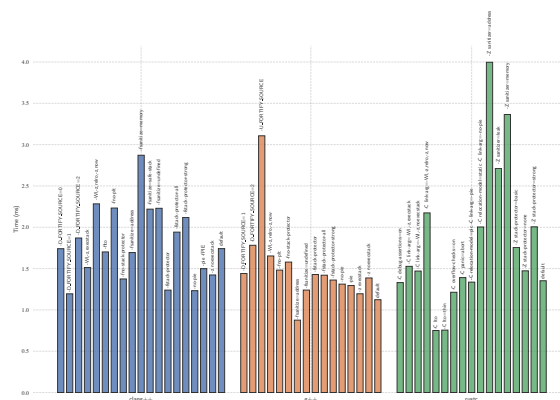
(b) Fortified Functions



(c) Memory Usage Kb



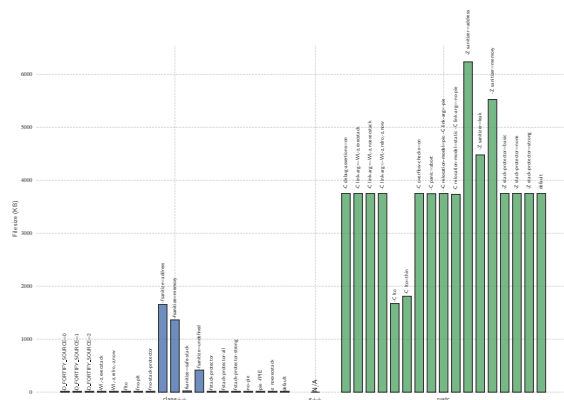
(d) Time Ms



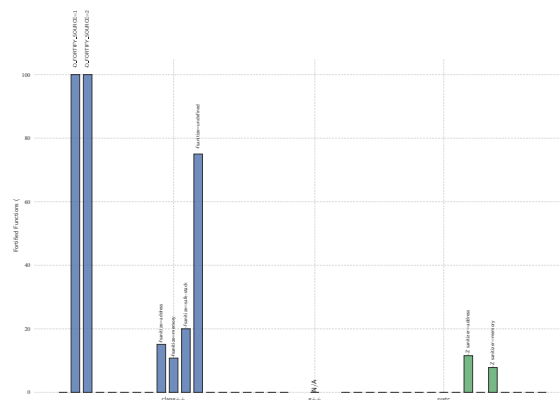
-OZ

Figura 13: Resultados de compilación con optimización -OZ

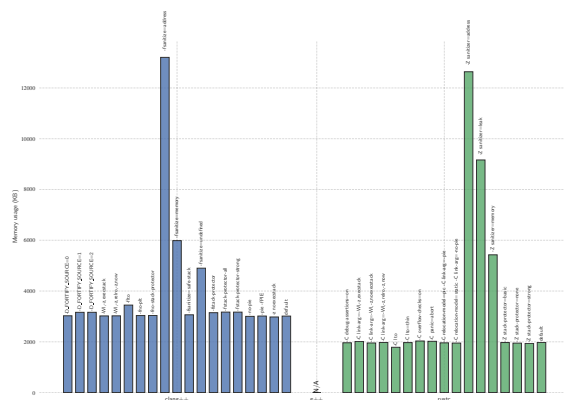
(a) File Size Kb



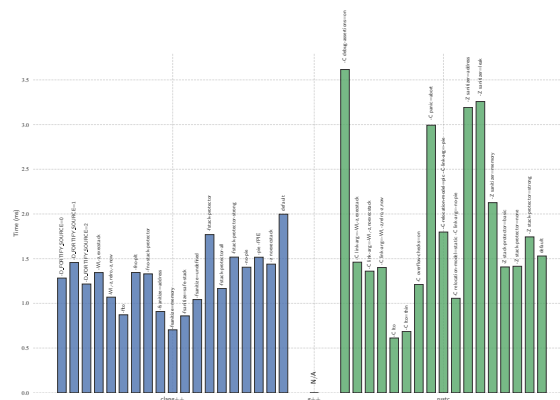
(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms



Gráficas Por Medida Seguridad

debug-assertions

Figura 14: Resultados de compilación con optimización debug-assertions

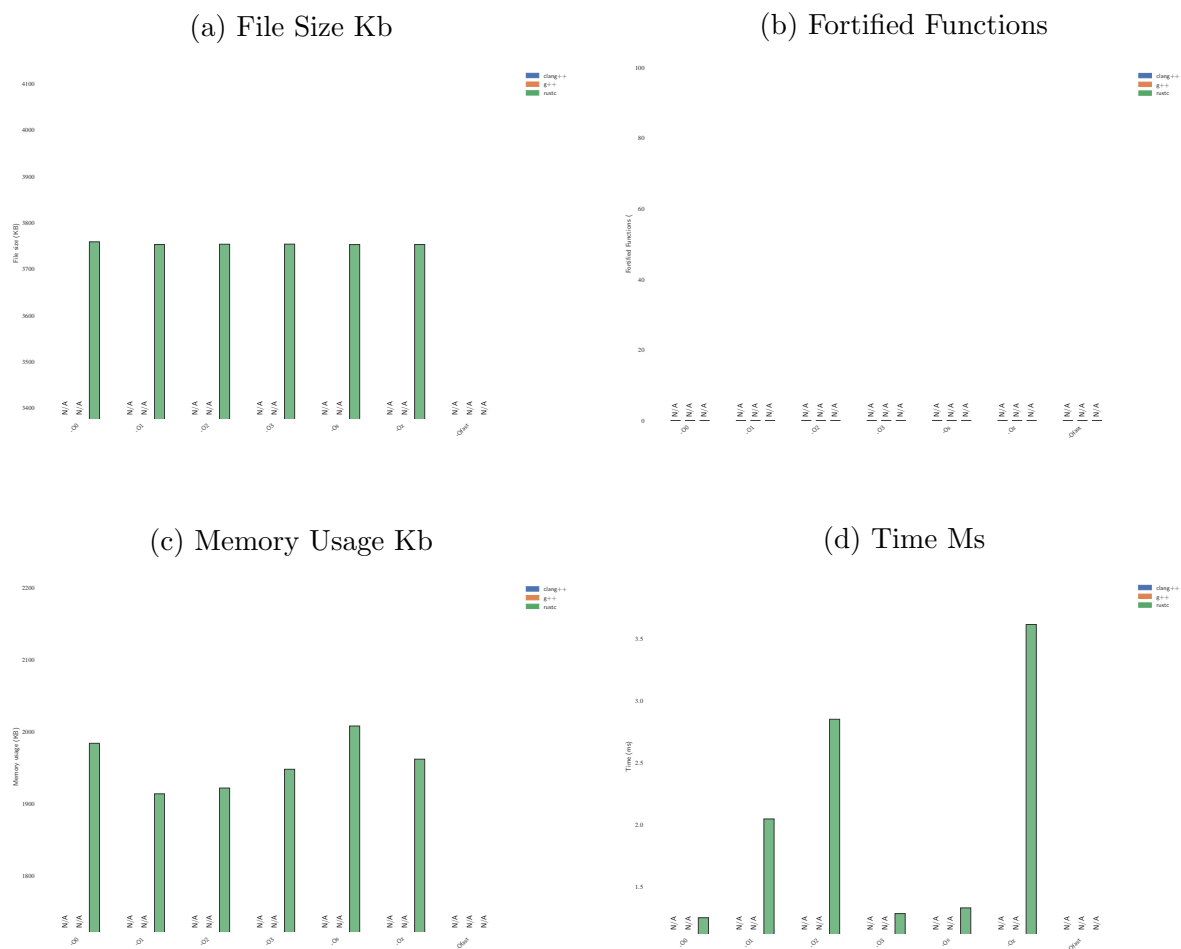
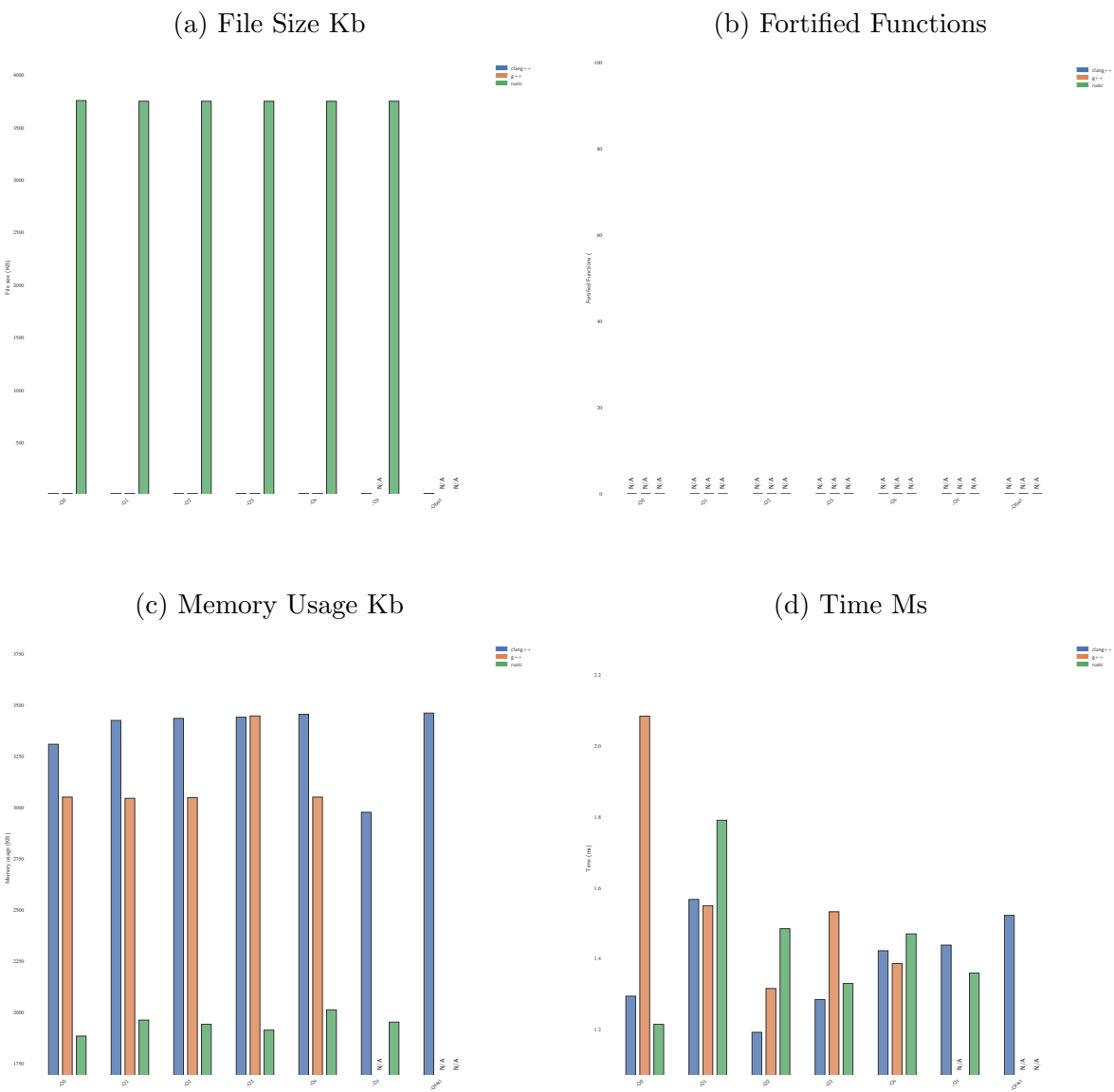
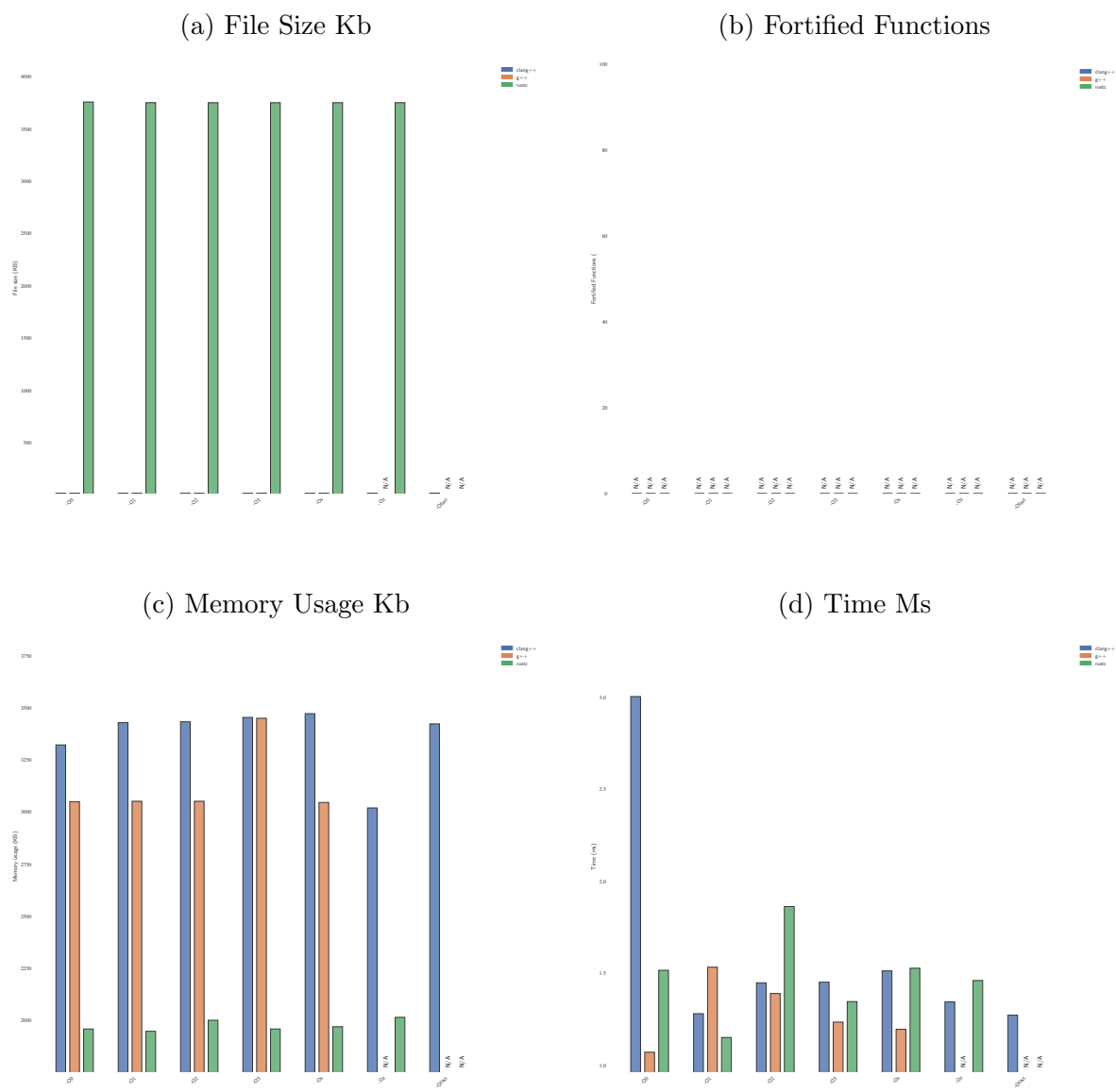


Figura 15: Resultados para execstack (disabled)



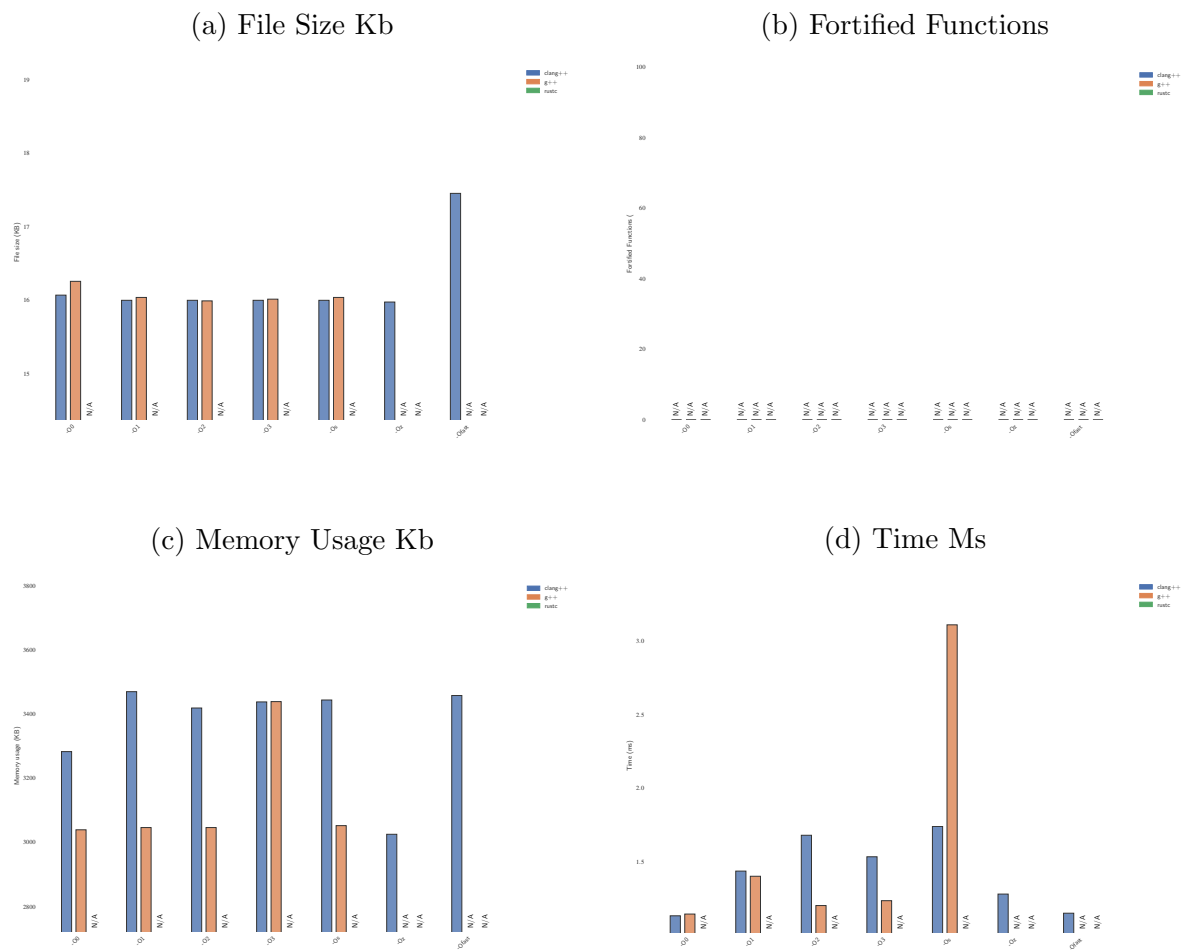
execstack / enabled

Figura 16: Resultados para execstack (enabled)



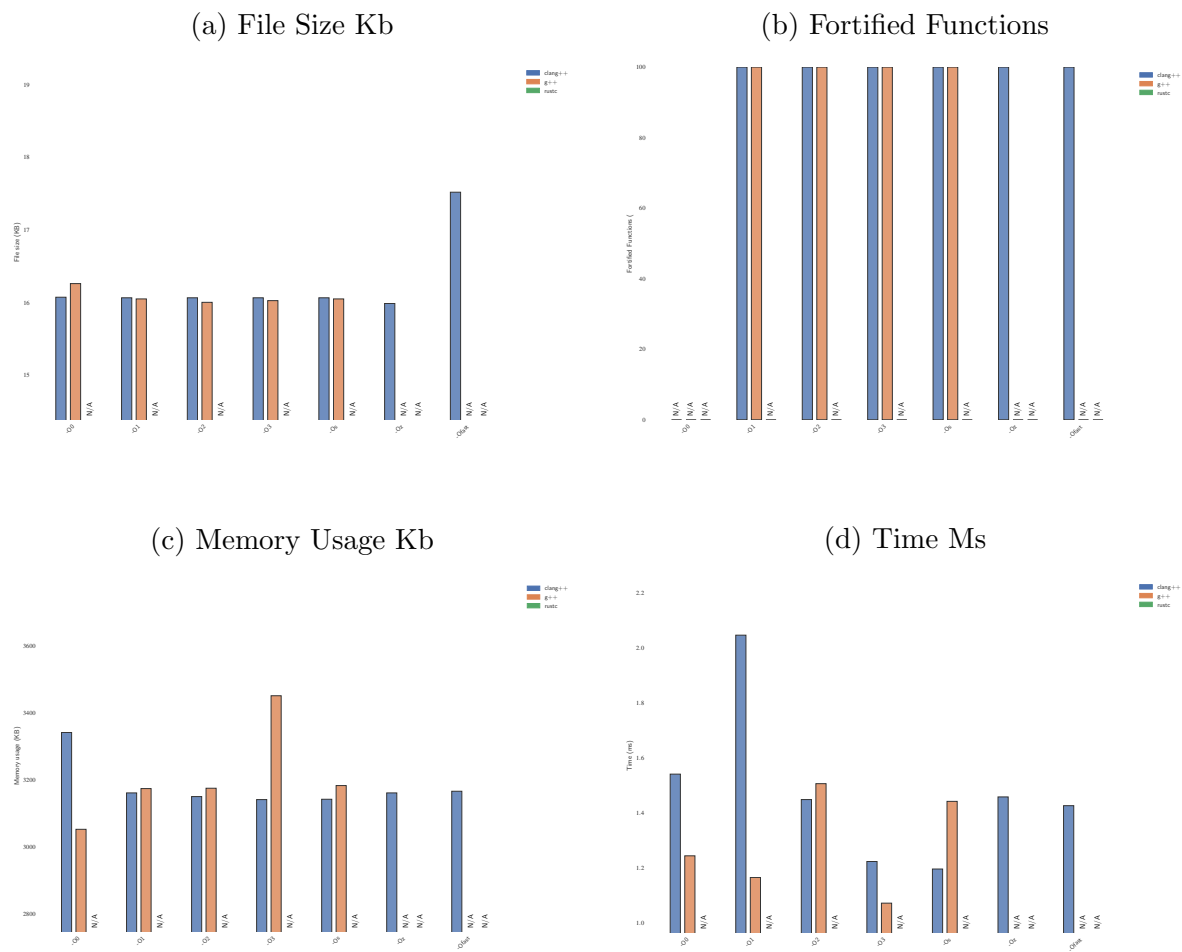
fortify-source / disabled

Figura 17: Resultados para fortify-source (disabled)



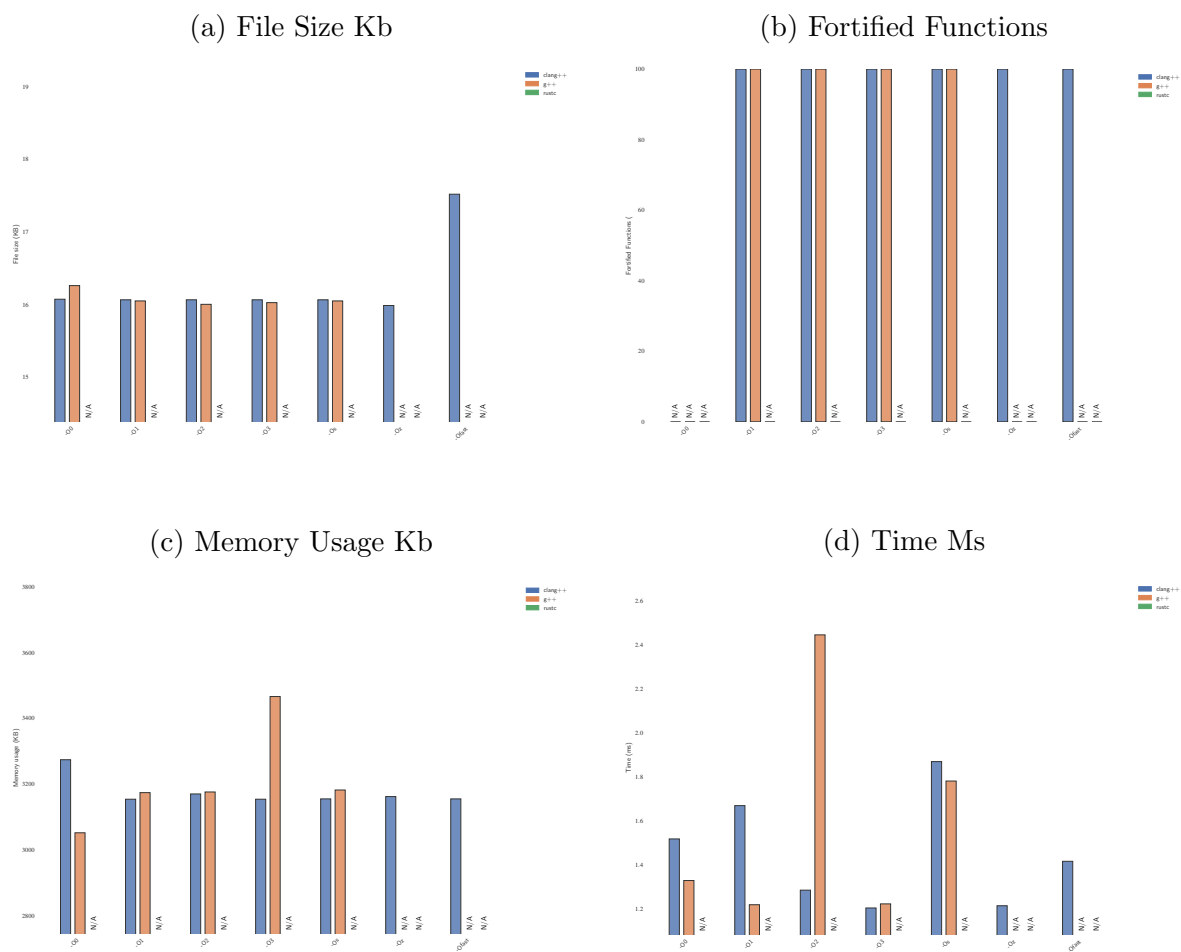
fortify-source / level-1

Figura 18: Resultados para fortify-source (level-1)



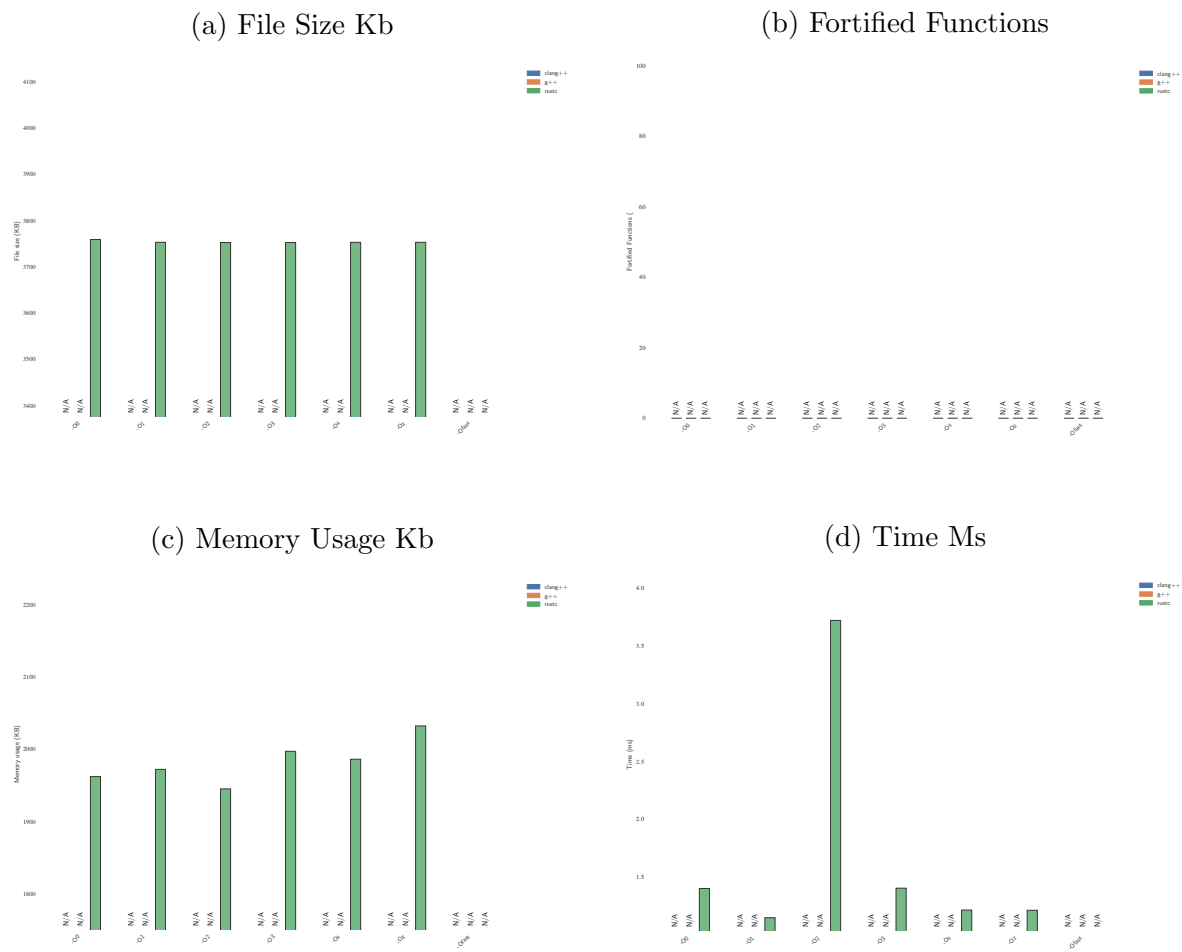
fortify-source / level-2

Figura 19: Resultados para fortify-source (level-2)



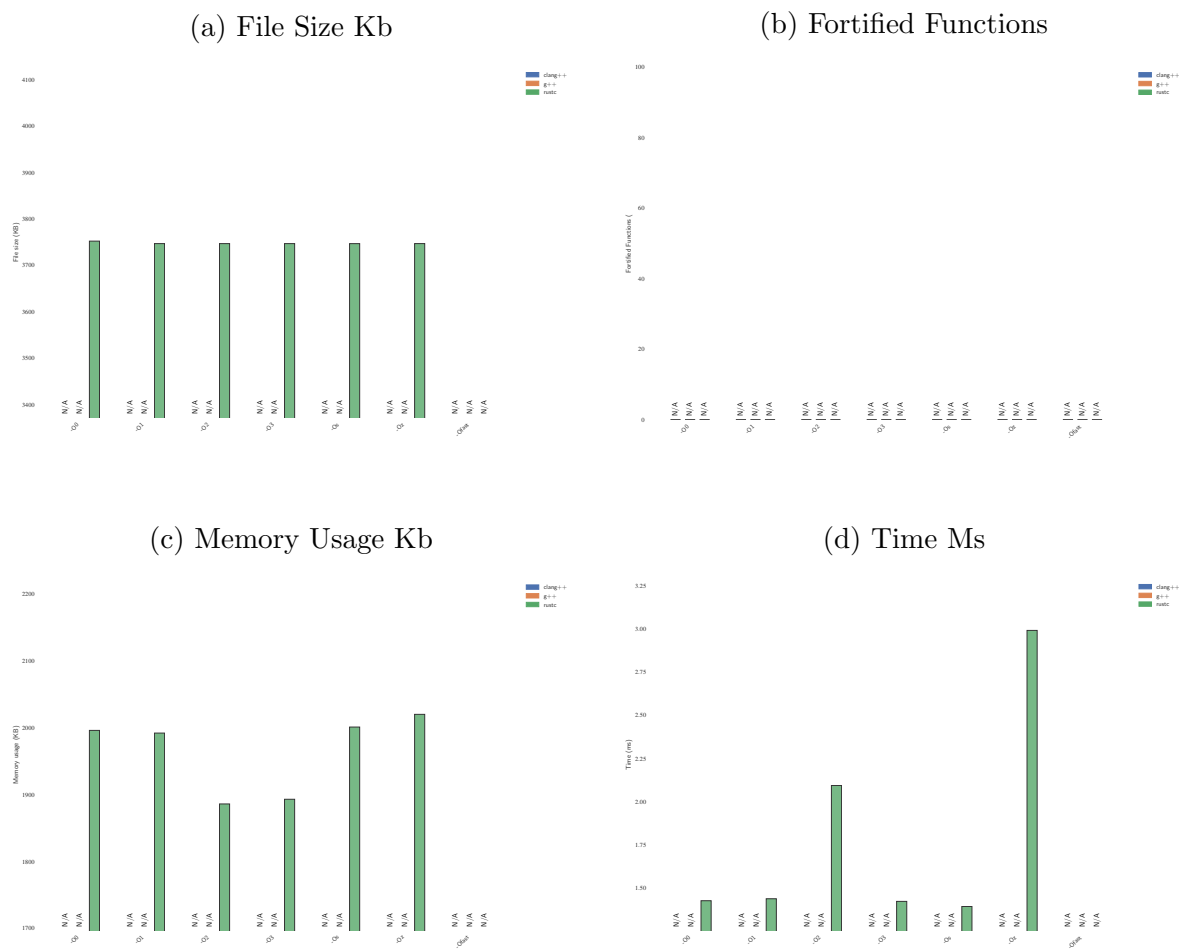
overflow-checks

Figura 20: Resultados de compilación con optimización overflow-checks



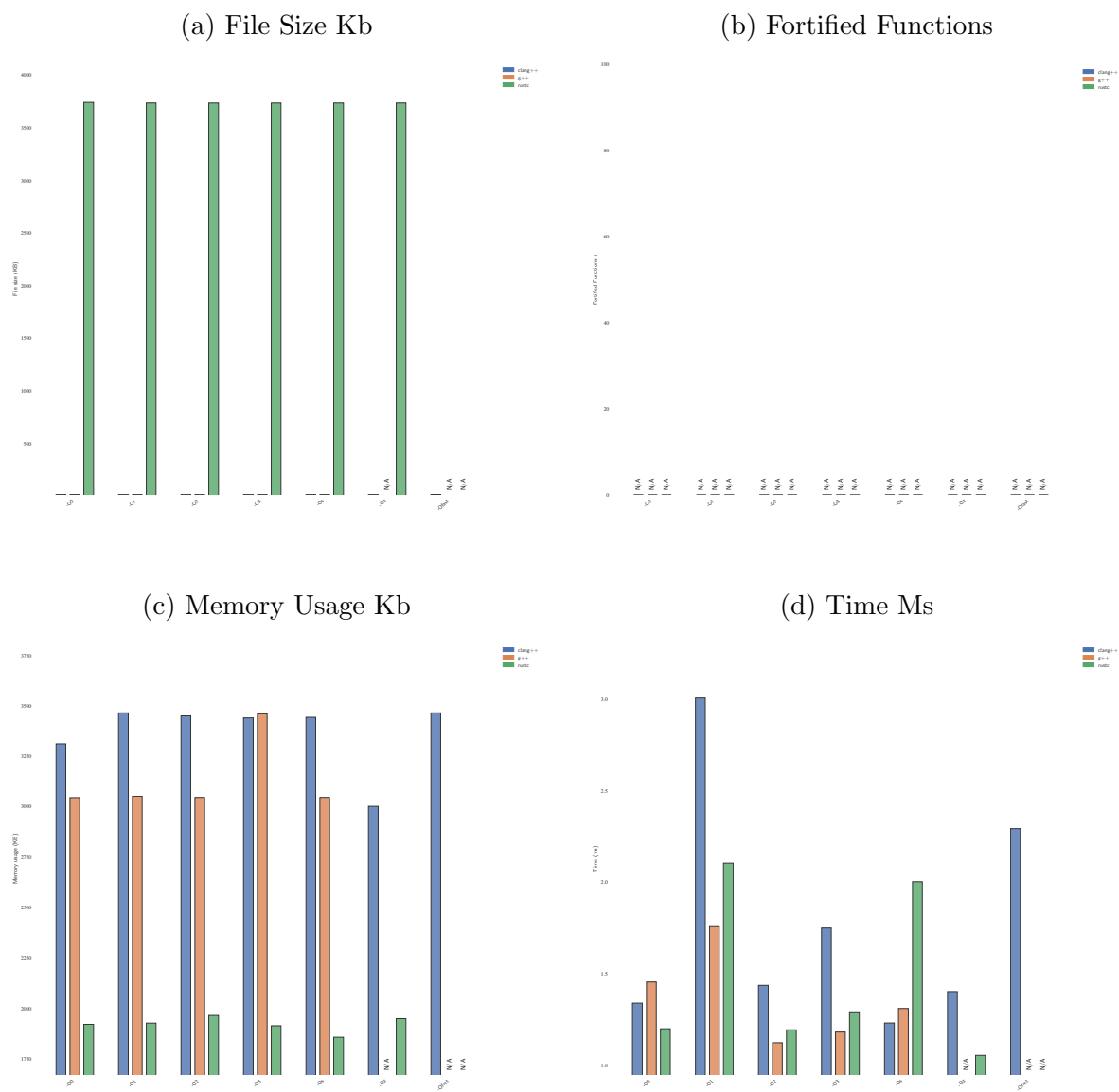
panic

Figura 21: Resultados de compilación con optimización panic



pie / disabled

Figura 22: Resultados para pie (disabled)



pie / enabled

Figura 23: Resultados para pie (enabled)

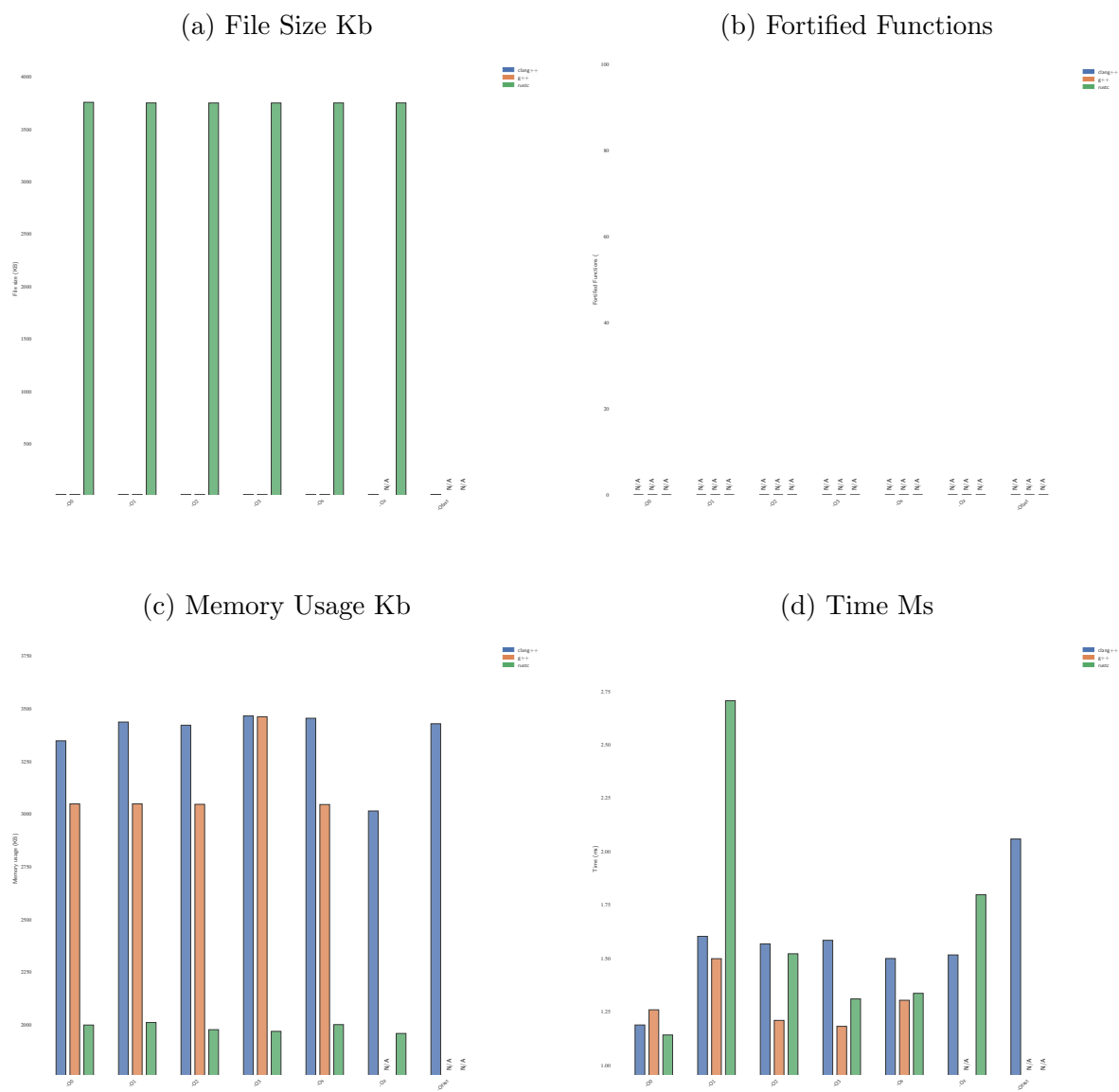
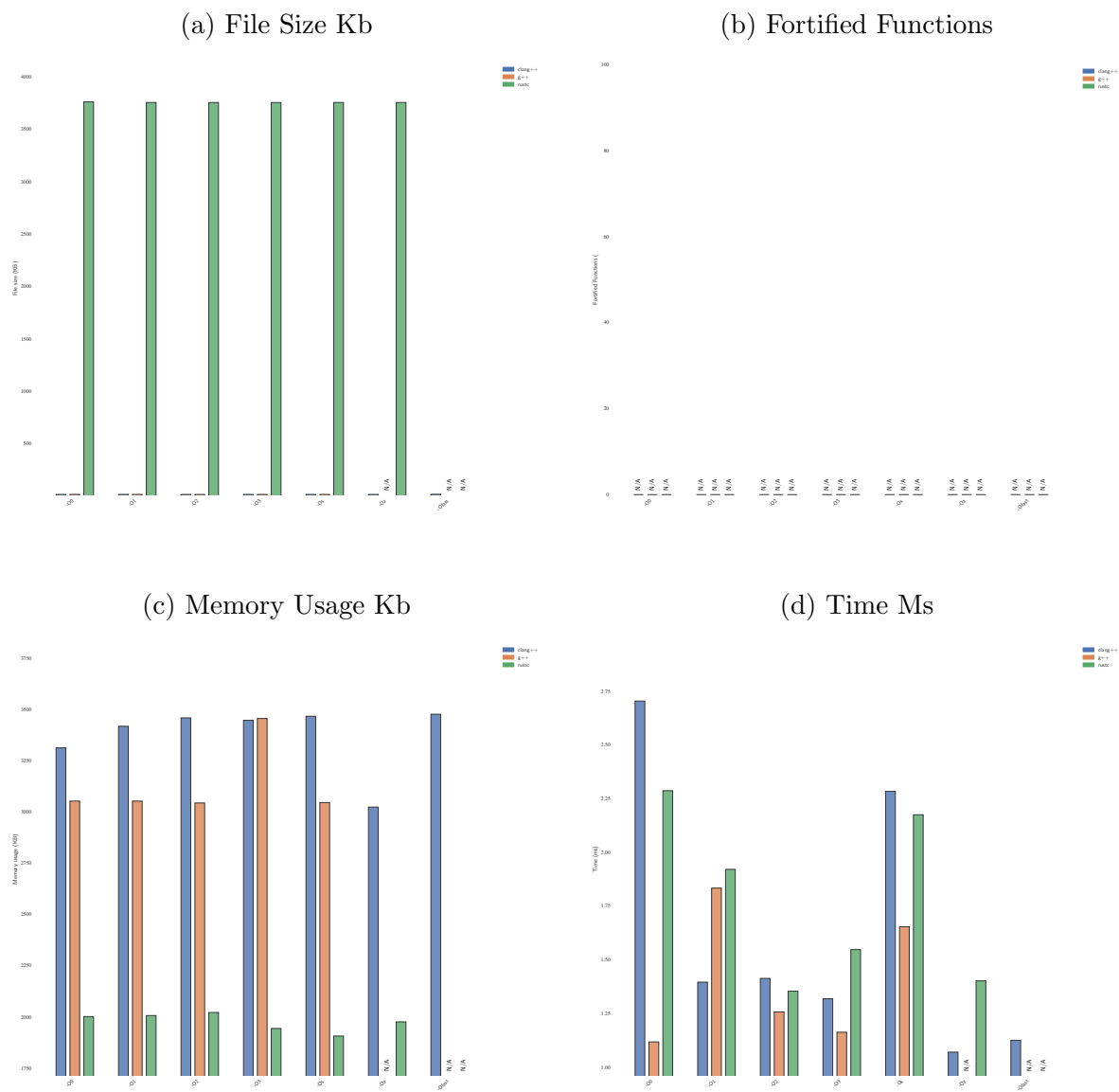


Figura 24: Resultados de compilación con optimización relro



safe-stack

Figura 25: Resultados de compilación con optimización safe-stack

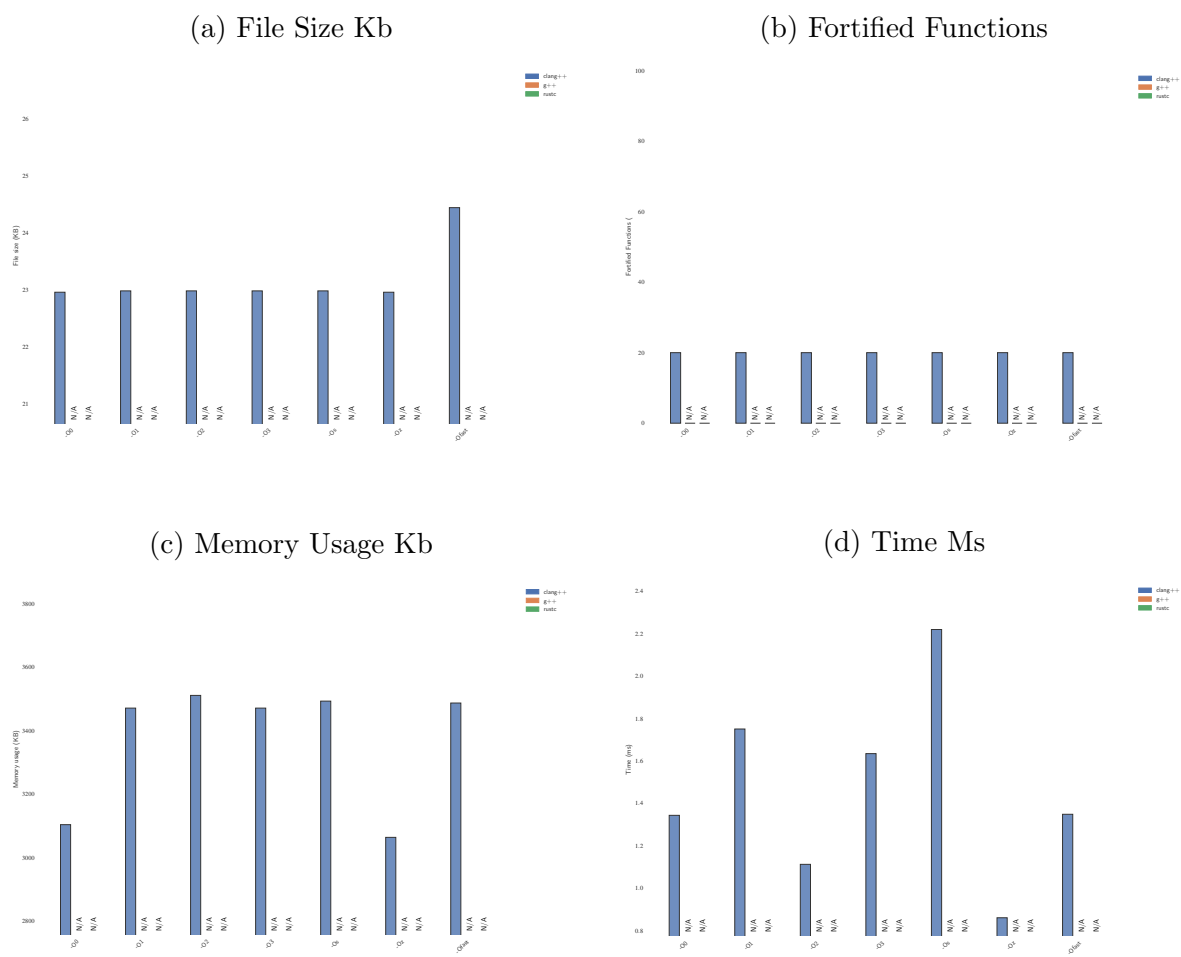


Figura 26: Resultados para sanitizers (address)

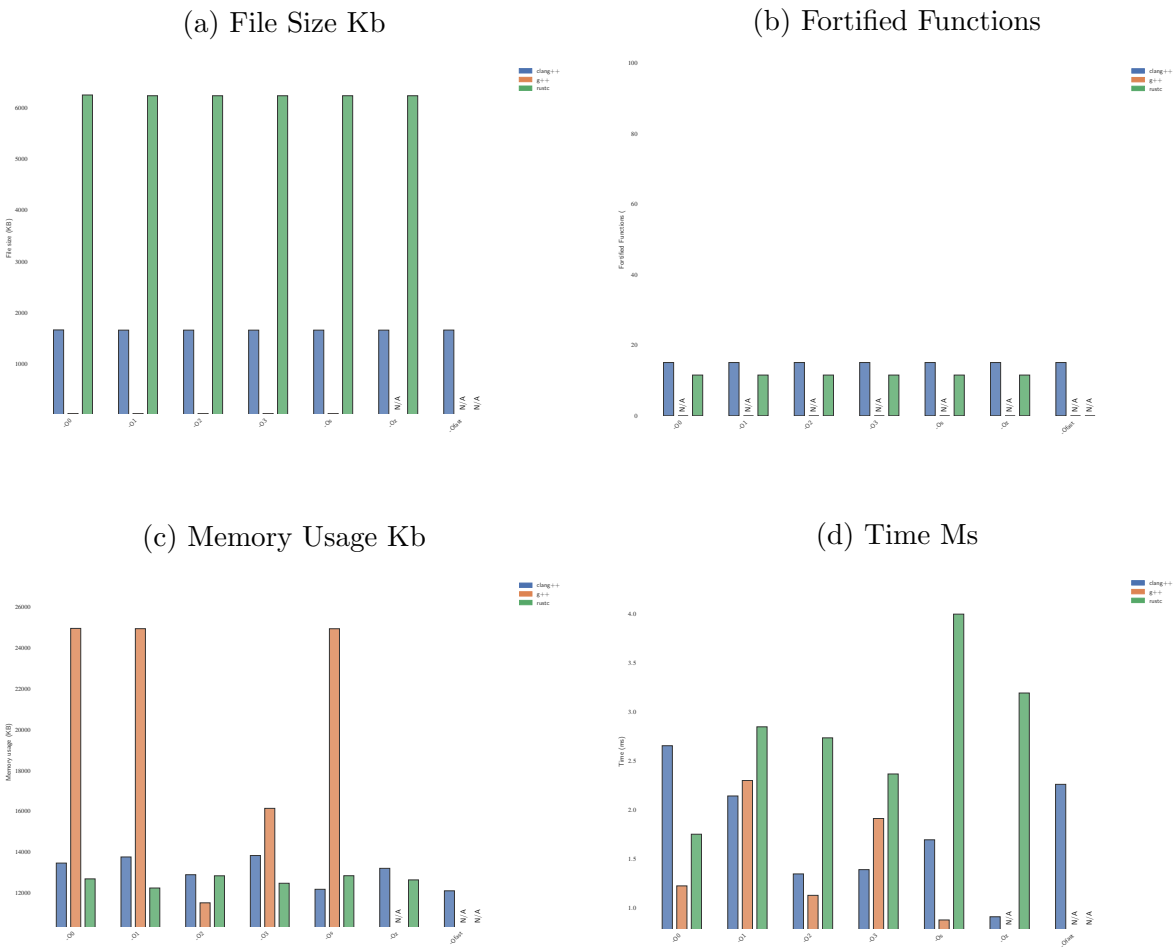


Figura 27: Resultados para sanitizers (undefined)

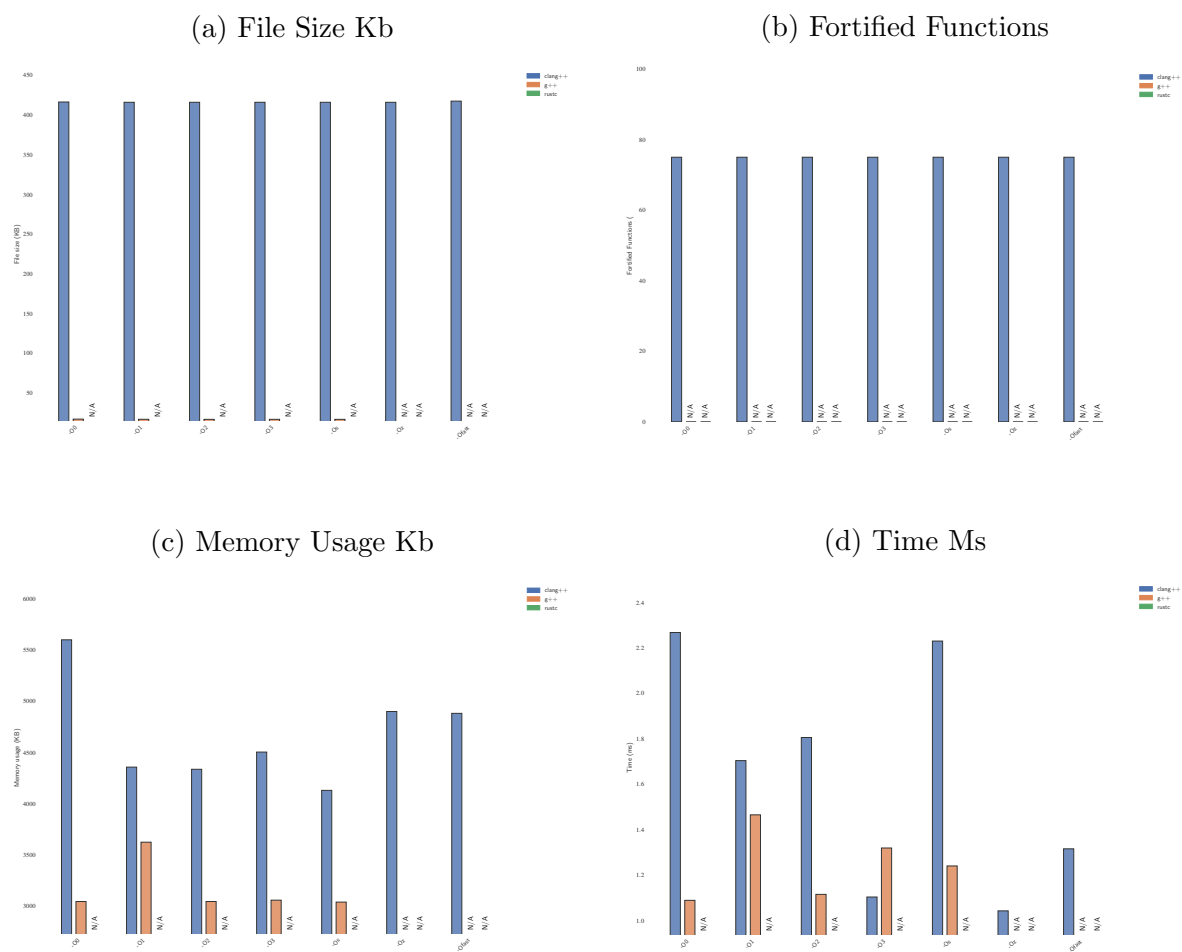
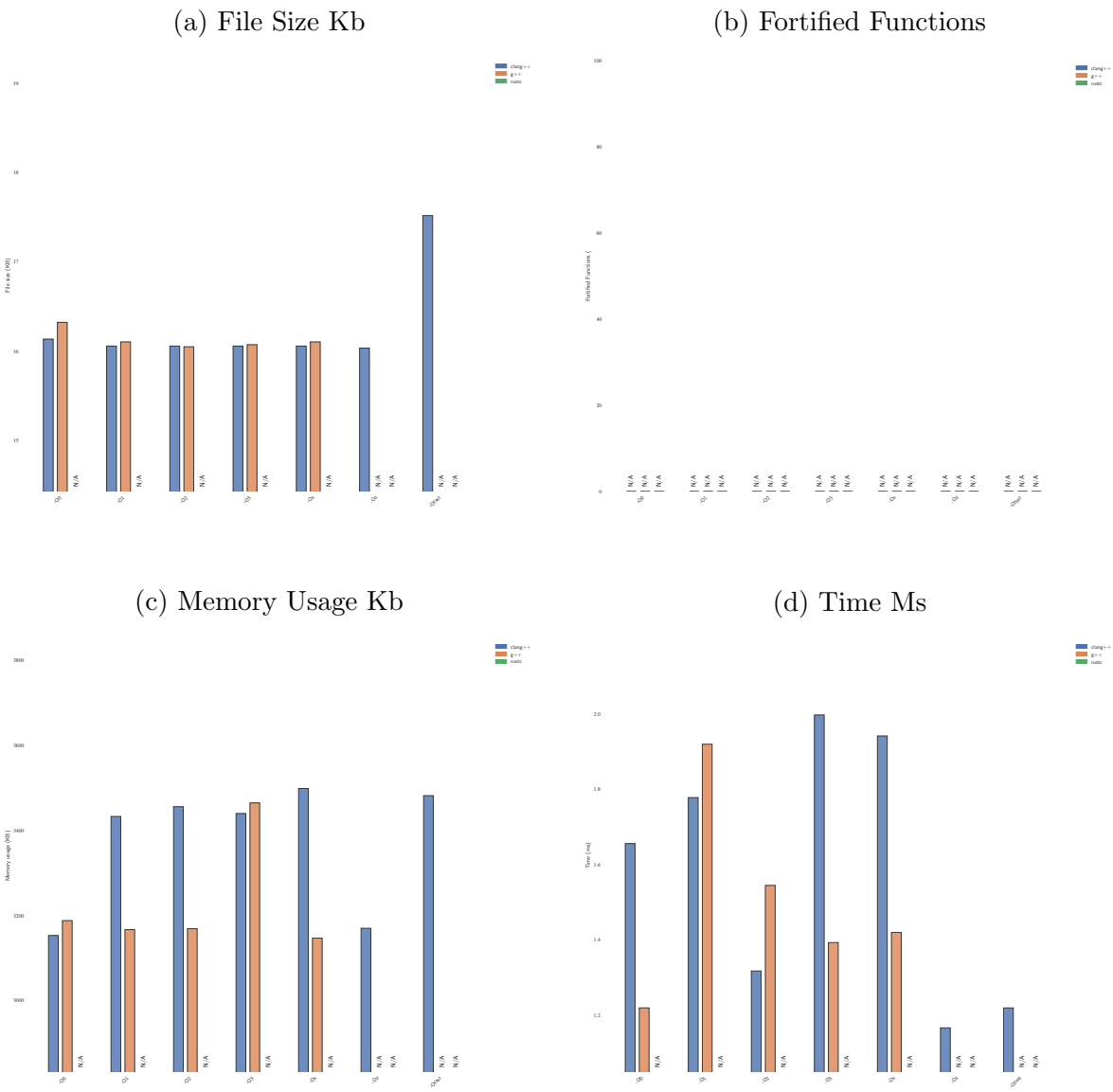
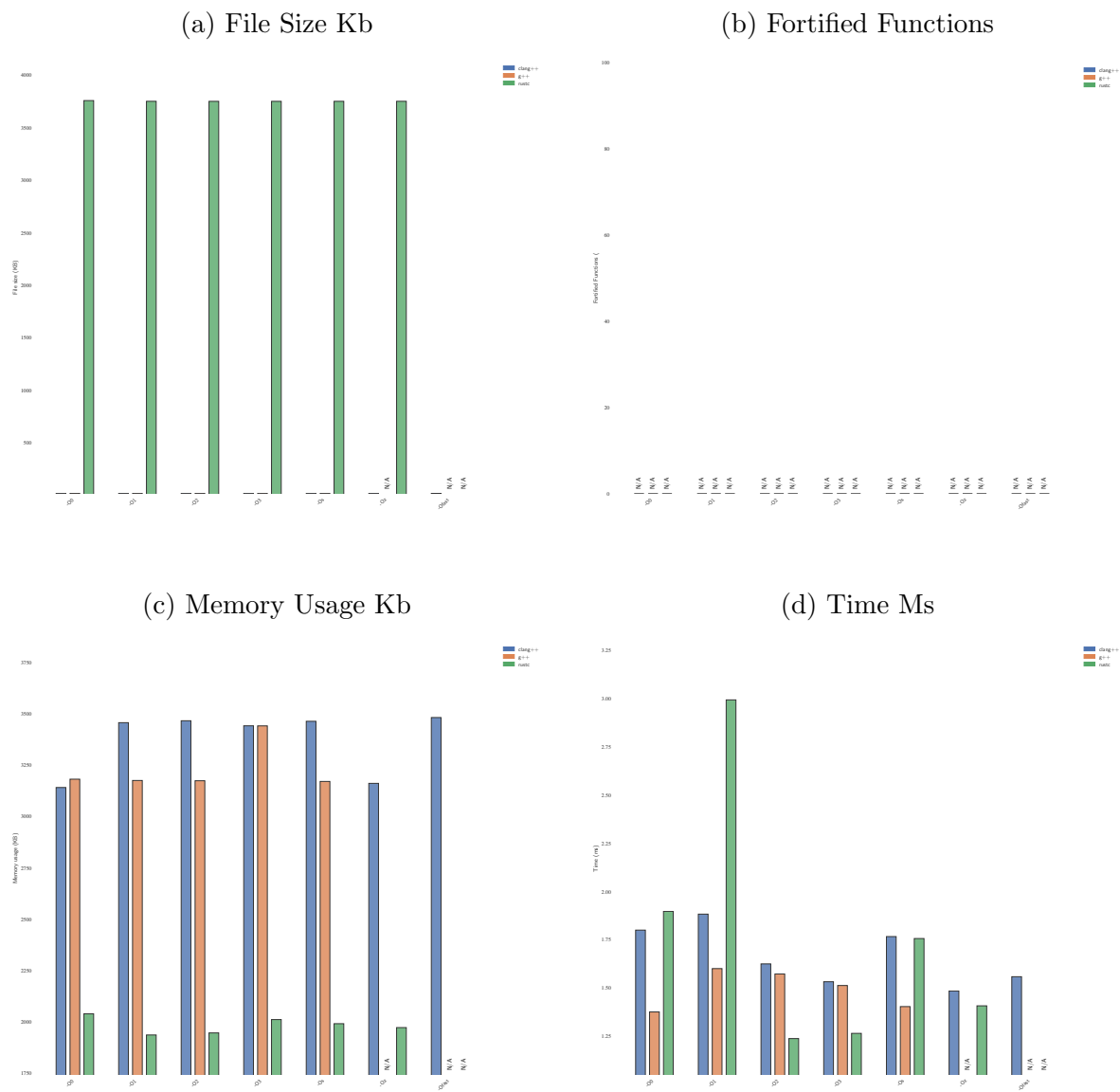


Figura 28: Resultados para stack-protection (all)



stack-protection / basic

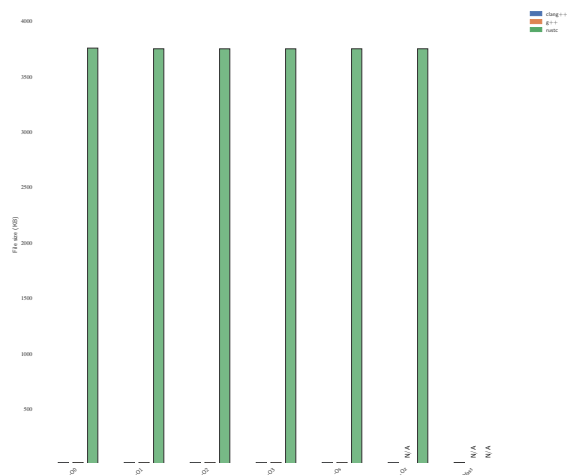
Figura 29: Resultados para stack-protection (basic)



stack-protection / disabled

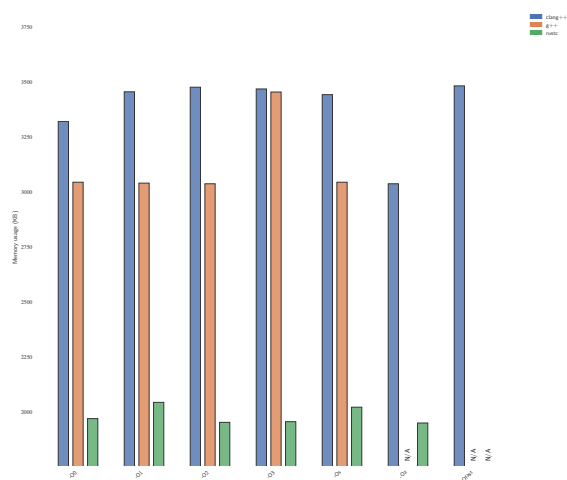
Figura 30: Resultados para stack-protection (disabled)

(a) File Size Kb

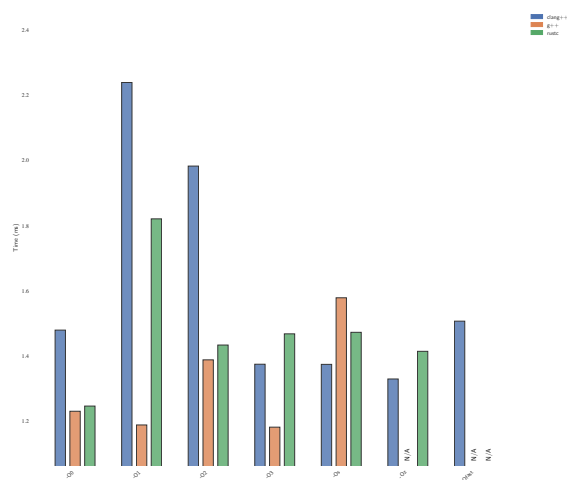


(b) Fortified Functions

(c) Memory Usage Kb

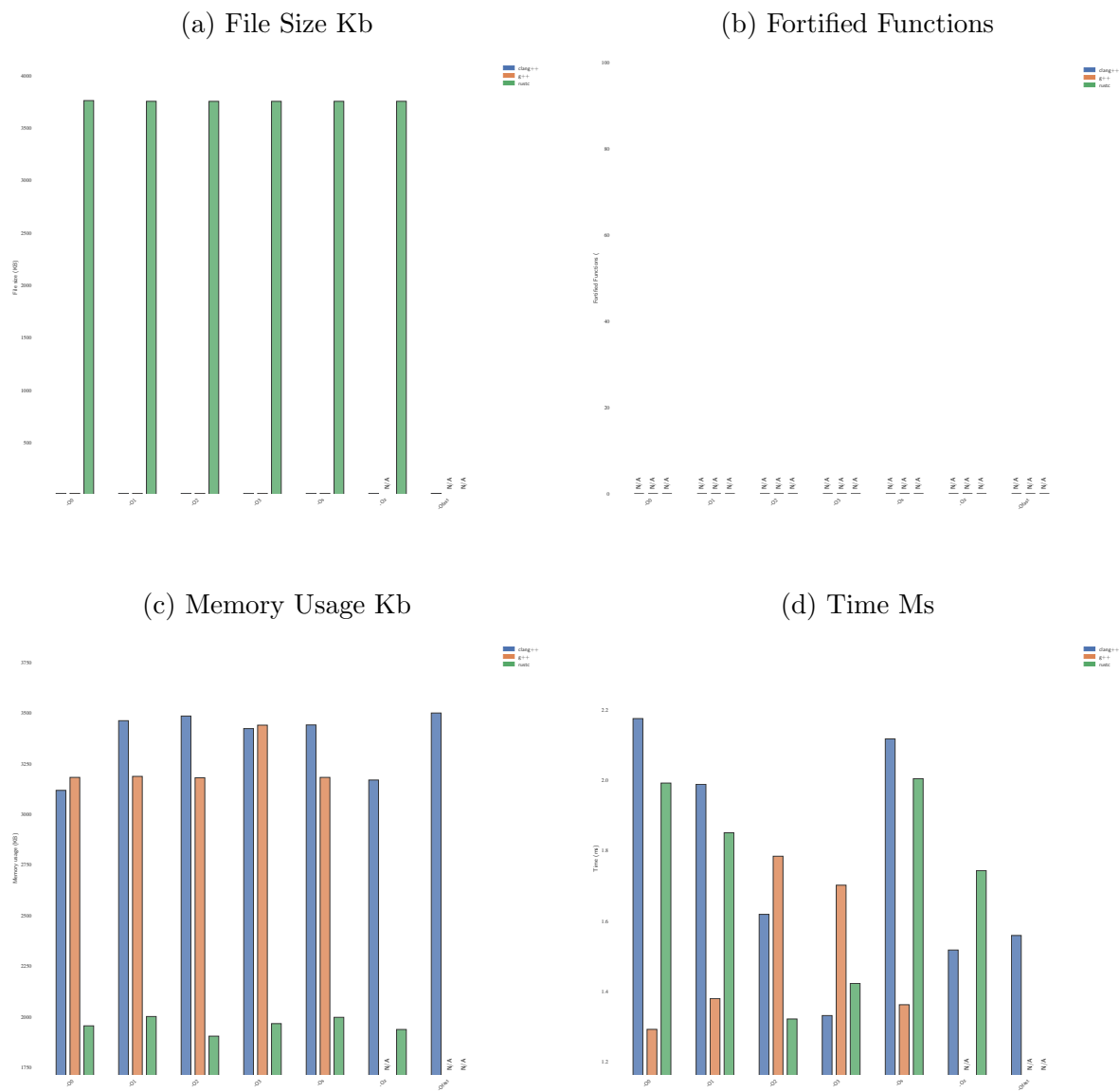


(d) Time Ms



stack-protection / strong

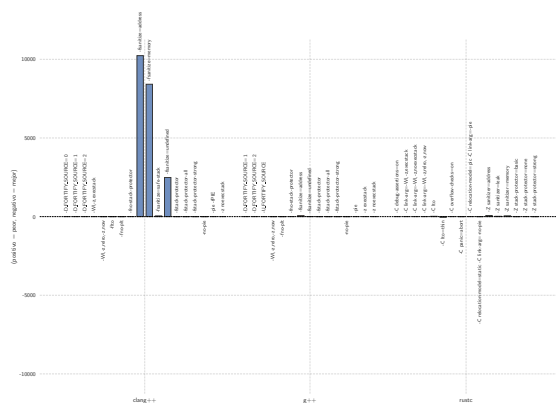
Figura 31: Resultados para stack-protection (strong)



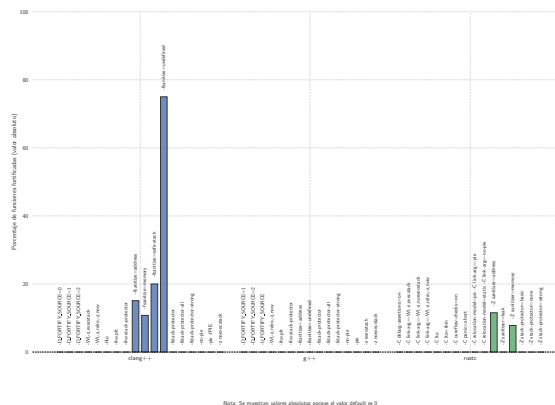
-00

Figura 32: Resultados de compilación con optimización -O0

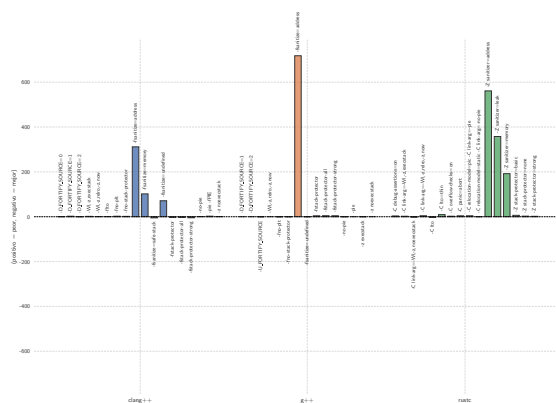
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

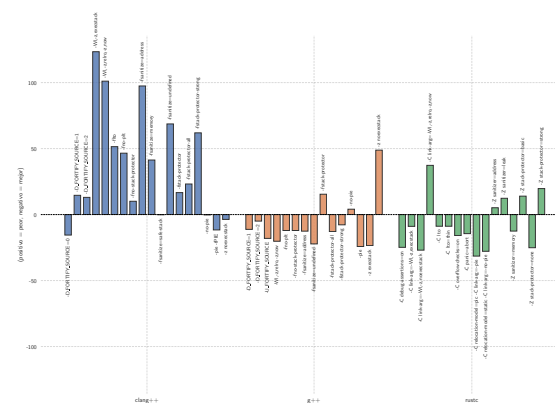
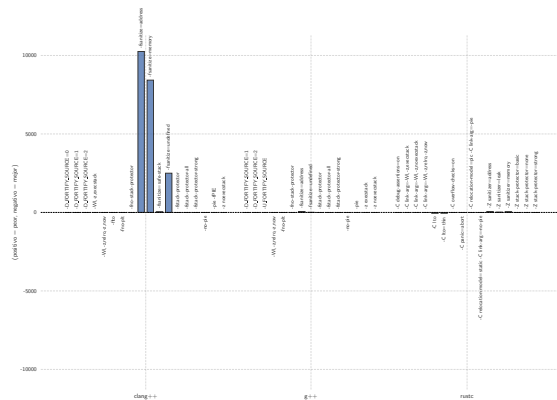
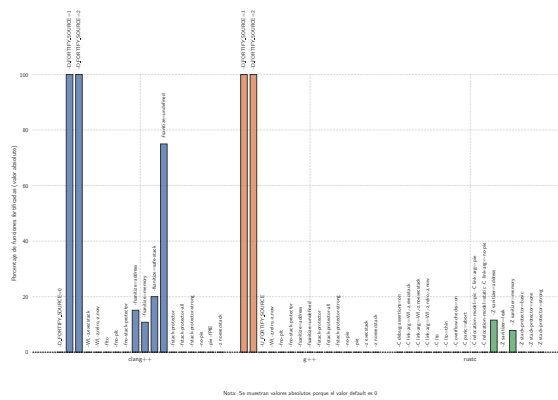


Figura 33: Resultados de compilación con optimización -O1

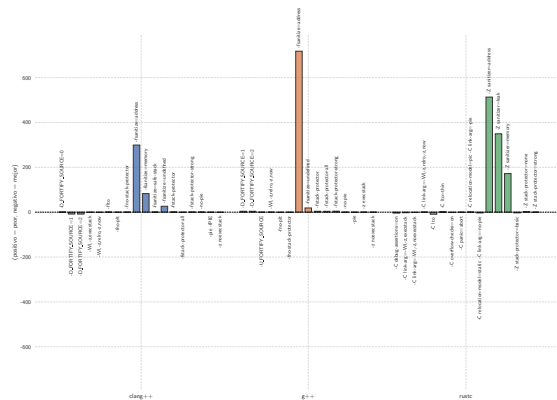
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

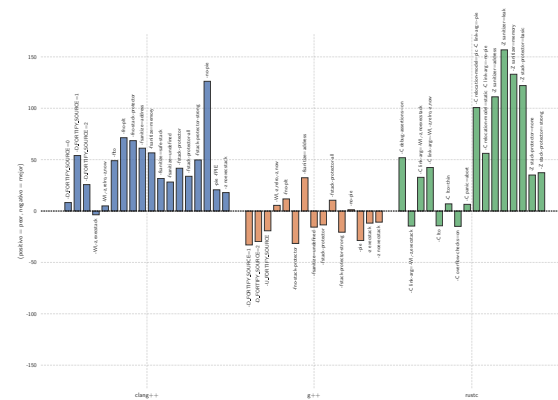
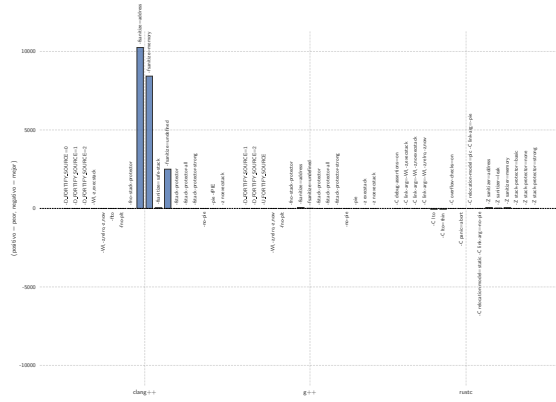
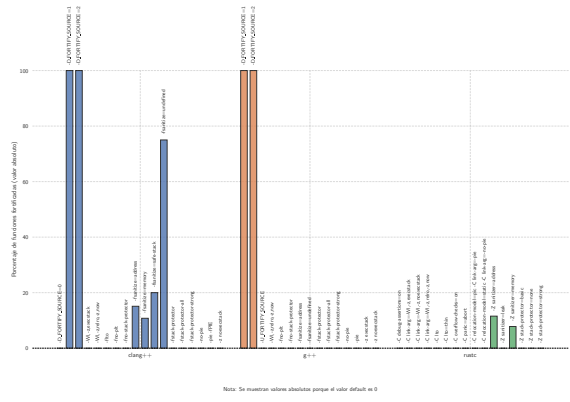


Figura 34: Resultados de compilación con optimización -O2

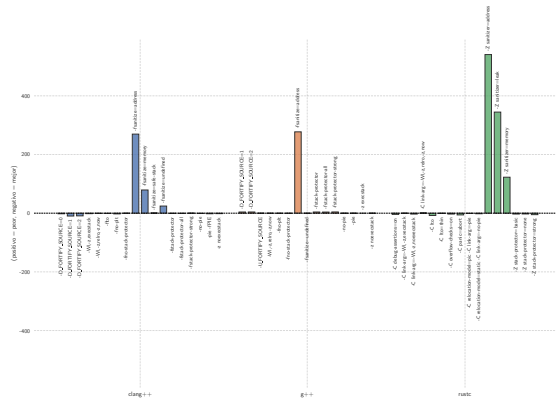
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

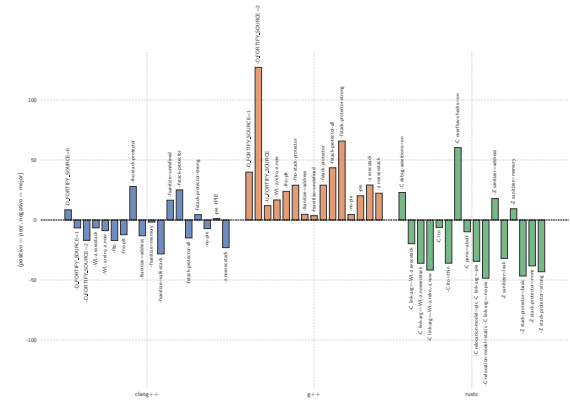
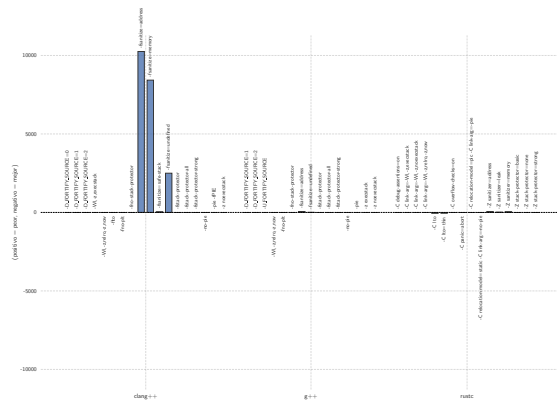
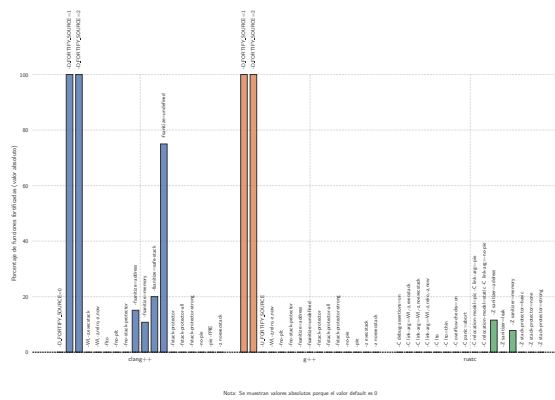


Figura 35: Resultados de compilación con optimización -O3

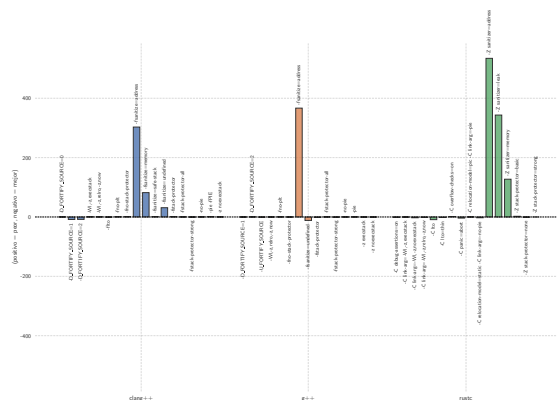
(a) File Size Kb



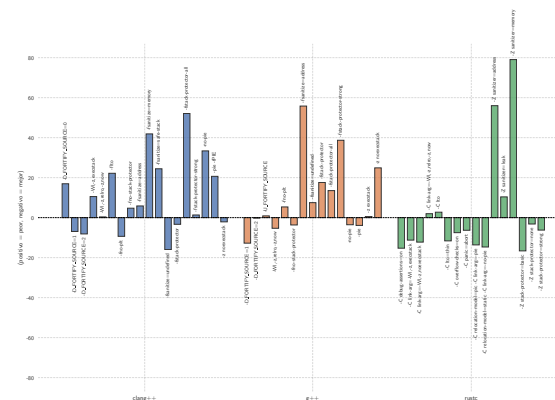
(b) Fortified Functions



(c) Memory Usage Kb



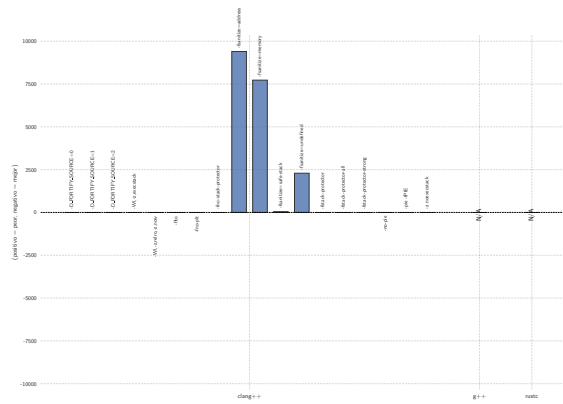
(d) Time Ms



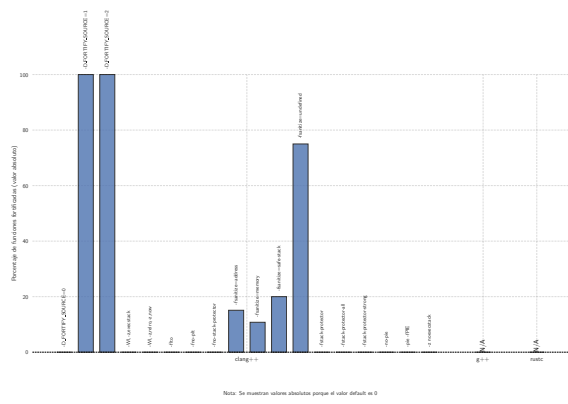
-OFAST

Figura 36: Resultados de compilación con optimización -OFAST

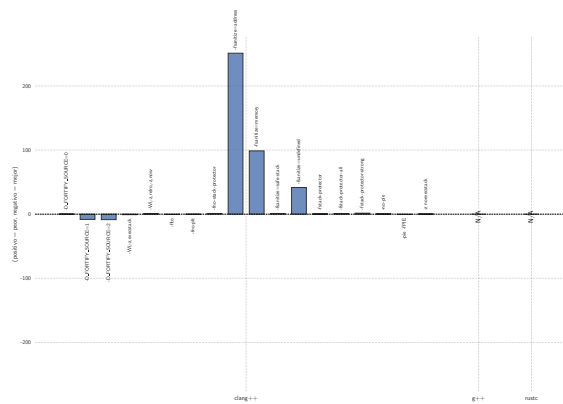
(a) File Size Kb



(b) Fortified Functions



(c) Memory Usage Kb



(d) Time Ms

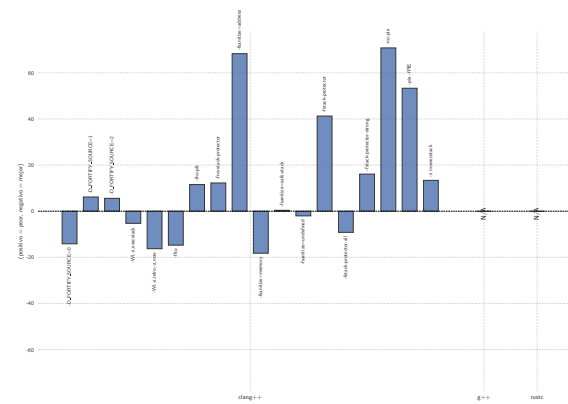
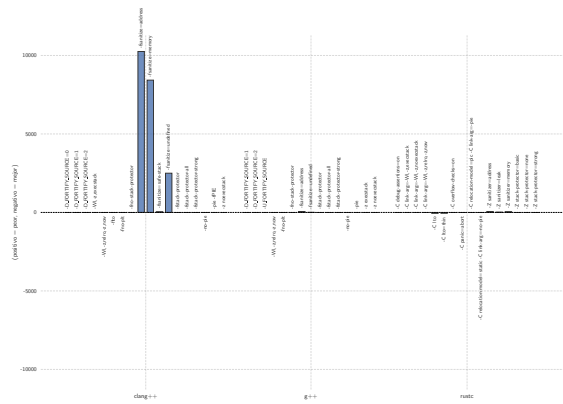
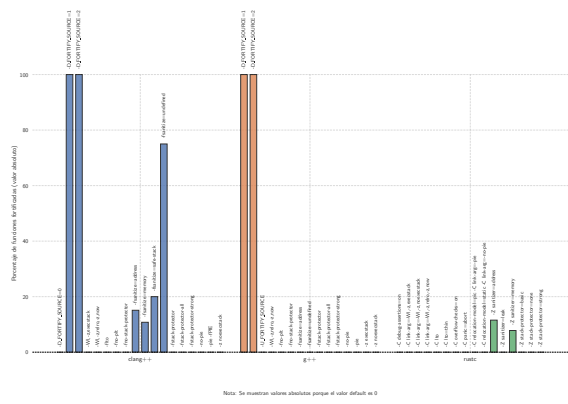


Figura 37: Resultados de compilación con optimización -OS

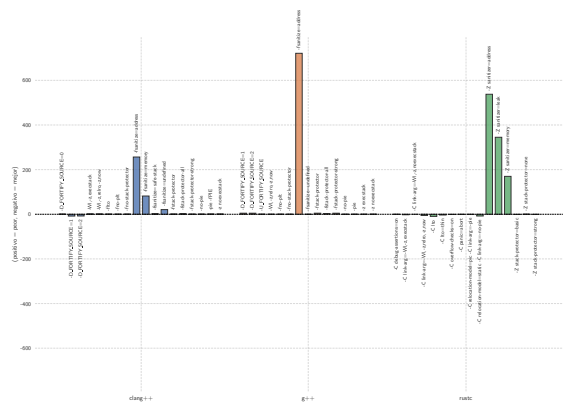
(a) File Size Kb



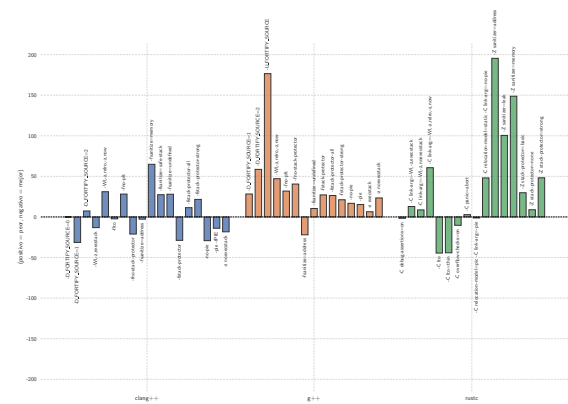
(b) Fortified Functions



(c) Memory Usage Kb



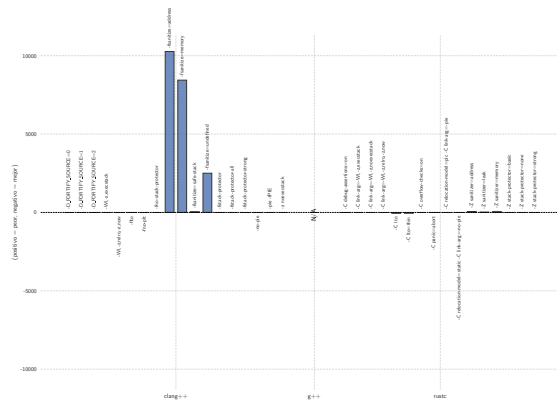
(d) Time Ms



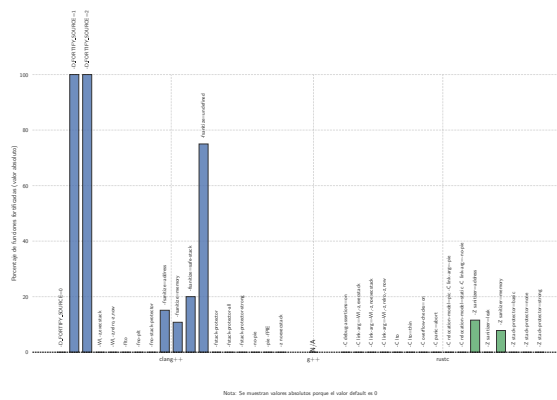
-OZ

Figura 38: Resultados de compilación con optimización -OZ

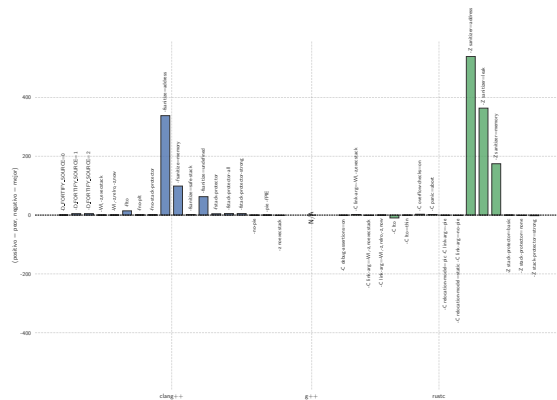
(a) File Size Kb



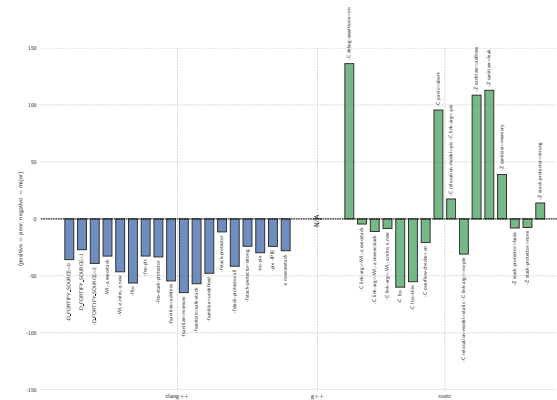
(b) Fortified Functions



(c) Memory Usage Kb



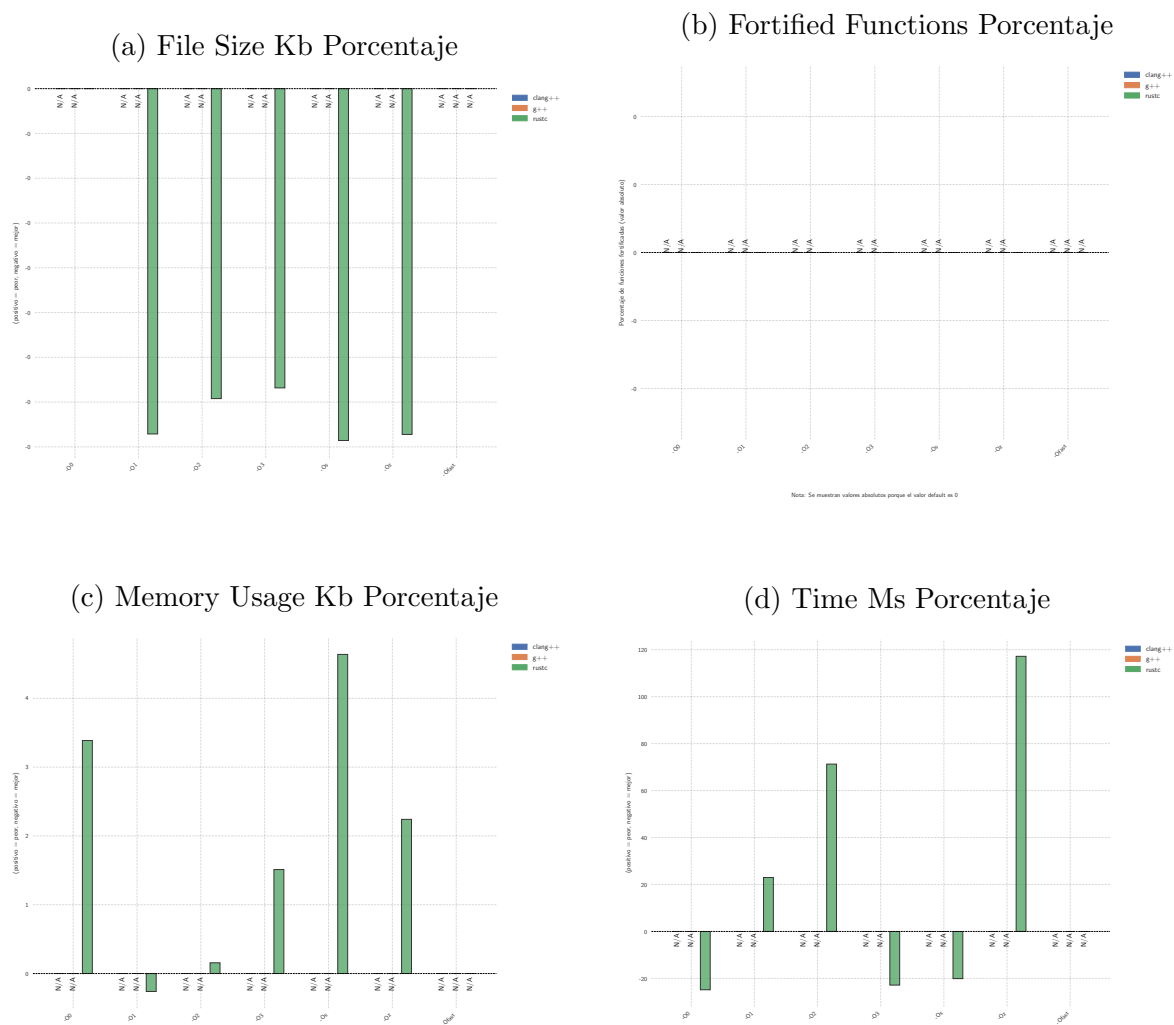
(d) Time Ms



Gráficas Por Medida Seguridad Porcentual

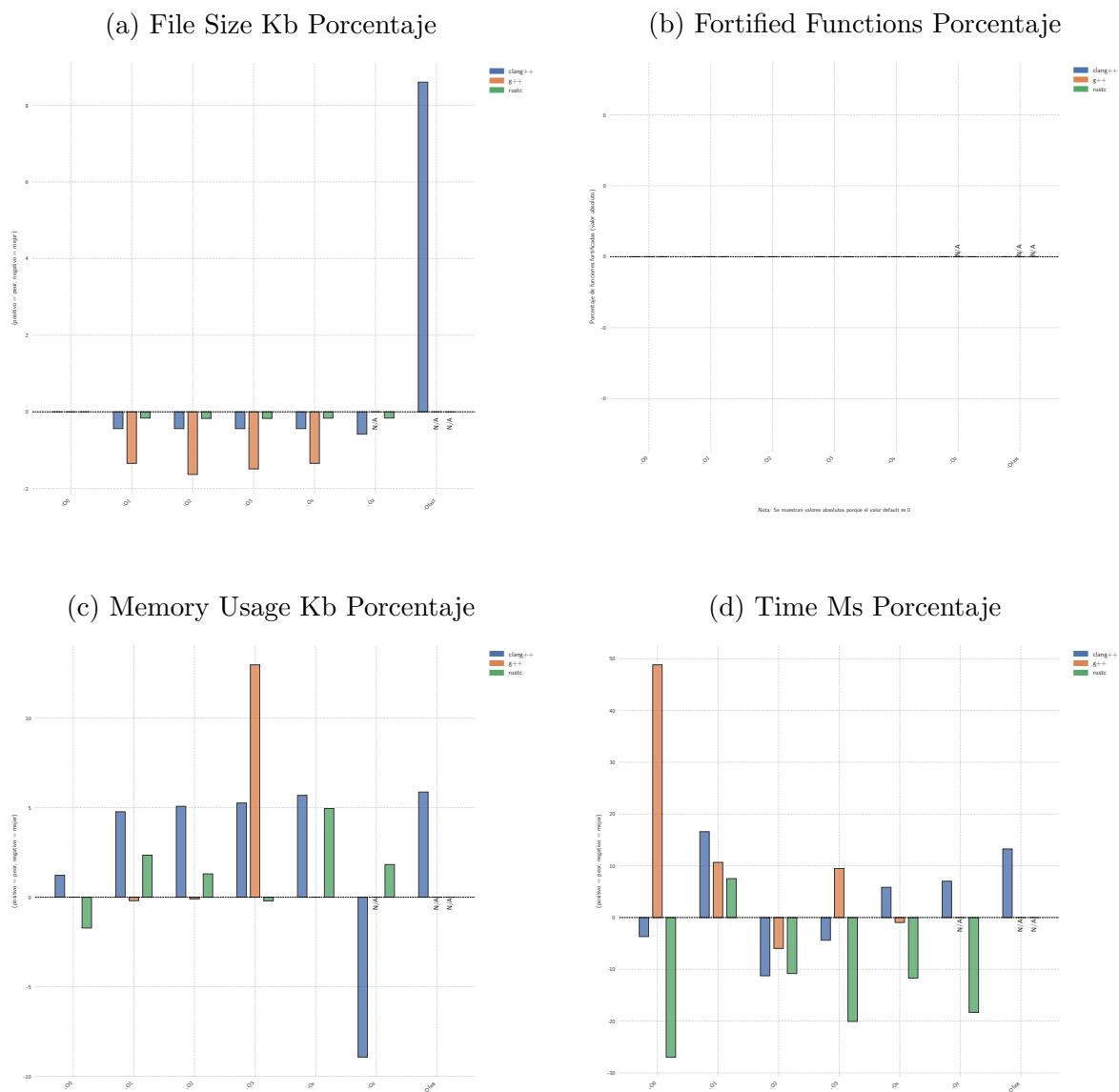
debug-assertions

Figura 39: Resultados de compilación con optimización debug-assertions



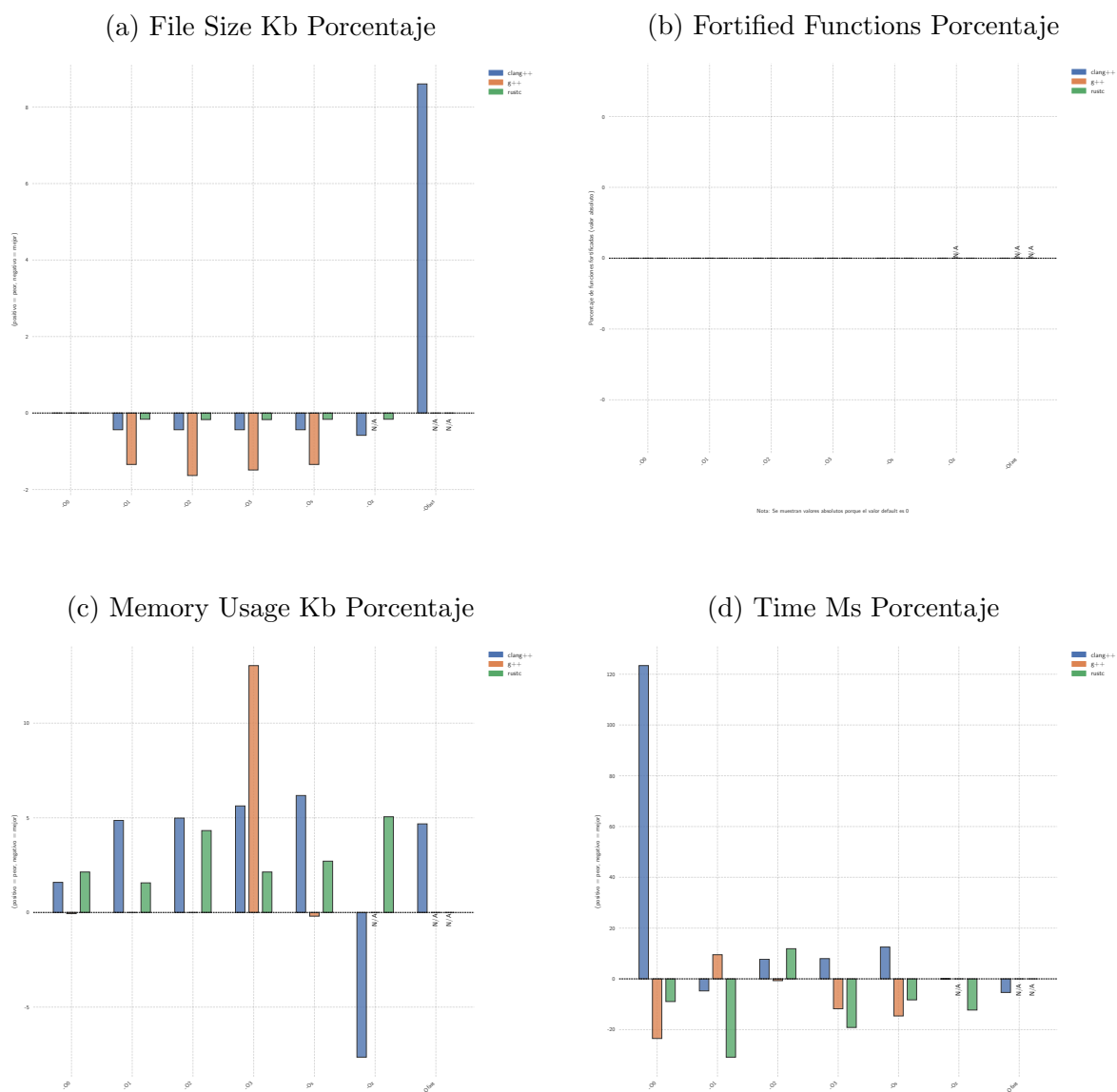
execstack / disabled

Figura 40: Resultados para execstack (disabled)



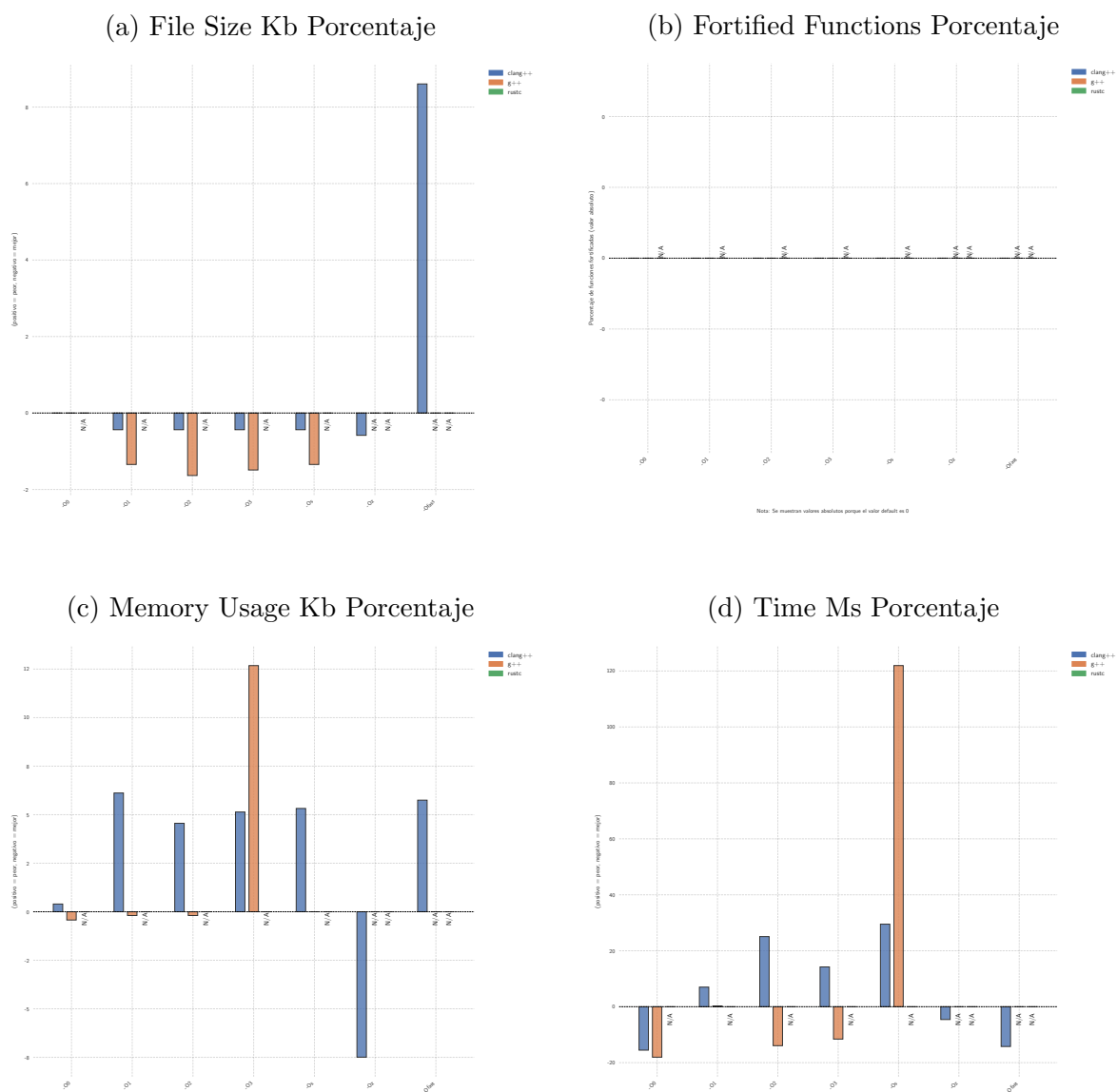
execstack / enabled

Figura 41: Resultados para execstack (enabled)



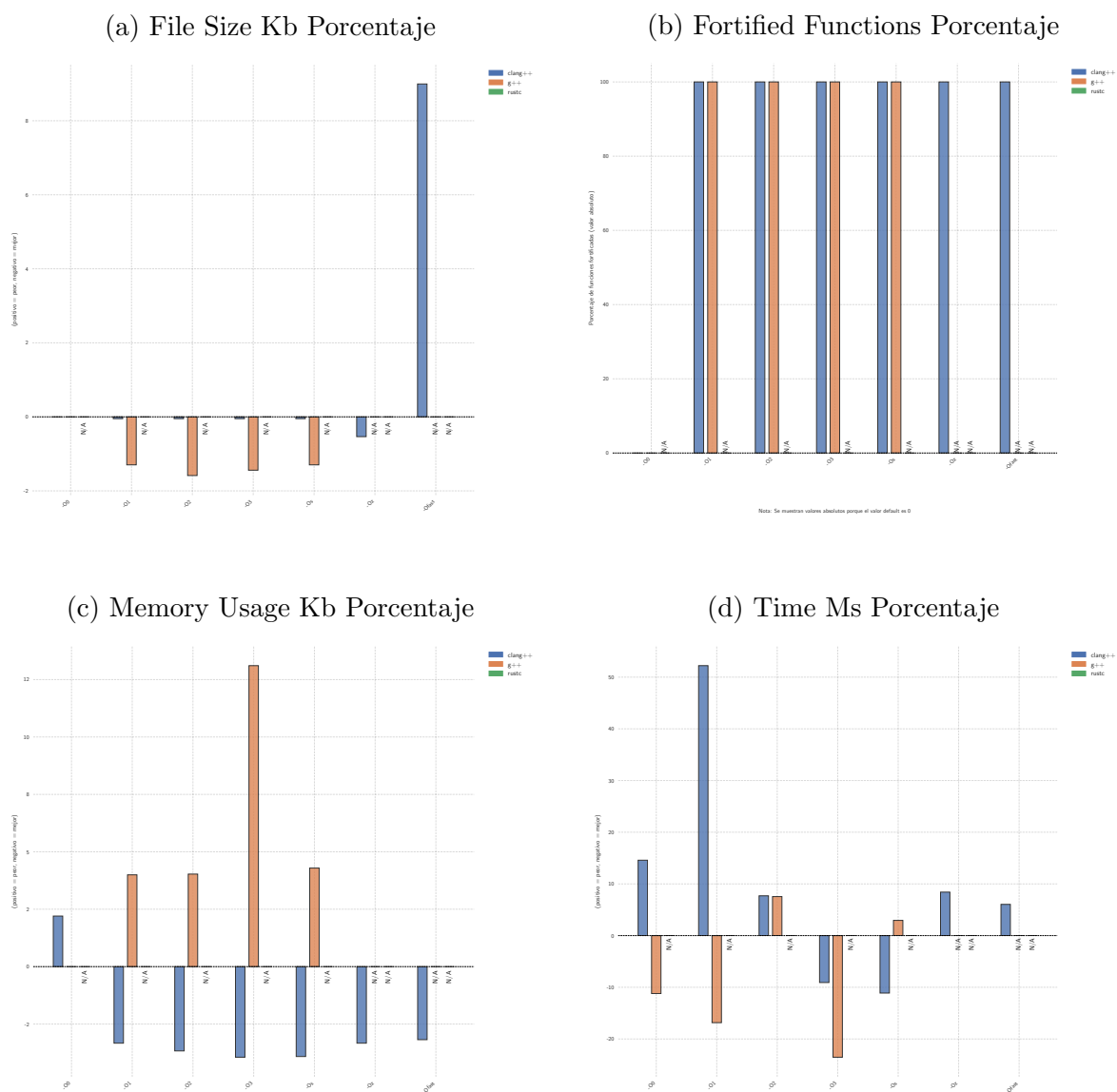
fortify-source / disabled

Figura 42: Resultados para fortify-source (disabled)



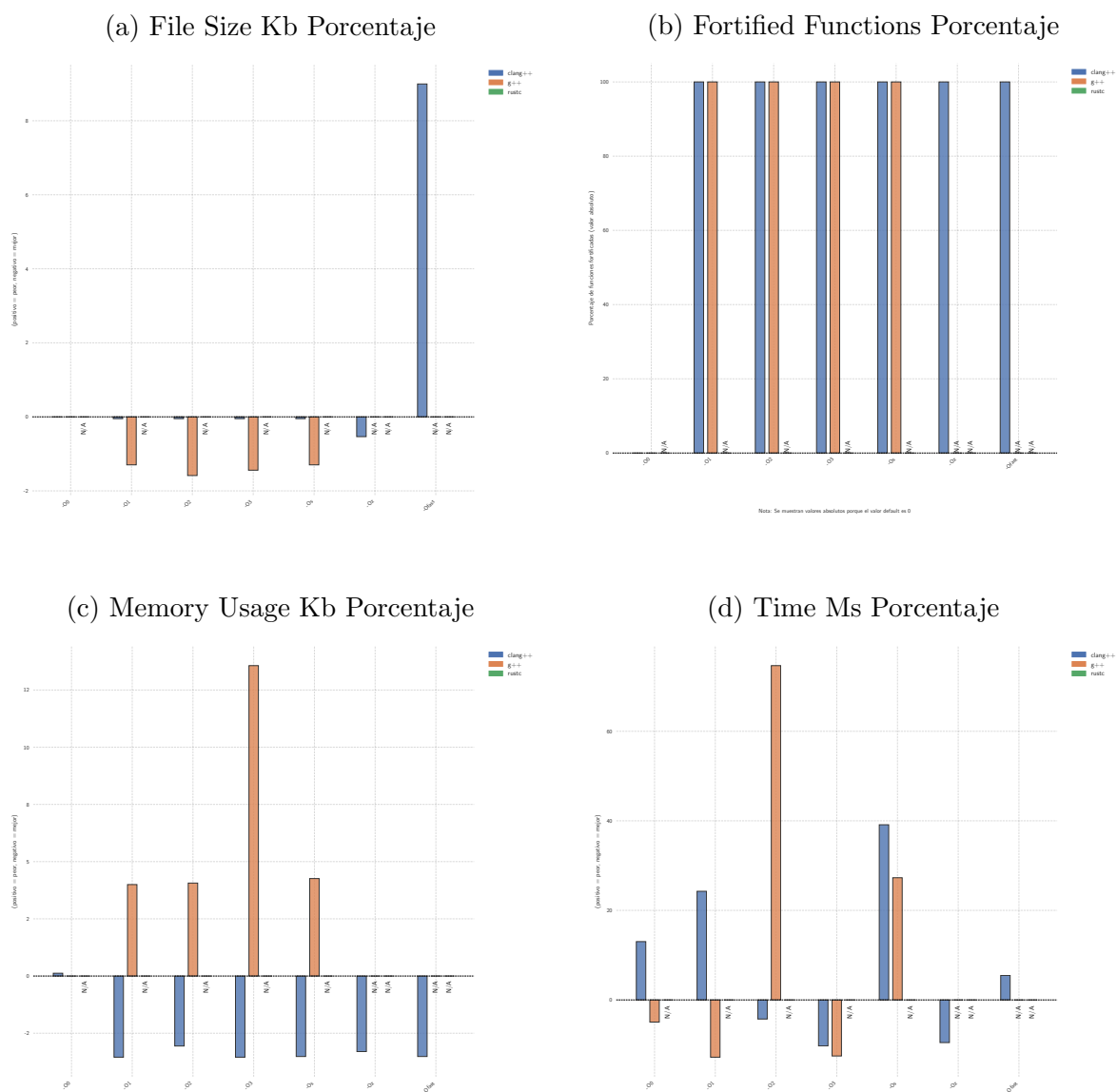
fortify-source / level-1

Figura 43: Resultados para fortify-source (level-1)



fortify-source / level-2

Figura 44: Resultados para fortify-source (level-2)



overflow-checks

Figura 45: Resultados de compilación con optimización overflow-checks

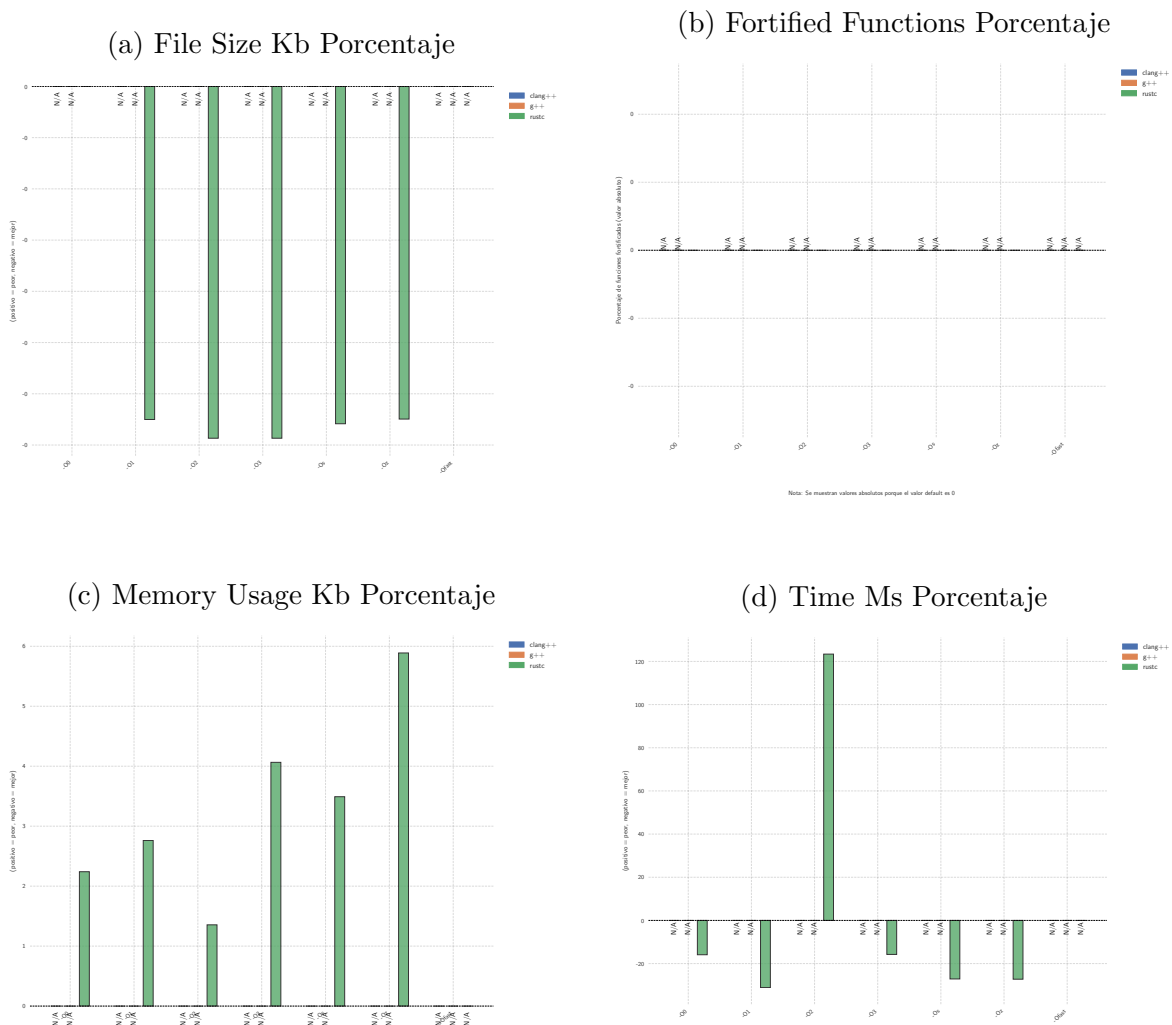
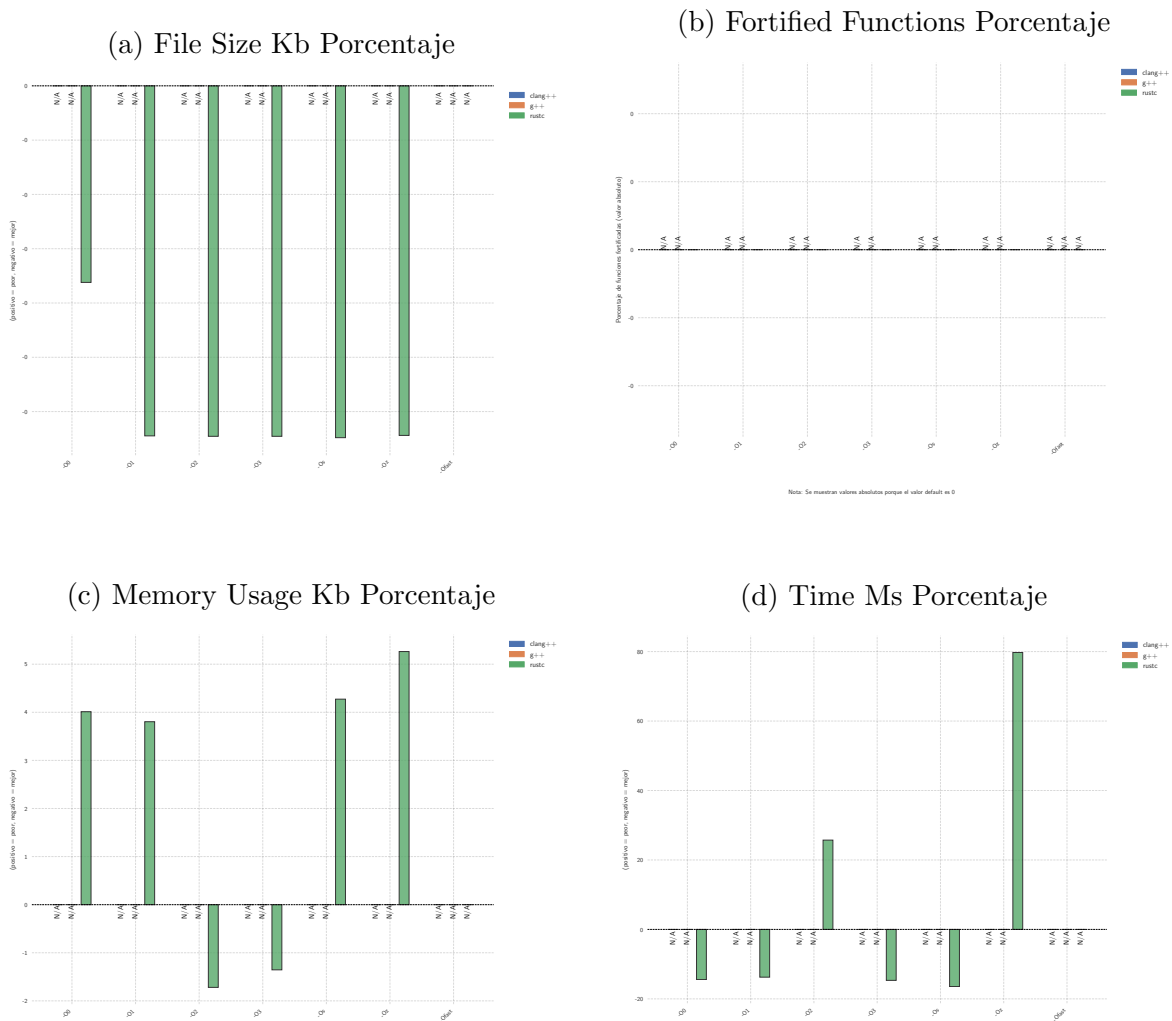
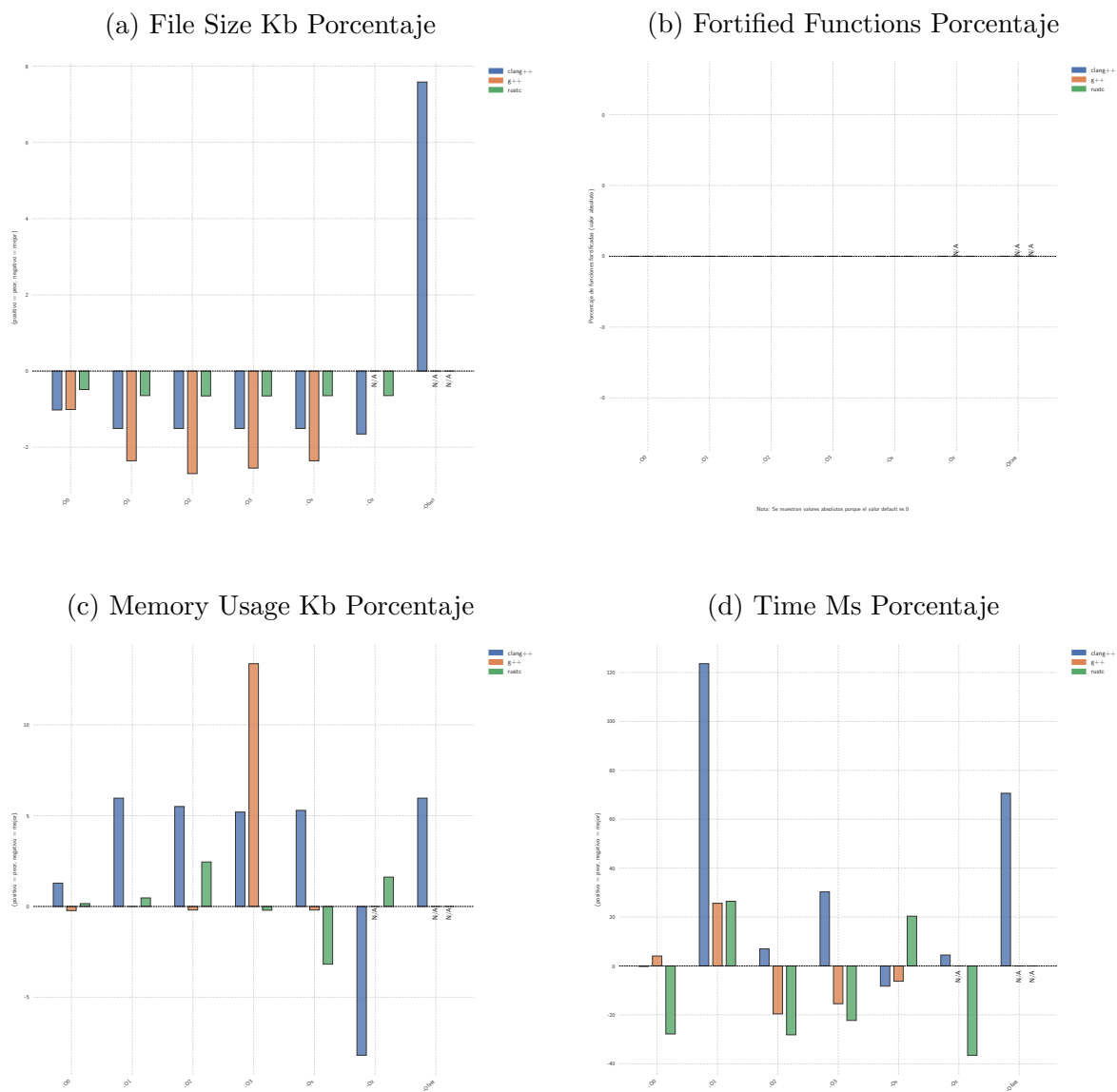


Figura 46: Resultados de compilación con optimización panic



pie / disabled

Figura 47: Resultados para pie (disabled)



pie / enabled

Figura 48: Resultados para pie (enabled)

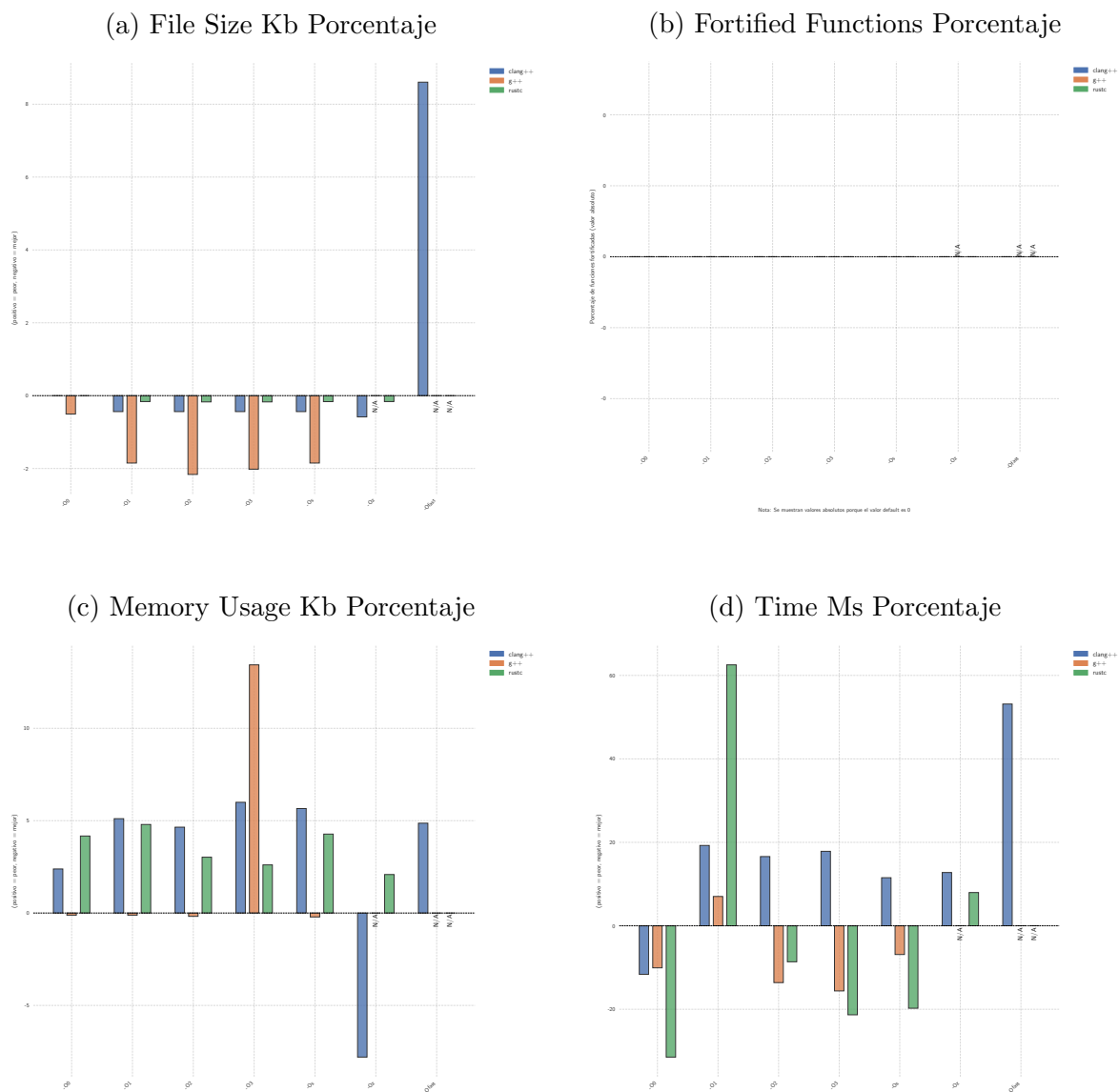
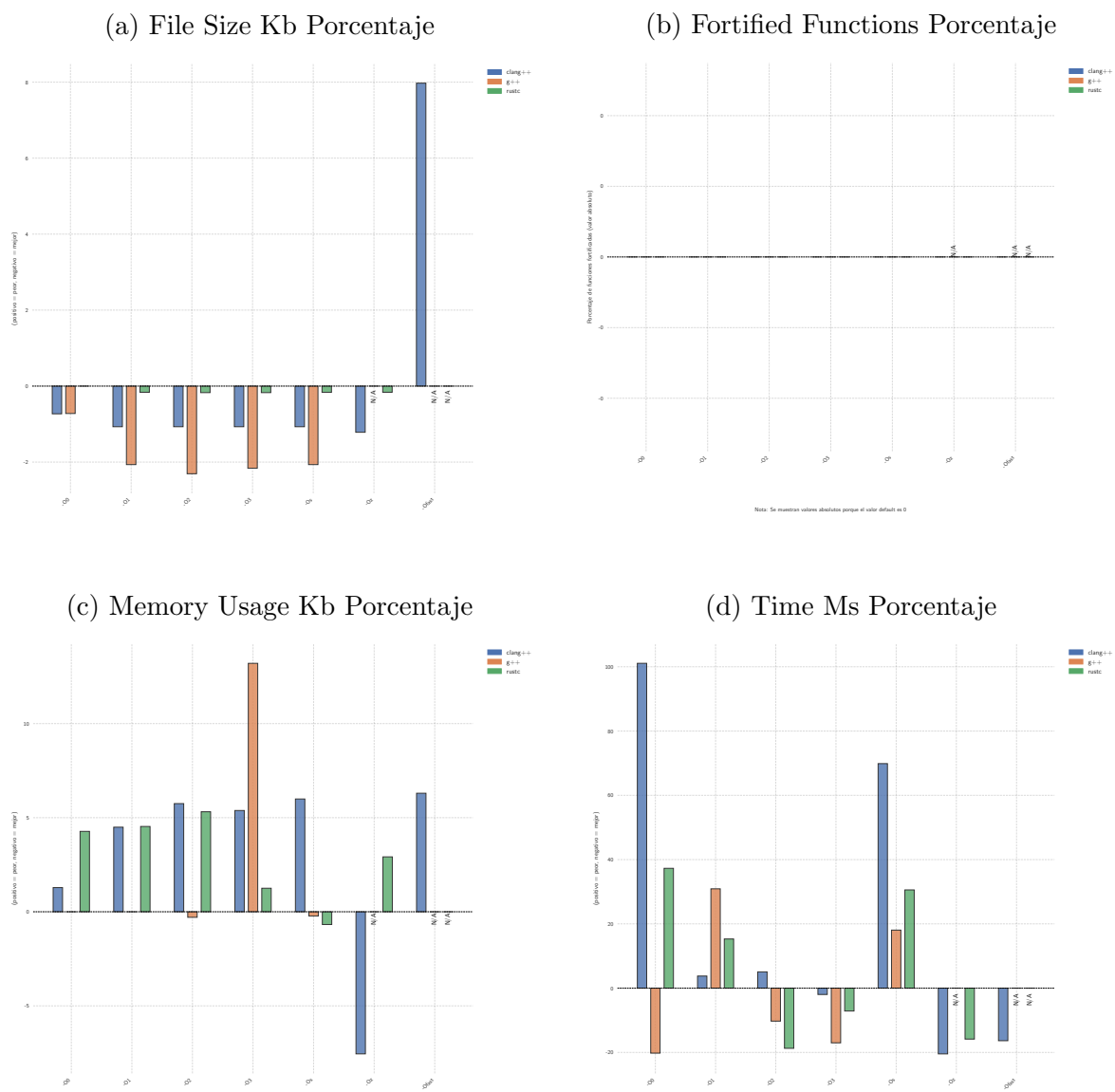


Figura 49: Resultados de compilación con optimización relro



safe-stack

Figura 50: Resultados de compilación con optimización safe-stack

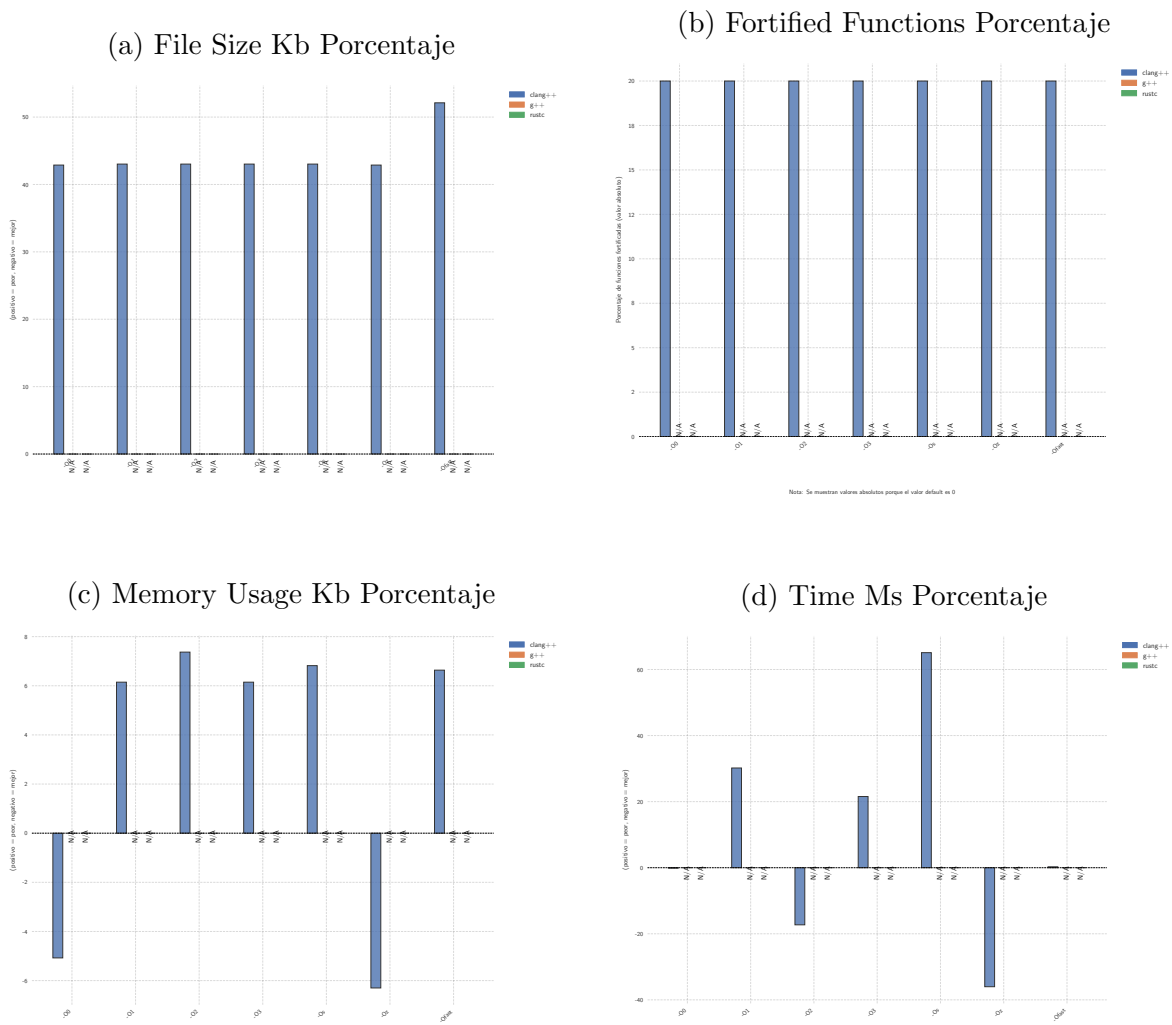
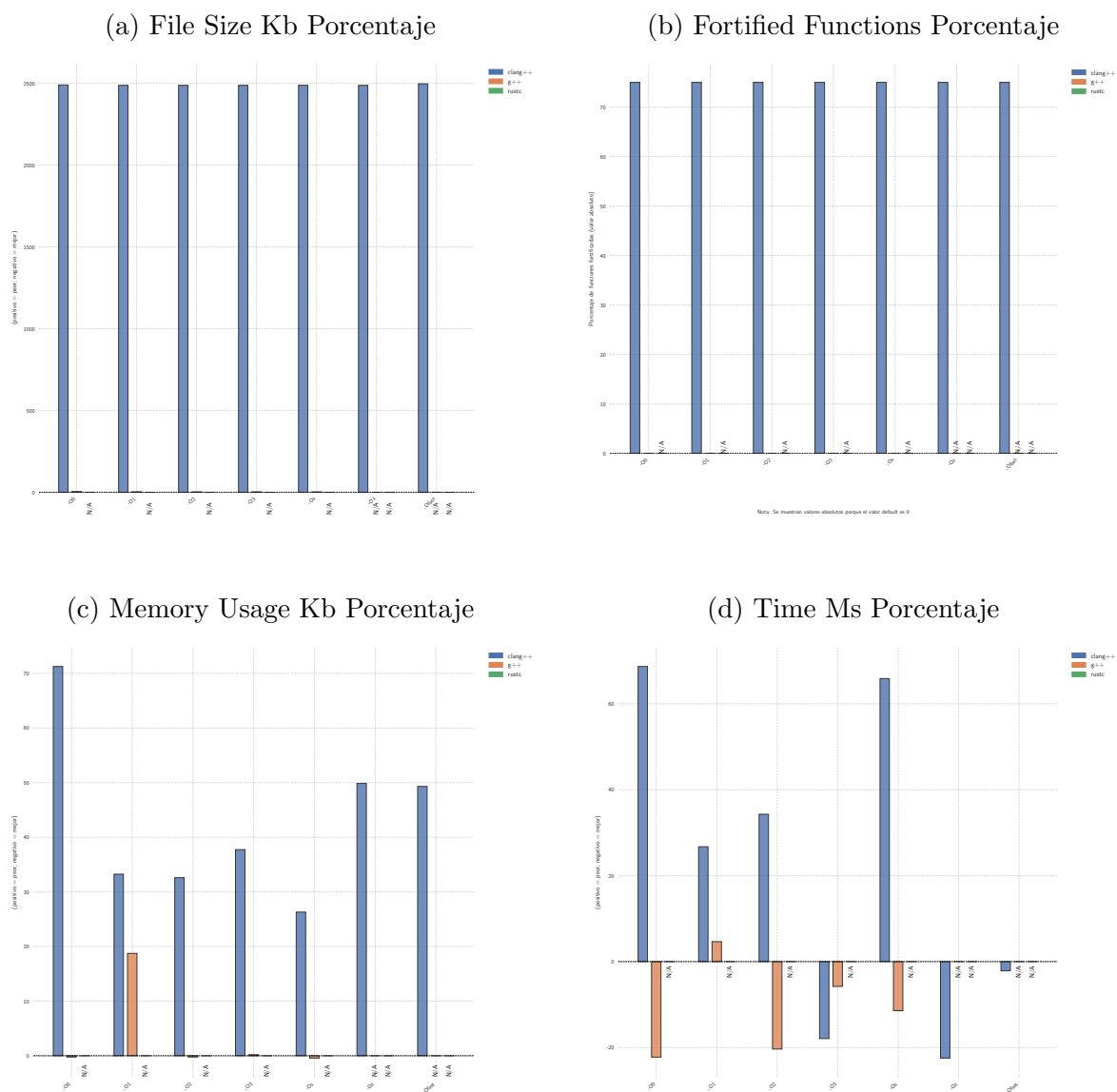
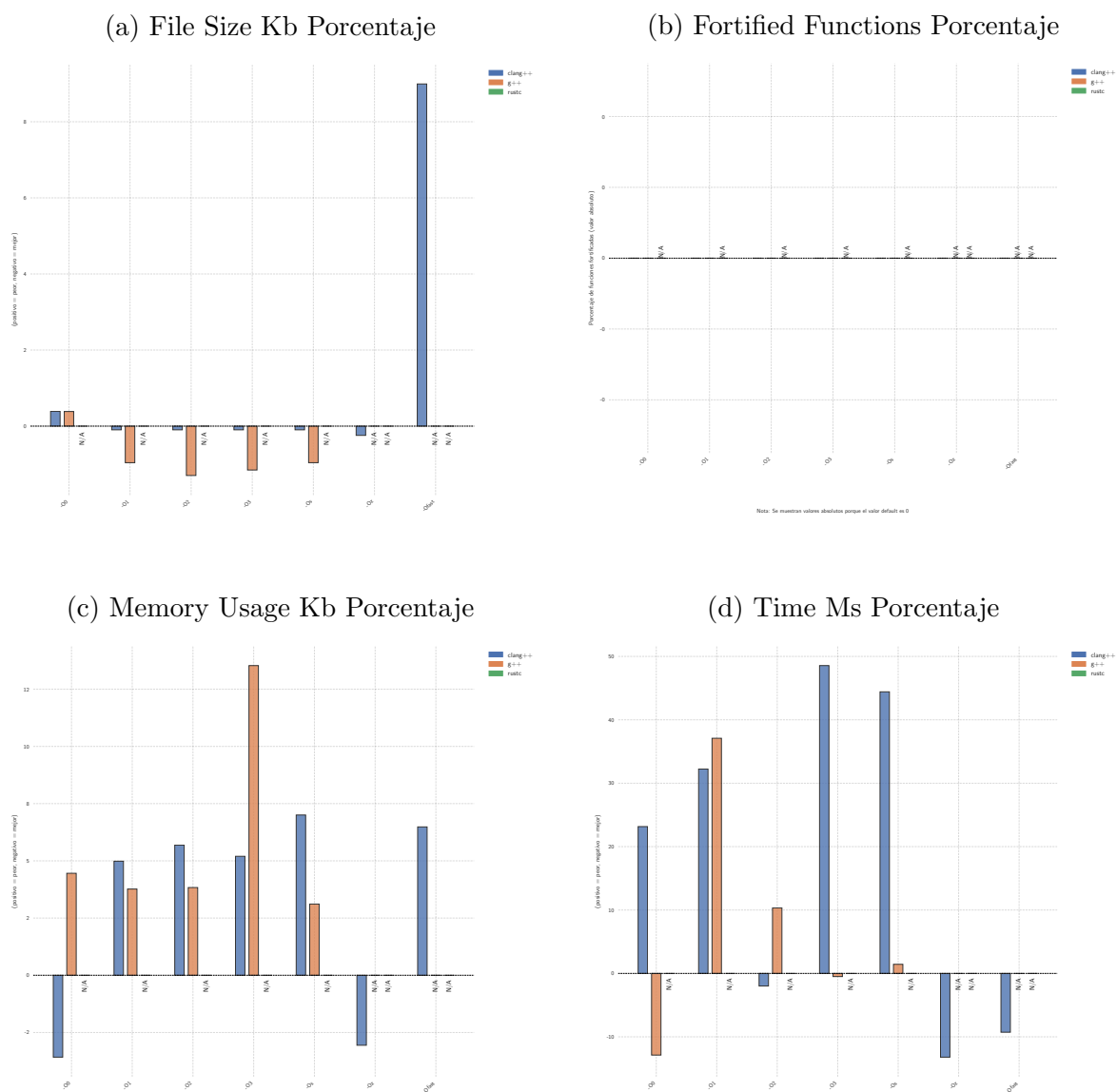


Figura 52: Resultados para sanitizers (undefined)



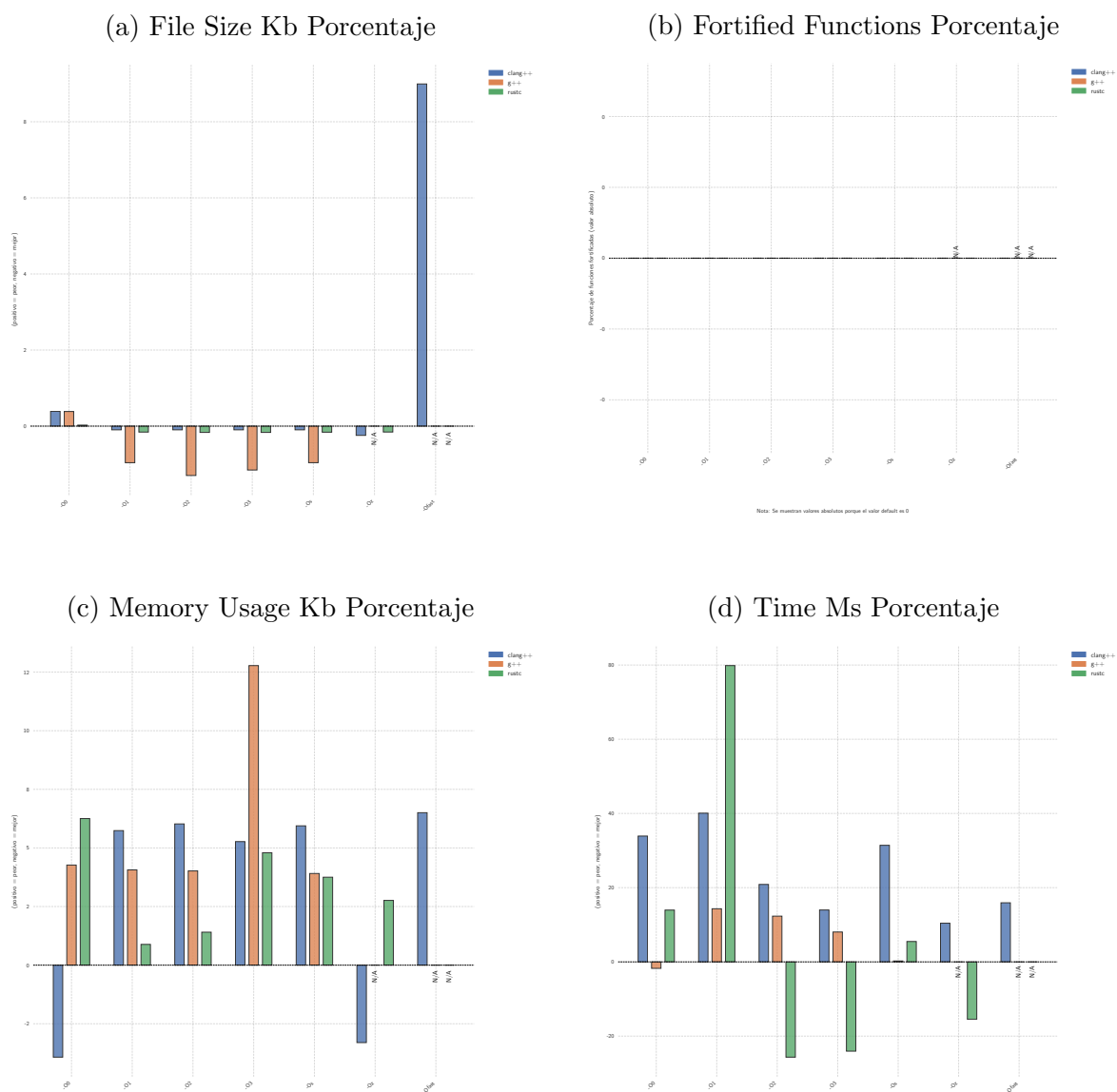
stack-protection / all

Figura 53: Resultados para stack-protection (all)



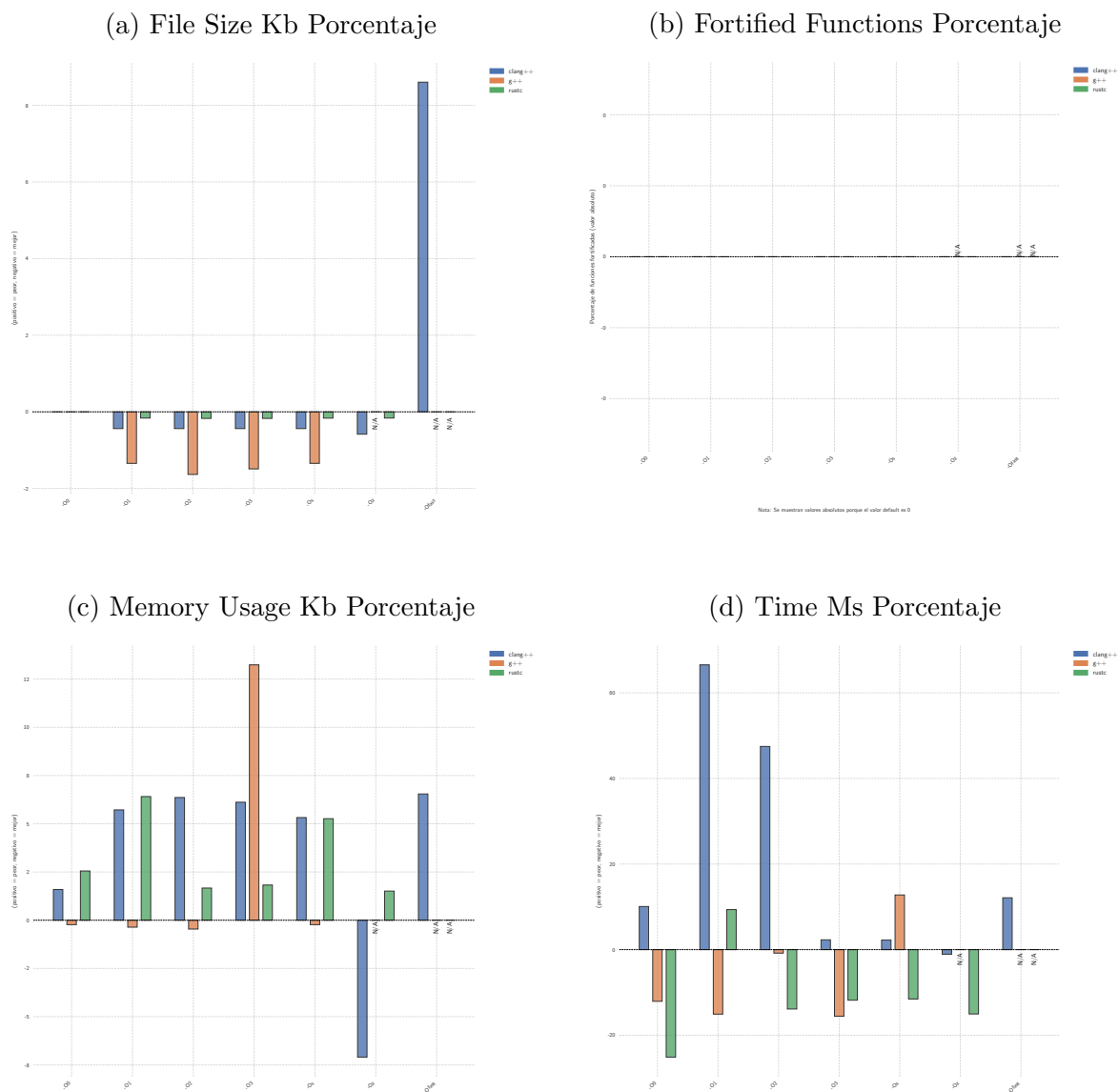
stack-protection / basic

Figura 54: Resultados para stack-protection (basic)



stack-protection / disabled

Figura 55: Resultados para stack-protection (disabled)



stack-protection / strong

Figura 56: Resultados para stack-protection (strong)

